



## Java и многопоточность Квест-курс для продвинутых

### Модуль 1 - Основы многопоточности

#### Урок 1 - Поток и процессы базовые понятия

##### Шаг 01:

###### Миссия 1

Вы — программист, который впервые запускает поток в Java. Какой класс или интерфейс чаще всего используется для создания нового потока?

Подсказка: Вспомните, что в Java есть отдельный класс и интерфейс для работы с потоками.

**Ответ:**

- Класс Thread
- Интерфейс Serializable
- Класс StringBuilder
- Интерфейс Comparable

##### Шаг 02:

###### Миссия 2

Вася пишет чат-сервер и хочет выполнить несколько задач параллельно. Какой метод у объекта Thread запускает новый поток?

Подсказка: Один метод только помечает поток как готовый, а другой — реально выполняет его.

**Ответ:**

- **start()**
- run()
- init()
- execute()

 **Шаг 03:**

### **Миссия 3**

Катя разрабатывает музыкальный плеер. Она хочет, чтобы один поток проигрывал музыку, а другой обрабатывал кнопки пользователя. Какой интерфейс в Java часто используют для передачи логики потоку?

Подсказка: Этот интерфейс содержит метод run().

**Ответ:**

- **Runnable**
- Callable
- Serializable
- Observer

 **Шаг 04:**

### **Миссия 4**

Игорь запускает поток, но вместо метода start() случайно вызывает run(). Что произойдёт?

Подсказка: Подумайте, будет ли создан новый поток или код выполнится в том же.

**Ответ:**

- **Код выполнится в текущем потоке**
- Будет создан новый поток
- Программа завершится с ошибкой
- Метод ничего не сделает

 **Шаг 05:**

### **Миссия 5**

Оля пишет игру и создала поток для обработки анимации. Она хочет сделать паузу в работе потока на 2 секунды. Какой метод поможет это сделать?

Подсказка: Этот метод статический и принимает миллисекунды.

**Ответ:**

- **Thread.sleep(2000)**
- Thread.wait(2000)
- Thread.pause(2000)
- Thread.halt(2000)

 **Шаг 06:**

### **Миссия 6**

Антон изучает разницу между процессами и потоками. Сопоставьте понятие и его характеристику.

Подсказка: Подумайте: процесс — это как отдельная программа, поток — её часть.

**Ответ:**

- **Процесс** — Отдельная программа с собственным адресным пространством
- **Поток** — Лёгкий процесс, выполняющий часть работы программы
- **Многопоточность** — Способность одной программы выполнять несколько задач параллельно
- **Многопроцессность** — Запуск нескольких независимых программ одновременно

## Шаг 07:

### Миссия 7

Сергей пишет сервер и хочет понять различия между методами. Сопоставьте метод Thread и его назначение.

Подсказка: Эти методы выполняют разные функции: запуск, ожидание, имя.

**Ответ:**

- **start()** — Запускает выполнение нового потока
- **run()** — Содержит код, который выполняется потоком
- **join()** — Ожидает завершения указанного потока
- **getName()** — Возвращает имя потока

## Шаг 08:

### Миссия 8

Марина проектирует приложение с чатами. Сопоставьте понятие и реальный пример.

Подсказка: Вспомните: поток — внутри программы, процесс — это отдельная программа.

**Ответ:**

- **Процесс браузера** — Отдельный экземпляр Chrome или Firefox
- **Поток браузера** — Обработка вкладки или рендеринг страницы
- **Процесс компилятора** — Запуск программы javac
- **Поток сервера** — Обслуживание одного клиента

## Шаг 09:

### Миссия 9

Ваня пишет приложение для загрузки файлов. Сопоставьте метод или класс и его назначение.

Подсказка: Все варианты связаны с управлением потоками.

**Ответ:**

- **Thread.sleep()** — Приостанавливает поток на заданное время
- **Thread.interrupt()** — Посылает сигнал прерывания потоку
- **Runnable** — Интерфейс с методом run() для запуска кода в потоке
- **Callable** — Интерфейс с методом call(), возвращающим результат

## Шаг 10:

### Миссия 10

Петя экспериментирует с потоками и хочет понять термины. Сопоставьте термин с его описанием.

Подсказка: Сфокусируйтесь на ключевых характеристиках: ядра, задачи, планировщик.

**Ответ:**

- **CPU core** — Аппаратный элемент, выполняющий инструкции
- **Планировщик ОС** — Компонент, распределяющий потоки между ядрами
- **Задача** — Логическая единица работы программы
- **Многозадачность** — Выполнение нескольких задач одновременно

🔗 **Шаг 11:**

### **Миссия 11**

Саша изучает многопоточность и хочет связать теорию с практикой. Сопоставьте понятие и пример из Java.

Подсказка: Некоторые интерфейсы возвращают результат, другие — нет.

**Ответ:**

- **Runnable** — Запуск задачи без возвращаемого результата
- **Callable** — Запуск задачи с возвращаемым результатом
- **Future** — Хранение результата асинхронной задачи
- **ExecutorService** — Пул потоков для управления задачами

🔗 **Шаг 12:**

### **Миссия 12**

Аня создает веб-приложение. Ей нужно понять жизненный цикл потока. Сопоставьте состояние и его описание.

Подсказка: Поток может быть в разных состояниях: новый, выполняется, ждёт и т. д.

**Ответ:**

- **NEW** — Поток создан, но ещё не запущен
- **RUNNABLE** — Поток готов к выполнению
- **BLOCKED** — Поток ожидает освобождения ресурса
- **TERMINATED** — Поток завершил выполнение

🔗 **Шаг 13:**

### **Миссия 13**

Расположите шаги запуска нового потока в правильном порядке.

Подсказка: Сначала нужно определить задачу, потом создать поток и только потом его запустить.

**Ответ:**

- Реализовать интерфейс Runnable или наследовать Thread
- Создать объект Thread
- Вызвать метод start()
- Выполнение кода в методе run()

🔗 **Шаг 14:**

### **Миссия 14**

Виталий отлаживает многопоточное приложение. Расположите действия в правильном порядке.

Подсказка: Сначала пишем код, затем запускаем, отлаживаем и завершаем.

**Ответ:**

- Написание кода потока
- Запуск потока
- Отладка поведения
- Завершение работы

 **Шаг 15:**

**Миссия 15**

Елена изучает конкурентность. Расположите состояния потока в порядке их типичного прохождения.

Подсказка: Начинается с создания, завершается завершением.

**Ответ:**

- NEW
- RUNNABLE
- TERMINATED

 **Шаг 16:**

**Миссия 16**

Максим хочет синхронизировать задачи. Расположите шаги работы с ExecutorService в правильном порядке.

Подсказка: Сначала создаем сервис, потом отправляем задачи и в конце закрываем.

**Ответ:**

- Создание ExecutorService
- Отправка задач в пул потоков
- Получение результатов через Future
- Завершение работы с shutdown()

 **Шаг 17:**

**Миссия 17**

Завершающее испытание: Расположите шаги, необходимые для корректной работы многопоточного приложения.

Подсказка: Сначала планирование, затем запуск и контроль потоков, и только после этого завершение.

**Ответ:**

- Проектирование многопоточной архитектуры
- Создание потоков
- Запуск и управление потоками
- Завершение потоков и освобождение ресурсов

 **Урок 2 - Жизненный цикл потока**

 **Шаг 01:**

**Миссия 1**

Андрей запускает поток для загрузки данных из сети. Какой метод он должен вызвать, чтобы поток действительно начал выполняться?

Подсказка: Вызов конструктора или метода `run()` сам по себе не запускает новый поток.

**Ответ:**

- **`start()`**
- `run()`
- `execute()`
- `init()`

## Шаг 02:

### Миссия 2

Мария создала поток и вызвала `start()`. Какое состояние у потока сразу после этого вызова?

Подсказка: В момент вызова `start()` поток ещё не выполняет код, но готов к запуску.

**Ответ:**

- **`NEW → RUNNABLE`**
- `RUNNING`
- `BLOCKED`
- `TERMINATED`

## Шаг 03:

### Миссия 3

Игорь пишет чат-приложение. Поток клиента завершил выполнение метода `run()`. В каком состоянии окажется поток?

Подсказка: После завершения работы поток уже нельзя запустить повторно.

**Ответ:**

- **`TERMINATED`**
- `WAITING`
- `RUNNABLE`
- `BLOCKED`

## Шаг 04:

### Миссия 4

Ольга использует метод `sleep(1000)` в потоке. Какое состояние принимает поток во время сна?

Подсказка: В это время поток не активен, но автоматически проснётся после тайм-аута.

**Ответ:**

- **`TIMED_WAITING`**
- `WAITING`
- `RUNNABLE`
- `TERMINATED`

## Шаг 05:

### Миссия 5

Пётр запустил несколько потоков. Один из них вызвал `join()` на другом. Что произойдёт?

Подсказка: `join()` заставляет один поток ждать завершения другого.

**Ответ:**

- **Первый поток приостановится, пока второй не завершится**

- Оба потока выполняются параллельно без ожидания
- `join()` немедленно завершает поток
- `join()` переводит поток в состояние `NEW`

#### 💡 Шаг 06:

##### Миссия 6

Сергей поставил поток в ожидание `notify()`. В каком состоянии поток находится?

Подсказка: Поток ждёт, пока другой поток его разбудит.

**Ответ:**

- **WAITING**
- `TIMED_WAITING`
- `RUNNABLE`
- `TERMINATED`

#### 💡 Шаг 07:

##### Миссия 7

📖 Задача: сопоставьте состояния потока с их описанием.

Подсказка: Вспомните `RUNNABLE` и что происходит после `TERMINATED`.

**Ответ:**

- **NEW** — Поток создан, но ещё не запущен
- **RUNNABLE** — Поток готов к выполнению, но планировщик ещё не выбрал его
- **TERMINATED** — Поток завершил работу и не может быть перезапущен

#### 💡 Шаг 08:

##### Миссия 8

Сопоставьте методы управления потоком с их назначением.

Подсказка: Один метод запускает поток, другой завершает ожидание, третий приостанавливает выполнение на время.

**Ответ:**

- **start()** — Запускает поток
- **sleep()** — Приостанавливает поток на заданное время
- **join()** — Ожидает завершения другого потока
- **notify()** — Пробуждает поток из `WAITING` состояния

#### 💡 Шаг 09:

##### Миссия 9

Сопоставьте состояния ожидания с их особенностями.

Подсказка: В одном случае ожидание ограничено временем, в другом — бесконечно.

**Ответ:**

- **WAITING** — Поток ждёт бесконечно до вызова `notify()`
- **TIMED\_WAITING** — Поток спит или ждёт ограниченное время
- **BLOCKED** — Поток ожидает освобождения монитора
- **TERMINATED** — Поток завершил выполнение

## 🔮 Шаг 10:

### Миссия 10

Сопоставьте ситуации с правильным состоянием потока.

Подсказка: Подумайте, когда поток ожидает ресурса, а когда просто завершился.

**Ответ:**

- Поток вызвал **sleep(5000)** — **TIMED\_WAITING**
- Поток закончил метод **run()** — **TERMINATED**
- Поток ждёт на **synchronized-блоке** — **BLOCKED**
- Поток был создан, но **start()** не вызван — **NEW**

## 🔮 Шаг 11:

### Миссия 11

Сопоставьте методы Object с их назначением при работе с потоками.

Подсказка: Эти методы используются вместе с **synchronized-блоками**.

**Ответ:**

- **wait()** — Переводит поток в ожидание уведомления
- **notify()** — Пробуждает один ожидающий поток
- **notifyAll()** — Пробуждает все ожидающие потоки
- **toString()** — Не связан с многопоточностью, возвращает строковое представление объекта

## 🔮 Шаг 12:

### Миссия 12

Сопоставьте события жизненного цикла потока с их причинами.

Подсказка: Каждое состояние возникает из-за конкретного вызова или завершения кода.

**Ответ:**

- **NEW → RUNNABLE** — Вызван метод **start()**
- **RUNNABLE → RUNNING** — Планировщик выбрал поток для выполнения
- **RUNNING → WAITING** — Вызван метод **wait()**
- **RUNNING → TERMINATED** — Завершение метода **run()**

## 🔮 Шаг 13:

### Миссия 13

Расположите этапы жизненного цикла потока в правильном порядке.

Подсказка: Начните с создания, затем запуск, выполнение и завершение.

**Ответ:**

- **NEW**
- **RUNNABLE**
- **RUNNING**
- **TERMINATED**

## 🔮 Шаг 14:

### Миссия 14

Программист запускает поток, он затем спит, выполняется и завершается. Расположите состояния.

Подсказка: Сон — это `TIMED_WAITING`, а финал всегда `TERMINATED`.

**Ответ:**

- `NEW`
- `RUNNABLE`
- `TIMED_WAITING`
- `RUNNING`
- `TERMINATED`

### Шаг 15:

#### **Миссия 15**

Расположите шаги при использовании `join()`.

Подсказка: Один поток запускается, другой ждёт, и только после завершения первый продолжает.

**Ответ:**

- Поток А вызывает `start()`
- Поток В вызывает `join(A)`
- Поток А завершается
- Поток В продолжает выполнение

### Шаг 16:

#### **Миссия 16**

Расставьте в порядке этапы при использовании `wait()` и `notify()`.

Подсказка: Сначала поток ждёт, потом другой поток его пробуждает.

**Ответ:**

- Поток входит в `synchronized`-блок
- Вызов `wait()` и переход в `WAITING`
- Другой поток вызывает `notify()`
- Первый поток возобновляет выполнение

## Урок 3 - Создание потоков через `Thread` и `Runnable`

### Шаг 01:

#### **Миссия 1**

Программист Оля хочет создать поток, расширив класс `Thread`. Какой метод ей нужно переопределить?

Подсказка: Этот метод вызывается при запуске потока с помощью `start()`.

**Ответ:**

- `run()`
- `start()`
- `execute()`
- `launch()`

## Шаг 02:

### Миссия 2

Дима написал класс, реализующий Runnable. Как он запустит поток?

Подсказка: Runnable сам по себе не имеет метода start().

**Ответ:**

- **Передать объект Runnable в конструктор Thread и вызвать start()**
- Вызвать run() у Runnable напрямую
- Создать Thread и вызвать run()
- Добавить метод start() в Runnable

## Шаг 03:

### Миссия 3

Андрей спорит с коллегой: можно ли один и тот же объект Runnable запустить в нескольких потоках?

Подсказка: Runnable — это всего лишь задача, её можно передавать многим потокам.

**Ответ:**

- **Да, можно** — несколько потоков будут выполнять один и тот же код
- Нет, Runnable можно использовать только один раз
- Да, но только если Runnable наследует Thread
- Нет, потому что это нарушает синтаксис Java

## Шаг 04:

### Миссия 4

Катя написала свой поток и вызвала метод run() напрямую. Что произойдёт?

Подсказка: Подумайте, чем отличается run() от start().

**Ответ:**

- **Метод выполнится в текущем потоке, новый поток не создастся**
- Создастся новый поток
- Будет исключение
- Программа завершится

## Шаг 05:

### Миссия 5

Сергей хочет использовать наследование Thread, но у него класс уже наследует другой. Как лучше поступить?

Подсказка: В Java класс может наследовать только один другой класс.

**Ответ:**

- **Использовать интерфейс Runnable**
- Удалить родительский класс
- Создать анонимный класс Thread
- Превратить класс в абстрактный

## Шаг 06:

### Миссия 6

Сопоставьте способы создания потоков с их особенностями.

Подсказка: Вспомните, чем отличается наследование Thread от реализации Runnable.

**Ответ:**

- **Thread** — Наследование класса, переопределение run()
- **Runnable** — Реализация интерфейса, передача в Thread
- **start()** — Запускает новый поток
- **run()** — Выполняет код в текущем потоке

## Шаг 07:

### Миссия 7

Разработчики спорят, что делать, если нужен общий ресурс для Runnable. Сопоставьте понятие с объяснением.

Подсказка: Runnable может быть общим, но важно помнить о синхронизации.

**Ответ:**

- **Runnable общий объект** — Можно передавать в несколько потоков
- **Проблема** — Доступ к разделяемым данным
- **Решение** — Синхронизация
- **Thread.run()** — Выполняется без запуска нового потока

## Шаг 08:

### Миссия 8

Сопоставьте методы и их описание в контексте многопоточности.

Подсказка: Внимательно отличайте запуск потока от выполнения кода.

**Ответ:**

- **start()** — Создаёт новый поток и вызывает run()
- **run()** — Обычный метод, выполняется в текущем потоке
- **sleep()** — Приостанавливает выполнение текущего потока
- **join()** — Ожидание завершения потока

## Шаг 09:

### Миссия 9

Компания решила провести код-ревью: сопоставьте класс или интерфейс с правильным использованием.

Подсказка: Подумайте, когда удобнее использовать Runnable, а когда Thread.

**Ответ:**

- **Thread** — Подходит, если не нужен другой родительский класс
- **Runnable** — Подходит, если нужно реализовать поведение без ограничения наследования
- **Анонимный класс** — Удобен для быстрых одноразовых задач
- **Лямбда-выражение** — Краткий способ реализовать Runnable

## Шаг 10:

### Миссия 10

Программисты обсуждают: как правильно создать и запустить поток. Сопоставьте фрагменты кода и их результат.

Подсказка: Обратите внимание на вызовы start() и run().

**Ответ:**

- **new Thread(r).start()** — Запускает новый поток
- **new Thread(r).run()** — Выполняется в текущем потоке
- **r.run()** — Тоже выполняется в текущем потоке
- **new Thread().start() без run()** — Поток запустится, но делать нечего

## Шаг 11:

### Миссия 11

Сопоставьте задачи программистов и подход к созданию потока.

Подсказка: Один способ удобен при простых задачах, другой при гибкости.

**Ответ:**

- **Нужно переопределить только поведение потока** — extends Thread
- **Нужна возможность наследовать другой класс** — implements Runnable
- **Нужна компактность кода** — Лямбда для Runnable
- **Хотим повторно использовать задачу** — Один Runnable в нескольких Thread

## Шаг 12:

### Миссия 12

Расставьте шаги создания потока через Runnable в правильном порядке.

Подсказка: Сначала создаём задачу, потом поток, затем запускаем.

**Ответ:**

- Создать класс, реализующий Runnable
- Переопределить метод run()
- Передать объект Runnable в конструктор Thread
- Вызвать start()

## Шаг 13:

### Миссия 13

Поставьте шаги при создании потока через наследование Thread в правильном порядке.

Подсказка: Подумайте, что нужно сделать перед вызовом start().

**Ответ:**

- Создать класс, наследующий Thread
- Переопределить метод run()
- Создать объект класса
- Вызвать start()

## Шаг 14:

### Миссия 14

Упорядочьте действия: Дима написал задачу Runnable, теперь хочет запустить её несколько раз в разных потоках.

Подсказка: Один Runnable можно использовать в нескольких потоках.

**Ответ:**

- Создать объект Runnable
- Создать несколько объектов Thread с одним Runnable

- Вызвать start() у каждого потока
- Ждать завершения с помощью join()

#### Шаг 15:

##### **Миссия 15**

Расставьте шаги запуска анонимного потока в правильном порядке.

Подсказка: Можно сразу передать Runnable в Thread без отдельного класса.

##### **Ответ:**

- Создать Runnable как анонимный класс
- Передать в конструктор Thread
- Вызвать start()
- Выполнение задачи в новом потоке

#### Шаг 16:

##### **Миссия 16**

Программисты используют лямбды для Runnable. Упорядочьте шаги.

Подсказка: Лямбда упрощает создание Runnable.

##### **Ответ:**

- Написать Runnable через лямбду
- Создать объект Thread с Runnable
- Вызвать start()
- Поток начинает выполнять задачу

#### Шаг 17:

##### **Миссия 17**

Расположите шаги для правильного завершения потока после запуска Runnable.

Подсказка: Иногда нужно дождаться завершения работы потока.

##### **Ответ:**

- Создать Runnable
- Создать Thread
- Вызвать start()
- Вызвать join() для ожидания завершения

### Урок 4 - ThreadLocal и его применение

#### Шаг 01:

##### **Миссия 1**

Программист Олег запускает веб-сервис и хочет использовать переменную для каждого пользователя отдельно, даже если они работают в разных потоках. Какой класс ему поможет?

Подсказка: Этот инструмент из JDK позволяет хранить данные в изоляции для каждого потока.

##### **Ответ:**

- **ThreadLocal**
- volatile
- synchronized
- AtomicInteger

## Шаг 02:

### Миссия 2

Мария пишет чат-приложение. Каждый поток обрабатывает сообщения своего пользователя. Что произойдет, если использовать одну и ту же глобальную переменную вместо ThreadLocal?

Подсказка: Подумайте, как будут делиться данными разные потоки при работе с общей переменной.

**Ответ:**

- **Потоки будут перезаписывать данные друг друга**
- Каждый поток сохранит собственные данные без конфликтов
- Переменная будет автоматически копироваться для каждого потока
- Программа завершится с ошибкой компиляции

## Шаг 03:

### Миссия 3

Василий пишет финансовое приложение и хранит курс валюты в ThreadLocal. Что произойдет, если поток завершит работу?

Подсказка: Подумайте, как живет объект в ThreadLocal после смерти потока.

**Ответ:**

- **Данные будут автоматически удалены вместе с потоком**
- Данные останутся доступны всем другим потокам
- Данные переносятся в глобальную область памяти
- ThreadLocal уничтожает JVM

## Шаг 04:

### Миссия 4

Андрей создает ThreadLocal для хранения номера запроса. Как правильно инициализировать значение по умолчанию?

Подсказка: Вспомните про метод, который можно переопределить при создании анонимного подкласса ThreadLocal.

**Ответ:**

- **Переопределить метод initialValue()**
- Использовать synchronized блок
- Присвоить значение через System.setProperty
- Добавить конструктор с параметром

## Шаг 05:

### Миссия 5

Кирилл работает над системой логирования. Какой вызов нужно сделать, чтобы явно удалить значение из ThreadLocal?

Подсказка: Подумайте о методе, который освобождает ресурс вручную.

**Ответ:**

- **remove()**
- clear()
- reset()
- delete()

## Шаг 06:

### Миссия 6

Валя тестирует работу ThreadLocal. Она устанавливает значение в одном потоке и пробует получить его в другом. Что она увидит?

Подсказка: Вспомните: ThreadLocal хранит данные изолированно для каждого потока.

**Ответ:**

- **null, так как данные не передаются между потоками**
- Значение из другого потока
- Ошибку компиляции
- RuntimeException

## Шаг 07:

### Миссия 7

Петр хранит в ThreadLocal объект SimpleDateFormat. Почему это хорошая практика?

Подсказка: Подумайте о том, потокобезопасен ли SimpleDateFormat.

**Ответ:**

- **SimpleDateFormat не потокобезопасен, поэтому каждому потоку нужен свой экземпляр**
- ThreadLocal делает объект глобальным для всех
- SimpleDateFormat всегда потокобезопасен
- ThreadLocal ускоряет работу GC

## Шаг 08:

### Миссия 8

Разработчик Артем случайно оставил значение в ThreadLocal и не вызвал remove(). Что может произойти в долгоживущем пуле потоков?

Подсказка: Вспомните про утечки памяти при повторном использовании потоков.

**Ответ:**

- **Появится утечка памяти**
- Данные будут автоматически очищены при каждом запросе
- ThreadLocal уничтожит объект при GC без проблем
- Ничего страшного, это безопасно

## Шаг 09:

### Миссия 9

Сопоставьте методы ThreadLocal с их назначением.

Подсказка: Вспомните: get, set, remove и initialValue() выполняют разные роли.

**Ответ:**

- **get()** — Получить текущее значение потока
- **set(value)** — Установить новое значение для текущего потока
- **remove()** — Удалить значение для текущего потока
- **initialValue()** — Определить значение по умолчанию

## Шаг 10:

### Миссия 10

Сопоставьте ситуации с тем, как ThreadLocal помогает.

Подсказка: Подумайте о сценариях с форматированием дат, хранением контекста пользователя, транзакциями и случайными числами.

**Ответ:**

- **Хранение SimpleDateFormat** — Каждому потоку нужен свой экземпляр
- **Хранение ID пользователя** — У каждого потока — свой пользовательский контекст
- **Хранение транзакции** — Каждый поток работает в своей транзакции
- **Хранение Random** — Потоки не должны использовать общий генератор

## Шаг 11:

### Миссия 11

Сопоставьте проблему с тем, что произойдет без ThreadLocal.

Подсказка: Общие переменные в потоках часто ведут к непредсказуемым последствиям.

**Ответ:**

- **Общий SimpleDateFormat** — Неправильное форматирование дат
- **Общий объект Random** — Некорректная генерация случайных чисел
- **Общая переменная userId** — Смешение данных пользователей
- **Отсутствие remove()** — Утечка памяти в пуле потоков

## Шаг 12:

### Миссия 12

Сопоставьте понятие с его описанием.

Подсказка: Определите, что означает изоляция потока, что такое утечка и зачем нужен remove().

**Ответ:**

- **Изоляция потока** — Каждый поток хранит свои данные отдельно
- **Утечка памяти** — Объекты остаются в памяти из-за неосвобожденных ссылок
- **Метод remove()** — Очищает значение для потока
- **initialValue()** — Определяет дефолтное значение

## Шаг 13:

### Миссия 13

Расположите шаги работы с ThreadLocal в правильном порядке.

Подсказка: Сначала создается объект, потом задается значение, затем читается, и в конце очищается.

**Ответ:**

- Создать ThreadLocal
- Установить значение через set()
- Прочитать значение через get()
- Удалить значение через remove()

## Шаг 14:

### Миссия 14

Выстройте порядок действий программиста при использовании ThreadLocal для транзакций.  
Подсказка: Сначала транзакция создается, потом сохраняется в ThreadLocal, используется, и наконец очищается.

#### Ответ:

- Создание транзакции
- Сохранение транзакции в ThreadLocal
- Использование транзакции
- Удаление из ThreadLocal

## Шаг 15:

### Миссия 15

Расположите этапы при работе с ThreadLocal и пулом потоков.

Подсказка: Подумайте, почему важно освобождать ресурсы после использования.

#### Ответ:

- Получение потока из пула
- Установка значения в ThreadLocal
- Выполнение задачи
- Удаление значения из ThreadLocal

## Шаг 16:

### Миссия 16

Расположите этапы правильного использования initialValue().

Подсказка: Сначала метод определяется, потом используется потоками, после чего данные читаются и удаляются.

#### Ответ:

- Определение initialValue()
- Создание потока
- Чтение значения через get()
- Удаление значения через remove()

## Модуль 2 - Синхронизация и блокировки

### Урок 1 - Синхронизация с synchronized

## Шаг 01:

### Миссия 1

Андрей пишет программу для чата. Два потока одновременно обращаются к истории сообщений. Что произойдет, если не использовать synchronized ?

Подсказка: Подумайте о том, что может произойти при одновременной записи в общую коллекцию.

#### Ответ:

- **Может возникнуть состояние гонки (race condition)**
- Программа всегда завершится быстрее

- Будет гарантирована корректная работа
- Выполнение перейдет только в один поток

## 🔗 Шаг 02:

### Миссия 2

Марина пишет игру, где несколько потоков начисляют очки игроку. Какой ключевое слово нужно использовать, чтобы избежать некорректных сумм?

Подсказка: В Java есть специальное слово для блокировки доступа к методу или блоку кода.

**Ответ:**

- **synchronized**
- volatile
- static
- transient

## 🔗 Шаг 03:

### Миссия 3

Игорь заметил, что при использовании `synchronized` доступ к методу становится последовательным. Как это влияет на производительность?

Подсказка: Подумайте, что произойдет, если много потоков будут ждать один метод.

**Ответ:**

- **Может снизиться параллелизм, так как потоки ждут доступа**
- Метод всегда выполняется быстрее
- Производительность растет линейно с количеством потоков
- Ошибки синхронизации гарантированы

## 🔗 Шаг 04:

### Миссия 4

Светлана пишет банковское приложение. Метод `transferMoney` переводит средства между счетами. Что нужно сделать, чтобы несколько переводов не испортили данные?

Подсказка: Представьте, что два перевода происходят одновременно на один и тот же счет.

**Ответ:**

- **Сделать метод `transferMoney` `synchronized`**
- Добавить больше логов
- Разбить метод на части без блокировок
- Запустить метод в отдельном процессе

## 🔗 Шаг 05:

### Миссия 5

Вадим использует `synchronized` на нестатическом методе. На чем в таком случае происходит блокировка?

Подсказка: Подумайте о том, что означает ключевое слово для методов экземпляра.

**Ответ:**

- **На объекте экземпляра (`this`)**
- На всем классе
- На JVM

- На случайном объекте

## 🔗 Шаг 06:

### Миссия 6

Анна делает утилиту для отчетов. Метод `generateReport` объявлен как `static synchronized`. На чем происходит блокировка?

Подсказка: Подумайте о разнице между статическим и нестатическим методом.

**Ответ:**

- **На объекте класса (Class)**
- На экземпляре объекта
- На случайной строке
- На JVM

## 🔗 Шаг 07:

### Миссия 7

Никита задумался: можно ли синхронизировать только часть метода, а не весь? Как это сделать?

Подсказка: Подумайте о синхронизированных блоках внутри методов.

**Ответ:**

- **Использовать `synchronized`-блок внутри метода**
- Объявить метод как `volatile`
- Разбить метод на два без синхронизации
- Сделать `return` раньше

## 🔗 Шаг 08:

### Миссия 8

Олег пишет онлайн-магазин. Он использует `synchronized` для управления доступом к корзине. Что произойдет, если два потока вызовут метод одновременно?

Подсказка: Подумайте, как работает монитор объекта.

**Ответ:**

- **Один поток выполнит метод, второй будет ждать**
- Оба выполнят метод параллельно
- Второй поток завершится с ошибкой
- Оба потока заблокируют друг друга навсегда

## 🔗 Шаг 09:

### Миссия 9

Сопоставьте понятия с правильными объяснениями:

Подсказка: Вспомните разницу между статической и нестатической синхронизацией.

**Ответ:**

- **`synchronized` метод экземпляра** — Блокировка на объекте `this`
- **`static synchronized` метод** — Блокировка на объекте класса
- **`synchronized` блок** — Блокировка на переданном объекте
- **монитор** — Механизм, управляющий доступом потоков

## 🔧 Шаг 10:

### Миссия 10

Влад и Катя спорят, как использовать `synchronized`. Помогите им сопоставить ситуации и правильные действия.

Подсказка: Важно выбрать нужный уровень блокировки.

**Ответ:**

- **Общий счетчик для всех пользователей** — `static synchronized` метод
- **Отдельная корзина для каждого пользователя** — `synchronized` метод экземпляра
- **Синхронизация только части кода** — `synchronized` блок
- **Доступ к объекту из разных потоков** — Монитор объекта

## 🔧 Шаг 11:

### Миссия 11

Сопоставьте ошибку и ее возможное последствие.

Подсказка: Подумайте, что может случиться при неправильной синхронизации.

**Ответ:**

- **Отсутствие `synchronized`** — Race condition
- **Чрезмерная синхронизация** — Снижение производительности
- **Неверный объект блокировки** — Непредсказуемое поведение
- **Блокировки в разных порядках** — Deadlock

## 🔧 Шаг 12:

### Миссия 12

Сопоставьте разработчика и его подход к синхронизации.

Подсказка: Вспомните типичные ошибки новичков и опытных программистов.

**Ответ:**

- **Новичок Петя** — Использует `synchronized` везде без разбора
- **Опытный Сергей** — Синхронизирует только критические секции
- **Внимательная Оля** — Следит, чтобы избежать deadlock
- **Торопливый Макс** — Забывает добавить `synchronized`

## 🔧 Шаг 13:

### Миссия 13

Сопоставьте объект и правильный способ синхронизации.

Подсказка: Важно выбрать корректный объект-блокировку.

**Ответ:**

- **Коллекция `List`** — Синхронизированный блок на `list`
- **Класс `ReportGenerator`** — `static synchronized` метод
- **Объект `User`** — `synchronized` метод экземпляра
- **Секция кода с общим ресурсом** — `synchronized (resource)` блок

## 🔧 Шаг 14:

### Миссия 14

Расположите шаги использования `synchronized` правильно:

Подсказка: Сначала определяем общий ресурс, затем выбираем объект блокировки, потом пишем код, и наконец тестируем.

**Ответ:**

- Определить общий ресурс
- Выбрать объект для блокировки
- Написать `synchronized`-блок
- Протестировать работу в многопоточной среде

 **Шаг 15:**

### Миссия 15

Расположите действия программиста при устранении `race condition`.

Подсказка: Сначала нужно обнаружить проблему, потом найти общий ресурс, затем применить `synchronized`, и в конце протестировать.

**Ответ:**

- Обнаружить проблему `race condition`
- Определить общий ресурс
- Добавить `synchronized`-блок
- Протестировать решение

 **Шаг 16:**

### Миссия 16

Поставьте в правильный порядок шаги при использовании `synchronized` в банковской системе.

Подсказка: Важно сначала определить, что синхронизировать, затем выбрать объект блокировки, написать код и проверить тестами.

**Ответ:**

- Определить критическую секцию (переводы)
- Выбрать объект блокировки (счет)
- Написать `synchronized`-блок
- Запустить тесты на многопоточность

## Урок 2 - Блокировки `ReentrantLock`

 **Шаг 01:**

### Миссия 1

Андрей пишет многопоточное приложение и впервые видит `ReentrantLock`. Что является его ключевым преимуществом по сравнению с `synchronized`?

Подсказка: Подумай о гибкости и дополнительных возможностях контроля, которых нет у `synchronized`.

**Ответ:**

- **Возможность попытки захвата блокировки с таймаутом**
- Скорость всегда выше, чем у `synchronized`
- `ReentrantLock` не требует освобождения явно
- Можно использовать только в одном потоке

 **Шаг 02:**

### Миссия 2

Марина решила использовать ReentrantLock вместо synchronized. Как правильно освободить блокировку?

Подсказка: Всегда помни про finally-блок.

**Ответ:**

- **Вызвать unlock() в блоке finally**
- Вызвать close() после использования
- **Ничего не нужно** — освобождается автоматически
- Использовать break для выхода из блока

### 🔮 Шаг 03:

#### Миссия 3

Дима пишет сервис и хочет проверить, свободна ли блокировка, прежде чем захватывать её. Какой метод ReentrantLock ему нужен?

Подсказка: Подумай о неблокирующем вызове.

**Ответ:**

- **tryLock()**
- checkLock()
- lockNow()
- isUnlocked()

### 🔮 Шаг 04:

#### Миссия 4

Оля внедряет ReentrantLock и хочет использовать его в режиме справедливости (fair). Что это значит?

Подсказка: Вспомни очередь потоков и порядок доступа.

**Ответ:**

- **Потоки получают доступ в порядке очереди (FIFO)**
- Блокировка проверяет права доступа пользователя
- Каждый поток получает блокировку случайным образом
- Можно делить блокировку между двумя потоками

### 🔮 Шаг 05:

#### Миссия 5

Сергей использует ReentrantLock и замечает, что один поток может захватывать её несколько раз. Как это работает?

Подсказка: Вспомни, почему название начинается с «Reentrant».

**Ответ:**

- **Один поток может входить в блокировку повторно без блокировки самого себя**
- Поток освобождает блокировку автоматически при повторном захвате
- Только главный поток может повторно захватывать блокировку
- ReentrantLock всегда отклоняет повторные попытки

### 🔮 Шаг 06:

#### Миссия 6

Катя хочет использовать условные переменные (Condition) вместе с ReentrantLock. Для чего они нужны?

Подсказка: Вспомни wait/notify у synchronized.

**Ответ:**

- **Для организации ожидания и уведомления потоков**
- Для автоматического освобождения памяти
- Для логирования состояний блокировки
- Для ограничения числа потоков в приложении

## Шаг 07:

### Миссия 7

Игорь тестирует ReentrantLock и случайно забывает вызвать unlock(). Что произойдёт?

Подсказка: Подумай, кто ещё ждёт блокировку.

**Ответ:**

- **Другие потоки могут заблокироваться навсегда**
- Lock автоматически освободится через 1 секунду
- Поток выбросит NullPointerException
- Lock сам переведётся в fair-режим

## Шаг 08:

### Миссия 8

 Сопоставь методы ReentrantLock с их назначением.

Подсказка: Подумай, какие из них блокирующие, а какие — нет.

**Ответ:**

- **lock()** — Захватывает блокировку, ожидая бесконечно
- **tryLock()** — Проверяет захватить блокировку без ожидания
- **lockInterruptibly()** — Захватывает блокировку с возможностью прерывания
- **unlock()** — Освобождает блокировку

## Шаг 09:

### Миссия 9

 Сопоставь сценарии с правильным выбором метода ReentrantLock.

Подсказка: Обрати внимание на таймауты и прерывания.

**Ответ:**

- **Нужно дождаться блокировки любой ценой** — lock()
- **Нужно проверить доступность без ожидания** — tryLock()
- **Нужно ждать, но иметь возможность прерывания** — lockInterruptibly()
- **Нужно завершить работу с ресурсом** — unlock()

## Шаг 10:

### Миссия 10

 Сопоставь элементы fair-режима с их описанием.

Подсказка: FIFO-очередь играет ключевую роль.

**Ответ:**

- **Fair режим** — Гарантирует порядок доступа потоков (FIFO)
- **Non-fair режим** — Может позволить обгон другим потокам
- **new ReentrantLock(true)** — Создаёт справедливую блокировку
- **new ReentrantLock(false)** — Создаёт блокировку без очереди

#### 🔗 Шаг 11:

##### Миссия 11

📖 Сопоставь использование Condition с его назначением.

Подсказка: Это как wait/notify, только гибче.

**Ответ:**

- **await()** — Ожидание сигнала
- **signal()** — Будит один ожидающий поток
- **signalAll()** — Будит все ожидающие потоки
- **newCondition()** — Создание новой условной переменной

#### 🔗 Шаг 12:

##### Миссия 12

📖 Сопоставь ошибку программиста с её последствием при использовании ReentrantLock.

Подсказка: Представь, что коллега забыл unlock().

**Ответ:**

- **Забыл вызвать unlock()** — Ресурс заблокирован навсегда
- **Не использовал finally** — unlock() может не выполниться при исключении
- **Использовал lock() без условия** — Поток ждёт бесконечно
- **Неправильно использовал tryLock()** — Поток может проскочить без ожидания

#### 🔗 Шаг 13:

##### Миссия 13

Расположи шаги корректного использования ReentrantLock.

Подсказка: Подумай про порядок: lock, try, finally, unlock.

**Ответ:**

- Создать экземпляр ReentrantLock
- Вызвать lock()
- Выполнить критическую секцию
- Вызвать unlock() в finally

#### 🔗 Шаг 14:

##### Миссия 14

Упорядочи действия при использовании Condition.

Подсказка: Сначала lock, потом работа с Condition, в конце unlock.

**Ответ:**

- Вызвать lock()
- Вызвать await()
- Получить сигнал через signal()

- Вызвать `unlock()`

#### 🔧 Шаг 15:

##### Миссия 15

Расставь шаги при работе с `fair ReentrantLock`.

Подсказка: Вспомни конструктор и порядок очереди.

**Ответ:**

- Создать `new ReentrantLock(true)`
- Вызвать `lock()`
- Потоки становятся в очередь
- Освободить через `unlock()`

#### 🔧 Шаг 16:

##### Миссия 16

Расположи шаги обработки исключения при использовании `ReentrantLock`.

Подсказка: `finally` — ключевой момент.

**Ответ:**

- `lock()`
- `try { критическая секция }`
- `catch { обработка ошибки }`
- `finally { unlock() }`

#### 🔧 Шаг 17:

##### Миссия 17

Упорядочи шаги при использовании `tryLock` с таймаутом.

Подсказка: Таймаут указывают перед выполнением критической секции.

**Ответ:**

- Вызвать `tryLock(timeout)`
- **Если `true`** — выполнить критическую секцию
- **Если `false`** — обработать недоступность
- Вызвать `unlock()`

### 📖 Урок 3 - Методы `wait()`, `notify()`, `notifyAll()`

#### 🔧 Шаг 01:

##### Миссия 1

Программист Дима запускает поток и хочет, чтобы он временно остановился до прихода сигнала. Какой метод из списка используется для этого?

Подсказка: Этот метод заставляет поток освободить монитор и ждать, пока его разбудят.

**Ответ:**

- **`wait()`**
- `notify()`
- `notifyAll()`
- `sleep()`

## Шаг 02:

### Миссия 2

Анна пишет чат-программу. Когда одно сообщение приходит, она хочет разбудить только один поток, обрабатывающий новые данные. Какой метод подходит?

Подсказка: Здесь речь идёт о сигнале для конкретного ожидающего потока, а не для всех сразу.

**Ответ:**

- **notify()**
- wait()
- notifyAll()
- yield()

## Шаг 03:

### Миссия 3

В команде Макс запускает сразу 5 потоков, которые ждут данных. Когда данные готовы, нужно разбудить всех сразу. Какой метод выбрать?

Подсказка: Здесь сигнал даётся не одному, а сразу всем ожидающим потокам.

**Ответ:**

- **notifyAll()**
- notify()
- wait()
- sleep()

## Шаг 04:

### Миссия 4

Игорь пытается вызвать метод wait() у объекта, но получает IllegalMonitorStateException . В чём причина?

Подсказка: Метод wait() требует, чтобы поток держал монитор объекта.

**Ответ:**

- **Метод вызван вне синхронизированного блока**
- Метод вызван в главном потоке
- Объект неинициализирован
- У объекта нет метода wait()

## Шаг 05:

### Миссия 5

Марина использует метод sleep() вместо wait() для синхронизации потоков. Что произойдёт?

Подсказка: Разница в том, кто держит монитор объекта.

**Ответ:**

- **Поток уснёт, но не отпустит монитор**
- Поток отпустит монитор
- Произойдёт исключение
- sleep() автоматически вызывает notify()

## 🔗 Шаг 06:

### Миссия 6

📖 Сопоставьте методы синхронизации с их назначением.

Подсказка: Подумайте, какой метод будит один поток, какой все, а какой останавливает.

**Ответ:**

- **wait()** — Останавливает поток и освобождает монитор
- **notify()** — Будит один из ожидающих потоков
- **notifyAll()** — Будит все ожидающие потоки
- **sleep()** — Останавливает поток, но монитор не освобождается

## 🔗 Шаг 07:

### Миссия 7

📖 Соотнесите ситуации с правильным методом.

Подсказка: Вспомните разницу между одиночным и массовым пробуждением.

**Ответ:**

- **Один поток ждёт сигнал о завершении задачи** — notify()
- **Много потоков ждут данные из базы** — notifyAll()
- **Поток временно приостановлен до события** — wait()
- **Поток спит без ожидания сигнала** — sleep()

## 🔗 Шаг 08:

### Миссия 8

📖 Соотнесите метод с типичным сценарием его использования.

Подсказка: Представьте работу офиса: кто-то ждёт сигнал, кто-то сообщает.

**Ответ:**

- **wait()** — Секретарь ждёт звонка телефона
- **notify()** — Менеджер вызывает одного сотрудника в кабинет
- **notifyAll()** — Менеджер собирает всю команду на собрание
- **sleep()** — Сотрудник берёт короткий перерыв

## 🔗 Шаг 09:

### Миссия 9

📖 Соотнесите правильные утверждения о поведении потоков.

Подсказка: Обратите внимание на освобождение монитора и взаимодействие потоков.

**Ответ:**

- **wait()** — Освобождает монитор при ожидании
- **sleep()** — Не освобождает монитор
- **notify()** — Выбирает один поток для пробуждения
- **notifyAll()** — Все ожидающие получают сигнал

## 🔗 Шаг 10:

### Миссия 10

📖 Соотнесите объекты и их поведение при вызове методов.

Подсказка: Всегда помните, что вызовы идут к объекту, а не к классу.

**Ответ:**

- **obj.wait()** — Ожидание сигнала для конкретного объекта
- **obj.notify()** — Сигнал одному потоку, ожидающему этот объект
- **obj.notifyAll()** — Сигнал всем потокам, ожидающим этот объект
- **Thread.sleep()** — Задержка выполнения текущего потока

#### **Шаг 11:**

##### **Миссия 11**

Расположите действия программиста при использовании wait() и notify() в правильном порядке.

Подсказка: Сначала поток ждёт, потом другой поток отправляет сигнал, и только после этого первый продолжает.

**Ответ:**

- Поток входит в синхронизированный блок
- Вызов метода wait()
- Другой поток входит в синхронизированный блок
- Вызов метода notify()
- Первый поток возобновляет работу

#### **Шаг 12:**

##### **Миссия 12**

Расположите этапы корректной работы продюсера и консюмера в порядке.

Подсказка: Продюсер кладёт данные, консюмер их забирает.

**Ответ:**

- Продюсер входит в синхронизированный блок
- Продюсер вызывает notify()
- Консюмер вызывает wait()
- Консюмер получает данные
- Оба выходят из синхронизированного блока

#### **Шаг 13:**

##### **Миссия 13**

Упорядочьте шаги для реализации пула задач с ожиданием.

Подсказка: Сначала добавляются задачи, потом пробуждаются потоки.

**Ответ:**

- Поток-производитель добавляет задачу
- Поток-производитель вызывает notifyAll()
- Потоки-потребители вызывают wait()
- Ожидющие потоки просыпаются
- Задачи выполняются

#### **Шаг 14:**

##### **Миссия 14**

Расположите шаги, чтобы избежать IllegalMonitorStateException .

Подсказка: Всегда сначала нужно захватить монитор.

**Ответ:**

- Вход в синхронизированный блок
- Вызов wait()
- Вызов notify()
- Выход из синхронизированного блока
- Поток продолжает работу

 **Шаг 15:**

**Миссия 15**

Расставьте шаги для организации ожидания ответа от сервера.

Подсказка: Клиент ждёт, пока сервер не ответит.

**Ответ:**

- Клиент входит в синхронизированный блок
- Клиент вызывает wait()
- Сервер входит в синхронизированный блок
- Сервер вызывает notify()
- Клиент получает ответ

 **Шаг 16:**

**Миссия 16**

Расположите шаги работы игровой очереди, где игроки ждут начала партии.

Подсказка: Игра стартует, когда ведущий даёт сигнал.

**Ответ:**

- Игроки входят в синхронизированный блок
- Игроки вызывают wait()
- Ведущий входит в синхронизированный блок
- Ведущий вызывает notifyAll()
- Игра начинается

 **Урок 4 - ReadWriteLock для разделяемых ресурсов**

 **Шаг 01:**

**Миссия 1**

Программист Маша пишет систему логирования. Несколько потоков читают логи, один поток периодически записывает их. Какой инструмент синхронизации лучше всего подойдёт?

Подсказка: Подумайте о том, что операции чтения можно делать параллельно, а запись должна быть эксклюзивной.

**Ответ:**

- **ReadWriteLock**
- ReentrantLock
- synchronized
- AtomicInteger

 **Шаг 02:**

**Миссия 2**

Сергей пишет онлайн-магазин. Несколько клиентов читают каталог товаров, иногда администратор обновляет список. Какой метод `ReadWriteLock` нужно использовать для записи? Подсказка: Этот метод противоположен чтению.

**Ответ:**

- **`writeLock()`**
- `readLock()`
- `lockInterruptibly()`
- `newCondition()`

### Шаг 03:

#### Миссия 3

Андрей оптимизирует систему аналитики. У него 10 потоков читают отчёты и один поток обновляет данные. Что произойдёт, если поток попытается взять `writeLock`, пока есть активные `readLock`?

Подсказка: Запись всегда ждёт, пока читатели освободят доступ.

**Ответ:**

- **Поток-запись будет ждать, пока все читатели отпустят lock**
- Поток-запись выполнится параллельно с читателями
- Будет исключение
- Чтение и запись всегда выполняются одновременно

### Шаг 04:

#### Миссия 4

Катя хочет понять разницу между `ReadWriteLock` и обычным `synchronized`. Что является главным преимуществом `ReadWriteLock`?

Подсказка: Вспомните, что `synchronized` всегда блокирует доступ всем, а `ReadWriteLock` умеет быть более гибким.

**Ответ:**

- **Одновременные чтения возможны**
- Код становится короче
- Работает быстрее всегда
- Не требует освобождения lock

### Шаг 05:

#### Миссия 5

Сопоставьте методы `ReadWriteLock` с их назначением.

Подсказка: Два основных метода — для чтения и записи, остальные связаны с интерфейсами `Lock`.

**Ответ:**

- **`readLock()`** — Возвращает объект `Lock` для операций чтения
- **`writeLock()`** — Возвращает объект `Lock` для операций записи
- **`lock()`** — Метод захвата блокировки у `Lock`
- **`unlock()`** — Метод освобождения блокировки у `Lock`

## Шаг 06:

### Миссия 6

Сопоставьте класс и его назначение в контексте ReadWriteLock.

Подсказка: Вспомните, что ReadWriteLock — это интерфейс, а у него есть конкретная реализация.

**Ответ:**

- **ReadWriteLock** — Интерфейс, определяющий методы для чтения и записи
- **ReentrantReadWriteLock** — Основная реализация интерфейса ReadWriteLock
- **Lock** — Базовый интерфейс для работы с блокировками
- **Condition** — Интерфейс для работы с условиями ожидания

## Шаг 07:

### Миссия 7

Сопоставьте метод с ожидаемым результатом.

Подсказка: У каждого метода Lock есть своя цель: либо захват, либо проверка, либо освобождение.

**Ответ:**

- **tryLock()** — Попытка захватить lock без ожидания
- **lockInterruptibly()** — Захват lock с возможностью прерывания
- **unlock()** — Освобождение lock
- **newCondition()** — Создание Condition для работы с ожиданием

## Шаг 08:

### Миссия 8

Расположите шаги правильного использования readLock в коде.

Подсказка: Всегда помните про finally для освобождения lock.

**Ответ:**

- Вызвать readLock().lock()
- Выполнить операции чтения
- Освободить lock в finally
- Закрыть блок try

## Шаг 09:

### Миссия 9

Расположите шаги правильного использования writeLock в коде.

Подсказка: Логика почти та же, что и у readLock, только используется запись.

**Ответ:**

- Вызвать writeLock().lock()
- Выполнить операции изменения данных
- Освободить lock в finally
- Закрыть блок try

## Шаг 10:

### Миссия 10

Упорядочите действия программиста, когда он работает с `ReentrantReadWriteLock`.

Подсказка: Сначала нужно создать объект, потом получать нужный lock.

**Ответ:**

- Создать объект `ReentrantReadWriteLock`
- Получить `readLock()` или `writeLock()`
- Захватить lock
- Выполнить операции
- Освободить lock

#### Шаг 11:

##### **Миссия 11**

Расположите шаги при одновременном чтении и записи, чтобы избежать блокировок.

Подсказка: Чтение можно выполнять параллельно, но запись должна ждать.

**Ответ:**

- Несколько потоков вызывают `readLock().lock()`
- Выполняются операции чтения
- Все читатели освобождают lock
- Поток записи вызывает `writeLock().lock()`
- Выполняются операции записи

#### Шаг 12:

##### **Миссия 12**

Расположите шаги при реализации кэша с помощью `ReadWriteLock`.

Подсказка: Чаще выполняется чтение, реже — обновление.

**Ответ:**

- Потоки читают данные через `readLock`
- Иногда `writeLock` блокирует всех
- Данные обновляются
- `writeLock` освобождается
- Чтение возобновляется

#### Шаг 13:

##### **Миссия 13**

Расположите шаги при использовании `tryLock` для `writeLock`.

Подсказка: Иногда запись можно пропустить, если lock занят.

**Ответ:**

- Попробовать `writeLock.tryLock()`
- **Если успешно** — обновить данные
- **Если неуспешно** — пропустить запись
- Освободить lock, если захвачен

#### Шаг 14:

##### **Миссия 14**

Расположите шаги при работе с условием на основе `writeLock.newCondition()`.

Подсказка: Condition используется для ожидания и сигналов внутри writeLock.

**Ответ:**

- Вызвать writeLock().lock()
- Создать Condition через newCondition()
- Вызвать await() или signal()
- Выполнить нужные операции
- Освободить lock

🔮 **Шаг 15:**

**Миссия 15**

Иван использует ReentrantReadWriteLock. Он хочет проверить, доступен ли lock прямо сейчас, без ожидания. Какой метод ему стоит использовать?

Подсказка: Этот метод возвращает true или false.

**Ответ:**

- **tryLock()**
- lock()
- lockInterruptibly()
- newCondition()

🔮 **Шаг 16:**

**Миссия 16**

Оля написала код с ReentrantReadWriteLock, но забыла вызвать unlock(). К чему это приведёт?

Подсказка: Если lock не освободить, другие потоки не смогут продолжить работу.

**Ответ:**

- **К блокировке других потоков (deadlock)**
- К автоматическому освобождению lock
- К выбросу исключения
- Ничего не произойдёт

🔮 **Шаг 17:**

**Миссия 17**

Паша хочет использовать ReentrantReadWriteLock вместо synchronized. Какой главный недостаток этого подхода?

Подсказка: В отличие от synchronized, lock нужно освобождать вручную.

**Ответ:**

- **Нужно вручную освобождать lock**
- Нельзя использовать в многопоточности
- Работает только с одним потоком
- Не поддерживает чтение и запись

## 📖 **Урок 5 - StampedLock продвинутая блокировка**

🔮 **Шаг 01:**

**Миссия 1**

Анна впервые использует StampedLock. Какое основное преимущество этого класса перед ReentrantReadWriteLock?

Подсказка: Подумайте о возможности работать без блокировки.

**Ответ:**

- **Поддержка оптимистичного чтения**
- Блокировка на уровне отдельных методов
- Автоматическая синхронизация всех потоков
- Запрещает параллельное чтение

## Шаг 02:

### Миссия 2

 Сопоставьте тип блокировки StampedLock с его описанием

Типы:

**Ответ:**

- Оптимистичная блокировка
- Писательская блокировка
- Читательская блокировка
- Конвертация в писательскую

Описание:

1. Позволяет читать данные без фактической блокировки, но с проверкой версии
2. Позволяет модифицировать ресурс, блокируя других писателей и читателей
3. Разрешает нескольким потокам читать одновременно
4. Преобразует существующую блокировку на чтение в эксклюзивную блокировку для записи

Подсказка: Обратите внимание на различия между чтением и записью.

- **Оптимистичная блокировка** — Позволяет читать данные без фактической блокировки, но с проверкой версии
- **Писательская блокировка** — Позволяет модифицировать ресурс, блокируя других писателей и читателей
- **Читательская блокировка** — Разрешает нескольким потокам читать одновременно
- **Конвертация в писательскую** — Преобразует существующую блокировку на чтение в эксклюзивную блокировку для записи

## Шаг 03:

### Миссия 3

Расположите действия при использовании StampedLock в правильном порядке:

1. Проверка версии после оптимистичного чтения
2. Получение писательской блокировки для обновления данных
3. Чтение данных под читательской блокировкой
4. Преобразование читательской блокировки в писательскую при необходимости

Подсказка: Оптимистичное чтение может завершиться неудачей, поэтому важно проверять версию перед записью.

**Ответ:**

- Чтение данных под читательской блокировкой

- Проверка версии после оптимистичного чтения
- Преобразование читательской блокировки в писательскую при необходимости
- Получение писательской блокировки для обновления данных

#### 🔗 Шаг 04:

##### Миссия 4

Петр хочет обновить данные с минимальной блокировкой других потоков. Какой метод StampedLock ему подходит?

Подсказка: Смотрите на методы, возвращающие штамп для проверки изменений.

**Ответ:**

- **tryOptimisticRead()**
- writeLock()
- readLock()
- unlock()

#### 🔗 Шаг 05:

##### Миссия 5

Катя изучает StampedLock. Что произойдёт, если поток использует оптимистичное чтение, но данные изменились другими потоками?

Подсказка: Оптимистичное чтение требует проверки версии после доступа.

**Ответ:**

- **Необходимо повторить чтение или использовать блокировку**
- Чтение завершится успешно вне зависимости от изменений
- Произойдет автоматическая коррекция данных
- Поток заблокируется навсегда

#### 🔗 Шаг 06:

##### Миссия 6

📖 Сопоставьте методы StampedLock с их назначением

**Ответ:**

- readLock()
- writeLock()
- tryOptimisticRead()
- tryConvertToWriteLock()

1. Получение эксклюзивного доступа для записи
2. Попытка преобразовать текущую блокировку чтения в писательскую
3. Чтение с проверкой версии без фактической блокировки
4. Блокировка для чтения несколькими потоками одновременно

Подсказка: Оптимистичное чтение – это лёгкий метод для чтения без блокировки.

- **readLock()** — Блокировка для чтения несколькими потоками одновременно
- **writeLock()** — Получение эксклюзивного доступа для записи
- **tryOptimisticRead()** — Чтение с проверкой версии без фактической блокировки
- **tryConvertToWriteLock()** — Попытка преобразовать текущую блокировку чтения в писательскую

## Шаг 07:

### Миссия 7

Иван использует StampedLock. Какой метод нужно вызвать, чтобы освободить блокировку, даже если это оптимистичное чтение?

Подсказка: Любая блокировка должна быть освобождена после использования.

**Ответ:**

- **unlock(stamp)**
- releaseLock()
- close()
- endRead()

## Шаг 08:

### Миссия 8

 Сопоставьте сценарий с методом StampedLock

**Ответ:**

- Чтение данных с минимальной задержкой
- Необходимость модифицировать ресурс
- Преобразование читательской блокировки в писательскую
- Обычное чтение несколькими потоками

1. readLock()
2. writeLock()
3. tryConvertToWriteLock()
4. tryOptimisticRead()

Подсказка: Смотрите на скорость и количество блокировок.

- **Чтение данных с минимальной задержкой** — tryOptimisticRead()
- **Необходимость модифицировать ресурс** — writeLock()
- **Преобразование читательской блокировки в писательскую** — tryConvertToWriteLock()
- **Обычное чтение несколькими потоками** — readLock()

## Шаг 09:

### Миссия 9

Когда лучше использовать tryOptimisticRead(), а не readLock()?

Подсказка: Смотрите на количество потенциальных конфликтов чтения/записи.

**Ответ:**

- **Когда конфликты редки и важна производительность**
- Когда нужно заблокировать все потоки на запись
- Когда данные изменяются каждый миллисекунд
- Когда необходимо синхронное чтение для всех потоков

## Шаг 10:

### Миссия 10

 Сопоставьте последствия неправильного использования StampedLock

**Ответ:**

- Неосвобожденная блокировка
  - Оптимистичное чтение без проверки версии
  - Попытка преобразования без блокировки чтения
  - Чтение под writeLock
1. Зависание потоков
  2. Чтение устаревших данных
  3. Неудача преобразования и возможная блокировка
  4. Нет ошибок, но снижается параллелизм

Подсказка: Подумайте о последствиях невыполнения обязательных проверок.

- **Неосвобожденная блокировка** — Зависание потоков
- **Оптимистичное чтение без проверки версии** — Чтение устаревших данных
- **Попытка преобразования без блокировки чтения** — Неудача преобразования и возможная блокировка
- **Чтение под writeLock** — Нет ошибок, но снижается параллелизм

 **Шаг 11:**

**Миссия 11**

Расположите последовательность действий при безопасном использовании StampedLock:

1. Попытка оптимистичного чтения
2. Проверка штампа после чтения
3. При необходимости преобразование в writeLock
4. Обновление данных
5. Освобождение блокировки

Подсказка: Важно проверять версию после оптимистичного чтения, чтобы избежать конфликтов.

**Ответ:**

- Попытка оптимистичного чтения
- Проверка штампа после чтения
- При необходимости преобразование в writeLock
- Обновление данных
- Освобождение блокировки

 **Шаг 12:**

**Миссия 12**

Расположите последовательность действий при безопасном использовании StampedLock:

1. Попытка оптимистичного чтения
2. Проверка штампа после чтения
3. При необходимости преобразование в writeLock
4. Обновление данных
5. Освобождение блокировки

Подсказка: Важно проверять версию после оптимистичного чтения, чтобы избежать конфликтов.

**Ответ:**

- Попытка оптимистичного чтения
- Проверка штампа после чтения
- При необходимости преобразование в writeLock
- Обновление данных
- Освобождение блокировки

### Шаг 13:

#### **Миссия 13**

Анна хочет минимизировать время ожидания потоков при частых чтениях. Какой подход StampedLock ей выбрать?

Подсказка: Подумайте о методе, который не блокирует поток сразу.

**Ответ:**

- **tryOptimisticRead()**
- writeLock()
- readLock()
- tryConvertToWriteLock()

### Шаг 14:

#### **Миссия 14**

 Сопоставьте ситуации с подходящим методом StampedLock:

**Ответ:**

- Чтение большого массива редко изменяемых данных
- Необходимо записать новую информацию
- Чтение с возможностью обновления при конфликте
- Множественное параллельное чтение

1. readLock()
2. writeLock()
3. tryOptimisticRead()
4. tryConvertToWriteLock()

Подсказка: Учитывайте частоту изменений и параллельность.

- **Чтение большого массива редко изменяемых данных** — tryOptimisticRead()
- **Необходимо записать новую информацию** — writeLock()
- **Чтение с возможностью обновления при конфликте** — tryConvertToWriteLock()
- **Множественное параллельное чтение** — readLock()

### Шаг 15:

#### **Миссия 15**

Игорь использует writeLock(), но блокировка зависает. Какое действие нужно предпринять?

Подсказка: Освобождение блокировки важно для предотвращения зависаний.

**Ответ:**

- **Вызвать unlock(stamp) в finally блоке**
- Закрыть программу
- Игнорировать зависание

- Попытаться читать данные параллельно

## Шаг 16:

### Миссия 16

 Сопоставьте методы StampedLock с уровнем блокировки:

**Ответ:**

- readLock()
  - writeLock()
  - tryOptimisticRead()
  - tryConvertToWriteLock()
1. Эксклюзивная блокировка
  2. Лёгкая, без фактической блокировки
  3. Совместная блокировка для чтения
  4. Промежуточная попытка повысить права на запись

Подсказка: Оптимистичное чтение минимизирует блокировки, а writeLock – строгая эксклюзивная.

- **readLock()** — Совместная блокировка для чтения
- **writeLock()** — Эксклюзивная блокировка
- **tryOptimisticRead()** — Лёгкая, без фактической блокировки
- **tryConvertToWriteLock()** — Промежуточная попытка повысить права на запись

## Шаг 17:

### Миссия 17

Расположите этапы безопасного обновления ресурса с StampedLock:

1. Попытка чтения оптимистично
2. Проверка штампа
3. Конвертация в writeLock при необходимости
4. Изменение данных
5. Освобождение блокировки

Подсказка: Проверка штампа после оптимистичного чтения – ключ к безопасности.

**Ответ:**

- Попытка чтения оптимистично
- Проверка штампа
- Конвертация в writeLock при необходимости
- Изменение данных
- Освобождение блокировки

## Шаг 18:

### Миссия 18

При каком условии tryConvertToWriteLock() вернёт 0?

Подсказка: Подумайте о текущем состоянии блокировки чтения и других потоках.

**Ответ:**

- **Если нельзя повысить права на запись, потому что другие потоки читают**
- Если ресурс пуст

- Если штамп равен 1
- Если вызван unlock()

## **Модуль 3 - Конкурентные коллекции и структуры данных**

### **Урок 1 - ConcurrentHashMap и CopyOnWriteArrayList**

#### **Шаг 01:**

##### **Миссия 1**

Алексей пишет многопоточный чат-сервер. Он думает, какую коллекцию выбрать для хранения списка активных пользователей, к которым часто добавляются новые элементы, но почти не выполняются изменения. Что лучше всего использовать?

Подсказка: Если коллекция редко изменяется, но часто читается, копирование при записи может быть оправданным.

**Ответ:**

- **CopyOnWriteArrayList**
- ArrayList
- LinkedList
- Vector

#### **Шаг 02:**

##### **Миссия 2**

Марина пишет сервис статистики, который часто читает данные, но иногда несколько потоков одновременно обновляют значения по ключу. Какую коллекцию стоит выбрать?

Подсказка: Коллекция должна поддерживать одновременное чтение и обновление без блокировки всей структуры.

**Ответ:**

- **ConcurrentHashMap**
- HashMap
- Hashtable
- TreeMap

#### **Шаг 03:**

##### **Миссия 3**

Игорь реализует систему кэширования. Ему важно, чтобы несколько потоков могли безопасно модифицировать карту. Почему HashMap использовать нельзя?

Подсказка: Подумайте, что может произойти при одновременной вставке элементов.

**Ответ:**

- **Она не потокобезопасна и может привести к бесконечному циклу или повреждению данных**
- HashMap слишком медленный
- HashMap не поддерживает null-ключи
- HashMap требует ручной синхронизации ключей

## 🔗 Шаг 04:

### Миссия 4

Ольга тестирует `CopyOnWriteArrayList`. Она заметила, что при добавлении элемента создаётся новая копия массива. Какой главный недостаток такого подхода?

Подсказка: Обратите внимание на затраты при частых модификациях.

**Ответ:**

- **Высокая стоимость при частых изменениях списка**
- Невозможность многопоточного доступа
- Отсутствие поддержки итераторов
- Нельзя хранить null-значения

## 🔗 Шаг 05:

### Миссия 5

Сергей пишет онлайн-игру, где список подключённых игроков почти не меняется, но часто читается. Какое преимущество даст `CopyOnWriteArrayList`?

Подсказка: Вспомните про неизменяемые снимки коллекции при итерации.

**Ответ:**

- **Итераторы не выбросят `ConcurrentModificationException`**
- Добавление будет быстрее, чем в `ArrayList`
- Поддерживает сортировку по умолчанию
- Использует меньше памяти

## 🔗 Шаг 06:

### Миссия 6

Компания разрабатывает аналитический сервис. Какое преимущество `ConcurrentHashMap` перед `Hashtable`?

Подсказка: Подумайте о том, как реализована синхронизация.

**Ответ:**

- **Более высокая масштабируемость за счёт сегментной блокировки**
- Поддержка сортированных ключей
- Возможность хранить null-ключи
- Отсутствие коллизий

## 🔗 Шаг 07:

### Миссия 7

На хакатоне Петя реализует систему логирования. Поток должен быстро добавлять сообщения в общий список. `CopyOnWriteArrayList` подходит?

Подсказка: Подумайте, что будет при частом добавлении.

**Ответ:**

- **Нет, слишком дорого при частых модификациях**
- Да, идеально подходит
- Подойдёт только при использовании итераторов
- Подойдёт только если список пуст

## 🔗 Шаг 08:

### Миссия 8

Анна использует `ConcurrentHashMap` для подсчёта просмотров страниц. Как лучше увеличить значение счётчика по ключу?

Подсказка: Есть методы, которые позволяют атомарно обновлять значения.

**Ответ:**

- **Использовать `compute` или `merge`**
- Просто получить значение, увеличить и снова положить
- Удалить ключ и вставить заново
- Синхронизировать метод вручную

## 🔗 Шаг 09:

### Миссия 9

Сопоставьте коллекцию с её характеристикой:

Подсказка: Вспомните, какая структура лучше при чтении, а какая — при записи.

**Ответ:**

- **`ConcurrentHashMap`** — Поддерживает конкурентные обновления ключей без блокировки всей карты
- **`CopyOnWriteArrayList`** — Оптимален для частого чтения и редких изменений
- **`Hashtable`** — Использует полную синхронизацию и медленнее в многопоточности
- **`ArrayList`** — Не потокобезопасен и требует внешней синхронизации

## 🔗 Шаг 10:

### Миссия 10

Разработчики обсуждают разные способы работы с `ConcurrentHashMap`. Сопоставьте метод с его назначением:

Подсказка: Обратите внимание на атомарность операций.

**Ответ:**

- **`putIfAbsent`** — Добавляет значение, только если ключ отсутствует
- **`computeIfPresent`** — Изменяет значение, если ключ существует
- **`merge`** — Обновляет значение, объединяя его с новым
- **`remove(key, value)`** — Удаляет запись только если совпадают ключ и значение

## 🔗 Шаг 11:

### Миссия 11

Сопоставьте ситуацию с правильной коллекцией:

Подсказка: Думайте о частоте модификаций и безопасности при многопоточности.

**Ответ:**

- **Логи сервера обновляются каждую секунду** — `ConcurrentLinkedQueue`
- **Список подключённых клиентов редко меняется** — `CopyOnWriteArrayList`
- **Кэш статистики с частыми обновлениями** — `ConcurrentHashMap`
- **Таблица настроек, доступная только в одном потоке** — `HashMap`

## Шаг 12:

### Миссия 12

Сопоставьте описание итераторов с коллекцией:

Подсказка: Вспомните, где создаются снимки массива, а где возможны изменения во время обхода.

**Ответ:**

- **CopyOnWriteArrayList** — Итератор работает с копией массива, изменения не видны
- **ConcurrentHashMap** — Итератор слабосогласованный, но допускает конкурентные изменения
- **HashMap** — Может бросить `ConcurrentModificationException`
- **Vector** — Итератор синхронизирован, но менее производителен

## Шаг 13:

### Миссия 13

Сопоставьте задачу с рекомендуемым методом `ConcurrentHashMap`:

Подсказка: У каждого метода есть особый случай применения.

**Ответ:**

- **Получить значение или установить, если его нет** — `computeIfAbsent`
- **Удалить элемент, только если значение совпадает** — `remove(key, value)`
- **Атомарно обновить существующее значение** — `computeIfPresent`
- **Объединить новое и старое значение** — `merge`

## Шаг 14:

### Миссия 14

Расставьте шаги работы `CopyOnWriteArrayList` при добавлении нового элемента:

Подсказка: Сначала копируется массив, затем вносятся изменения.

**Ответ:**

- Создание копии текущего массива
- Добавление нового элемента в копию
- Переназначение ссылки на новый массив
- Предоставление обновлённого массива для чтения

## Шаг 15:

### Миссия 15

Упорядочьте шаги при использовании `computeIfAbsent` в `ConcurrentHashMap`:

Подсказка: Метод проверяет ключ, и если его нет — создаёт значение.

**Ответ:**

- Проверка наличия ключа
- Вызов функции для создания значения
- Сохранение значения в карту
- Возврат найденного или созданного значения

## Шаг 16:

### Миссия 16

Расставьте шаги, как поток получает доступ к ConcurrentHashMap при чтении:

Подсказка: Чтение обычно не блокируется.

**Ответ:**

- Определение сегмента по хэшу ключа
- Доступ к соответствующему сегменту
- Поиск пары ключ-значение
- Возврат найденного значения

## Урок 2 - BlockingQueue и использование producer-consumer

### Шаг 01:

#### Миссия 1

Андрей пишет программу для очереди заказов в кафе. Какую коллекцию лучше выбрать для организации очереди с блокировкой потоков?

Подсказка: Подумайте о структурах из пакета java.util.concurrent, которые умеют ждать, если очередь пуста или переполнена.

**Ответ:**

- **BlockingQueue**
- ArrayDeque
- LinkedList
- PriorityQueue

### Шаг 02:

#### Миссия 2

Марина реализует схему producer-consumer для обработки изображений. Какой метод BlockingQueue заставит поток ждать, если очередь пуста?

Подсказка: Вспомните, что метод возвращает элемент, блокируясь до его появления.

**Ответ:**

- **take()**
- poll()
- remove()
- peek()

### Шаг 03:

#### Миссия 3

Сергей пишет систему очереди задач. Какой метод BlockingQueue блокирует поток при попытке вставить элемент в полную очередь?

Подсказка: Этот метод похож на add, но работает с блокировкой.

**Ответ:**

- **put()**
- offer()
- add()
- push()

## Шаг 04:

### Миссия 4

Игорь моделирует склад с фиксированным количеством ячеек. Какую реализацию `BlockingQueue` лучше использовать?

Подсказка: Выберите очередь с ограниченной ёмкостью и блокировкой при заполнении.

**Ответ:**

- **`ArrayBlockingQueue`**
- `ConcurrentLinkedQueue`
- `LinkedList`
- `PriorityBlockingQueue`

## Шаг 05:

### Миссия 5

Анна проектирует чат, где сообщения должны обрабатываться в порядке приоритета. Какую очередь стоит использовать?

Подсказка: Приоритетная очередь может работать и в многопоточной среде.

**Ответ:**

- **`PriorityBlockingQueue`**
- `ArrayBlockingQueue`
- `LinkedBlockingQueue`
- `DelayQueue`

## Шаг 06:

### Миссия 6

Виктор делает систему доставки. Задачи должны выполняться строго по порядку. Какая очередь подойдёт?

Подсказка: Подумайте о реализации с двусторонней связью и возможностью большой ёмкости.

**Ответ:**

- **`LinkedBlockingQueue`**
- `ArrayBlockingQueue`
- `PriorityBlockingQueue`
- `ConcurrentLinkedQueue`

## Шаг 07:

### Миссия 7

Саша тестирует `producer-consumer`. Что произойдёт, если `consumer` вызовет `take()` на пустой очереди?

Подсказка: Подумайте, как реализована блокировка при отсутствии элементов.

**Ответ:**

- **Поток будет заблокирован, пока элемент не появится**
- Метод вернёт `null`
- Метод выбросит исключение
- Будет возвращён последний элемент

## 🔗 Шаг 08:

### Миссия 8

Катя создаёт систему задержанной отправки писем. Какая очередь ей подойдёт?

Подсказка: Есть очередь, элементы которой становятся доступными через заданное время.

**Ответ:**

- **DelayQueue**
- PriorityBlockingQueue
- LinkedBlockingQueue
- ArrayBlockingQueue

## 🔗 Шаг 09:

### Миссия 9

Сопоставьте реализацию BlockingQueue с её особенностью:

Подсказка: Вспомните ключевые различия между видами очередей.

**Ответ:**

- **ArrayBlockingQueue** — Очередь фиксированного размера
- **LinkedBlockingQueue** — Может быть ограниченной или неограниченной
- **PriorityBlockingQueue** — Элементы извлекаются по приоритету
- **DelayQueue** — Элементы становятся доступными после тайм-аута

## 🔗 Шаг 10:

### Миссия 10

Сопоставьте метод BlockingQueue с его поведением:

Подсказка: Обратите внимание на отличие блокирующих и неблокирующих методов.

**Ответ:**

- **put()** — Блокирует поток при полной очереди
- **offer()** — Возвращает false, если очередь заполнена
- **take()** — Блокирует поток при пустой очереди
- **poll()** — Возвращает null, если очередь пуста

## 🔗 Шаг 11:

### Миссия 11

Сопоставьте ситуацию с правильной очередью:

Подсказка: Думайте о приоритетах, ограничениях и задержках.

**Ответ:**

- **Обработка писем с задержкой** — DelayQueue
- **Очередь заказов с ограничением по размеру** — ArrayBlockingQueue
- **Очередь задач без фиксированного лимита** — LinkedBlockingQueue
- **Очередь сообщений по уровню важности** — PriorityBlockingQueue

## 🔗 Шаг 12:

### Миссия 12

Сопоставьте элемент модели producer-consumer с его ролью:

Подсказка: Вспомните, кто кладёт задачи, а кто забирает.

**Ответ:**

- **Producer** — Создаёт и помещает элементы в очередь
- **Consumer** — Извлекает и обрабатывает элементы
- **BlockingQueue** — Обеспечивает синхронизацию потоков
- **Thread** — Выполняет действия producer или consumer

### 🔮 Шаг 13:

#### Миссия 13

Сопоставьте метод с его особенностью при таймауте:

Подсказка: Вспомните, какие методы имеют перегрузку с таймаутами.

**Ответ:**

- **offer(E e, long timeout, TimeUnit unit)** — Ждёт освобождения места до истечения времени
- **poll(long timeout, TimeUnit unit)** — Ждёт появления элемента до таймаута
- **put(E e)** — Блокируется без ограничения по времени при заполнении
- **take()** — Блокируется без ограничения по времени при пустой очереди

### 🔮 Шаг 14:

#### Миссия 14

Сопоставьте класс с его типичной сферой применения:

Подсказка: Каждая очередь лучше всего подходит для определённого сценария.

**Ответ:**

- **ArrayBlockingQueue** — Очередь фиксированного размера для ограниченных ресурсов
- **LinkedBlockingQueue** — Большое количество задач без строгого лимита
- **DelayQueue** — Задачи, доступные только через время
- **SynchronousQueue** — Непосредственная передача задачи от producer к consumer

### 🔮 Шаг 15:

#### Миссия 15

Расставьте шаги работы producer-consumer:

Подсказка: Сначала producer кладёт элемент, потом consumer его обрабатывает.

**Ответ:**

- Producer создаёт задачу
- Producer кладёт задачу в очередь
- Consumer извлекает задачу
- Consumer обрабатывает задачу

### 🔮 Шаг 16:

#### Миссия 16

Расставьте шаги при использовании put() и take():

Подсказка: Эти методы блокируют потоки при необходимости.

**Ответ:**

- Producer вызывает put()

- **Если очередь заполнена** — producer блокируется
- Consumer вызывает take()
- **Если очередь пуста** — consumer блокируется

#### 💡 Шаг 17:

##### Миссия 17

Упорядочьте этапы при работе DelayQueue:

Подсказка: Сначала элемент создаётся с задержкой, затем извлекается только после её окончания.

**Ответ:**

- Producer создаёт элемент с указанной задержкой
- Элемент помещается в DelayQueue
- Consumer ждёт окончания задержки
- Consumer извлекает элемент для обработки

### 📖 Урок 3 - DelayQueue и PriorityBlockingQueue

#### 💡 Шаг 01:

##### Миссия 1

Программист Лена решила организовать очередь задач с задержкой. Какую коллекцию в Java она должна использовать?

Подсказка: Эта очередь хранит элементы с отложенным временем доступа.

**Ответ:**

- **DelayQueue**
- PriorityBlockingQueue
- ConcurrentLinkedQueue
- ArrayBlockingQueue

#### 💡 Шаг 02:

##### Миссия 2

Саша хочет отложить выполнение уведомлений на 10 секунд. Как должен вести себя метод take() в DelayQueue?

Подсказка: Представьте будильник: звонит не сразу, а через заданное время.

**Ответ:**

- **Блокирует поток до тех пор, пока задержка элемента не истечёт**
- Возвращает элемент сразу после вызова
- Выбрасывает исключение, если элемент ещё недоступен
- Всегда возвращает null

#### 💡 Шаг 03:

##### Миссия 3

Тимур написал класс, который кладёт в DelayQueue задачи с разным временем выполнения. Как очередь определяет, какой элемент доступен первым?

Подсказка: Здесь решает не порядок добавления, а время готовности.

**Ответ:**

- **Элемент с истекшей задержкой становится доступным первым**
- Элемент, добавленный раньше
- Элемент с самым большим временем ожидания
- Случайный элемент

🔗 **Шаг 04:**

#### **Миссия 4**

Вася применил `PriorityBlockingQueue` для обработки заявок. Но при одновременной вставке нескольких элементов с одинаковым приоритетом — что произойдет?

Подсказка: Вспомните, что порядок равных элементов может не гарантироваться.

**Ответ:**

- **Элементы будут извлекаться в неопределённом порядке**
- Сначала всегда тот, который добавлен раньше
- Сначала всегда тот, который добавлен позже
- Очередь откажется хранить равные элементы

🔗 **Шаг 05:**

#### **Миссия 5**

Сопоставьте коллекцию с её ключевой особенностью.

Подсказка: Вспомните: одна коллекция ориентирована на время задержки, а другая — на приоритет.

**Ответ:**

- **`DelayQueue`** — Элементы доступны только после истечения задержки
- **`PriorityBlockingQueue`** — Элементы упорядочены по приоритету
- **`ConcurrentLinkedQueue`** — Очередь без блокировок
- **`ArrayBlockingQueue`** — Ограниченная очередь с фиксированным размером

🔗 **Шаг 06:**

#### **Миссия 6**

Сопоставьте сценарий и подходящую коллекцию.

Подсказка: Если нужно задерживать сообщения — это одна очередь, если сортировать по важности — другая.

**Ответ:**

- **Отложенные уведомления** — `DelayQueue`
- **Сортировка задач по приоритету** — `PriorityBlockingQueue`
- **Передача сообщений без сортировки** — `ConcurrentLinkedQueue`
- **Ограничение размера очереди** — `ArrayBlockingQueue`

🔗 **Шаг 07:**

#### **Миссия 7**

Сопоставьте класс с тем, что он должен реализовывать для работы в `DelayQueue`.

Подсказка: Подумайте, что нужно для вычисления времени готовности и порядка.

**Ответ:**

- **Delayed** — Интерфейс, который указывает задержку элемента
- **compareTo()** — Метод, определяющий порядок элементов
- **getDelay()** — Метод, вычисляющий оставшуюся задержку
- **Runnable** — Интерфейс для выполнения задачи

## 🔗 Шаг 08:

### Миссия 8

Сопоставьте вызов метода с его действием в `PriorityBlockingQueue`.

Подсказка: Подумайте о различии между `offer` и `put`.

**Ответ:**

- **offer(e)** — Добавляет элемент, возвращает `true/false`
- **put(e)** — Всегда добавляет элемент, ожидая при необходимости
- **poll()** — Извлекает элемент или возвращает `null`
- **peek()** — Возвращает элемент без удаления

## 🔗 Шаг 09:

### Миссия 9

Сопоставьте задачу и корректную реализацию очереди.

Подсказка: Обратите внимание на цель: приоритет или задержка.

**Ответ:**

- **Очередь email-уведомлений через 5 минут** — `DelayQueue`
- **Очередь заказов по срочности** — `PriorityBlockingQueue`
- **Кэш с FIFO** — `ArrayBlockingQueue`
- **Асинхронная обработка событий** — `ConcurrentLinkedQueue`

## 🔗 Шаг 10:

### Миссия 10

Сопоставьте понятие с описанием в контексте `DelayQueue`.

Подсказка: В `DelayQueue` ключевое — это время ожидания и готовность элемента.

**Ответ:**

- **getDelay()** — Возвращает время до доступности
- **compareTo()** — Определяет порядок при одинаковой задержке
- **take()** — Ждёт элемент до готовности
- **offer()** — Кладёт элемент с указанной задержкой

## 🔗 Шаг 11:

### Миссия 11

Расположите шаги для создания задачи с `DelayQueue`:

🔗 Подсказка:

**Ответ:**

- Сначала создаём класс.
- Реализуем нужный интерфейс.
- Реализуем методы для задержки.

- Добавляем объект в очередь.
- Создать класс задачи
- Реализовать интерфейс Delayed
- Реализовать методы getDelay и compareTo
- Добавить объект в DelayQueue

## 🔗 Шаг 12:

### Миссия 12

Расположите шаги работы PriorityQueue:

#### 🔗 Подсказка:

#### Ответ:

- Добавляем элементы.
- Элементы сортируются по приоритету.
- Извлекаем минимальный по приоритету.
- Продолжаем обработку.
- Добавление элементов
- Сортировка по приоритету
- Извлечение минимального
- Обработка задачи

## 🔗 Шаг 13:

### Миссия 13

Упорядочьте шаги использования DelayQueue для рассылки писем:

#### 🔗 Подсказка:

#### Ответ:

- Определите задачу с временем задержки.
- Добавьте её в DelayQueue.
- Подождите, пока она станет доступна.
- Извлеките и выполните.
- Определение задачи с задержкой
- Добавление в DelayQueue
- Ожидание доступности
- Извлечение и выполнение

## 🔗 Шаг 14:

### Миссия 14

Расставьте шаги для обработки заявок в PriorityQueue:

#### 🔗 Подсказка:

#### Ответ:

- Приоритет играет ключевую роль.
- Добавляем задачи.
- Извлекаем самую важную.
- Выполняем её.
- Добавление задач

- Упорядочивание по приоритету
- Извлечение самой важной
- Выполнение

## Урок 4 - ConcurrentSkipListMap и ConcurrentSkipListSet

### Шаг 01:

#### Миссия 1

Алексей разрабатывает систему логирования и хочет использовать коллекцию, которая автоматически поддерживает сортировку ключей в многопоточной среде. Какой класс ему подойдёт?

Подсказка: Подумайте о коллекциях, которые реализуют NavigableMap и работают безопасно с несколькими потоками.

**Ответ:**

- **ConcurrentSkipListMap**
- HashMap
- TreeMap
- ConcurrentHashMap

### Шаг 02:

#### Миссия 2

Марина хранит уникальные идентификаторы задач и хочет, чтобы они всегда были отсортированы. Какой класс стоит выбрать?

Подсказка: Здесь нужен аналог TreeSet, но потокобезопасный.

**Ответ:**

- **ConcurrentSkipListSet**
- HashSet
- CopyOnWriteArraySet
- LinkedHashSet

### Шаг 03:

#### Миссия 3

Игорь пишет сервис рейтингов и заметил, что при высоких нагрузках TreeMap даёт ошибки из-за многопоточности. Какое решение правильное?

Подсказка: Подумайте, какая коллекция является потокобезопасным аналогом TreeMap.

**Ответ:**

- **Заменить TreeMap на ConcurrentSkipListMap**
- Обернуть TreeMap в Collections.synchronizedMap
- Перейти на HashMap
- Использовать CopyOnWriteArrayList

### Шаг 04:

#### Миссия 4

Светлана сортирует строки по алфавиту и использует ConcurrentSkipListSet. Что произойдет, если она добавит дубликат?

Подсказка: Вспомните свойства Set.

**Ответ:**

- **Элемент не будет добавлен**
- Элемент добавится дважды
- Выбросится `ConcurrentModificationException`
- Коллекция перестанет быть отсортированной

#### 🔗 Шаг 05:

##### Миссия 5

Дмитрий хочет использовать `ConcurrentSkipListMap` и получить наименьший ключ. Какой метод выбрать?

Подсказка: Этот метод возвращает минимальный ключ или `null`, если карта пуста.

**Ответ:**

- **`firstKey()`**
- `peekFirst()`
- `lowest()`
- `headKey()`

#### 🔗 Шаг 06:

##### Миссия 6

Сопоставьте классы с их основными свойствами.

Подсказка: Думайте о различиях между `Map` и `Set`.

**Ответ:**

- **`ConcurrentSkipListMap`** — Хранит пары ключ-значение в отсортированном порядке
- **`ConcurrentSkipListSet`** — Хранит только уникальные элементы в отсортированном порядке
- **`TreeMap`** — Не потокобезопасная структура с сортировкой по ключам
- **`HashSet`** — Хранит уникальные элементы, но порядок не гарантирован

#### 🔗 Шаг 07:

##### Миссия 7

Свяжите методы `ConcurrentSkipListMap` с их назначением.

Подсказка: Эти методы связаны с получением крайних элементов.

**Ответ:**

- **`firstKey()`** — Возвращает минимальный ключ
- **`lastKey()`** — Возвращает максимальный ключ
- **`headMap(K)`** — Возвращает отображение с ключами меньше заданного
- **`tailMap(K)`** — Возвращает отображение с ключами больше или равными заданному

#### 🔗 Шаг 08:

##### Миссия 8

Свяжите коллекцию с примером использования.

Подсказка: Смотрите, где нужна сортировка и потокобезопасность.

**Ответ:**

- **`ConcurrentSkipListSet`** — Список активных пользователей, отсортированный по алфавиту

- **ConcurrentSkipListMap** — Кэш с упорядоченными ключами для быстрой навигации
- **CopyOnWriteArrayList** — Чтение данных при редких изменениях
- **ConcurrentLinkedQueue** — Организация очереди задач

## 🔮 Шаг 09:

### Миссия 9

Сопоставьте термин с правильным описанием.

Подсказка: Вспомните свойства Skip List.

**Ответ:**

- **Skip List** — Структура данных, обеспечивающая логарифмическое время поиска
- **ConcurrentSkipListMap** — Использует Skip List для реализации Map
- **ConcurrentSkipListSet** — Использует Skip List для реализации Set
- **TreeMap** — Реализован через красно-чёрное дерево

## 🔮 Шаг 10:

### Миссия 10

Соотнесите разработчика с задачей, где применим ConcurrentSkipListMap.

Подсказка: Обратите внимание на сортировку и многопоточность.

**Ответ:**

- **Анна** — Пишет чат, где нужно хранить сообщения в хронологическом порядке
- **Борис** — Создает структуру для поиска товаров по алфавиту
- **Виктор** — Разрабатывает систему учёта уникальных токенов
- **Галина** — Строит очередь задач без сортировки

## 🔮 Шаг 11:

### Миссия 11

Сопоставьте операции ConcurrentSkipListSet с их результатами.

Подсказка: Эти методы работают аналогично TreeSet, но потокобезопасны.

**Ответ:**

- **add(E)** — Добавляет элемент, если его ещё нет
- **remove(E)** — Удаляет элемент, если он существует
- **contains(E)** — Проверяет наличие элемента
- **iterator()** — Возвращает итератор по элементам в порядке возрастания

## 🔮 Шаг 12:

### Миссия 12

Свяжите метод ConcurrentSkipListMap с ситуацией из практики.

Подсказка: Подумайте, когда нужен диапазон ключей.

**Ответ:**

- **subMap(from,to)** — Получение заказов в заданном диапазоне дат
- **headMap(to)** — Все события до указанного времени
- **tailMap(from)** — Все уведомления, начиная с определённого времени
- **size()** — Подсчёт количества элементов

## Шаг 13:

### Миссия 13

Расставьте шаги использования ConcurrentSkipListMap в правильном порядке.

Подсказка: Начинаем с создания объекта и заканчиваем чтением данных.

#### Ответ:

- Создать объект ConcurrentSkipListMap
- Добавить пары ключ-значение
- Получить поддиапазон с помощью subMap
- Пройтись по элементам с помощью итератора

## Шаг 14:

### Миссия 14

Расположите операции ConcurrentSkipListSet в порядке их типичного использования.

Подсказка: Сначала наполнение, потом проверки и обход.

#### Ответ:

- Создание объекта ConcurrentSkipListSet
- Добавление элементов
- Проверка contains
- Обход с помощью итератора

## Шаг 15:

### Миссия 15

Упорядочьте шаги для извлечения минимального ключа и его удаления в ConcurrentSkipListMap.

Подсказка: Сначала читаем ключ, потом работаем с ним.

#### Ответ:

- Вызвать firstKey()
- Получить значение по ключу
- Удалить запись из карты
- Использовать данные

## Шаг 16:

### Миссия 16

Расположите действия при использовании subMap() в правильной последовательности.

Подсказка: Обратите внимание: сначала диапазон, потом работа с ним.

#### Ответ:

- Создать ConcurrentSkipListMap
- Добавить данные
- Вызвать subMap(from,to)
- Итерироваться по результатам

## Шаг 17:

### Миссия 17

Упорядочьте шаги при работе с ConcurrentSkipListSet для поиска минимального элемента.

Подсказка: Используйте методы `first()` и итерацию.

**Ответ:**

- Создать `ConcurrentSkipListSet`
- Добавить элементы
- Вызвать `first()`
- Использовать найденный элемент

## **Модуль 4 - Executor Framework**

### **Урок 1 - ThreadPoolExecutor и его настройки**

#### **Шаг 01:**

##### **Миссия 1**

Вы работаете над сервером для онлайн-игры. Вам нужно быстро обрабатывать тысячи коротких задач. Какой пул потоков лучше всего подойдёт?

Подсказка: Вспомните, какой пул создаёт фиксированное количество потоков, которые переиспользуются.

**Ответ:**

- **FixedThreadPool**
- `CachedThreadPool`
- `SingleThreadExecutor`
- `ForkJoinPool`

#### **Шаг 02:**

##### **Миссия 2**

Программист Аня пишет Telegram-бота и хочет, чтобы задачи выполнялись последовательно, без параллельности. Какой пул выбрать?

Подсказка: Есть пул, у которого всегда один поток.

**Ответ:**

- `CachedThreadPool`
- **SingleThreadExecutor**
- `WorkStealingPool`
- `ScheduledThreadPool`

#### **Шаг 03:**

##### **Миссия 3**

Ивану нужно обрабатывать задачи разной длительности, и он боится, что потоки будут простаивать. Какой пул потоков создаёт новые потоки при необходимости и переиспользует их?

Подсказка: Этот пул не ограничен фиксированным числом потоков, но может быть риск переполнения.

**Ответ:**

- `FixedThreadPool`
- **CachedThreadPool**
- `ScheduledThreadPool`
- `SingleThreadExecutor`

## Шаг 04:

### Миссия 4

Команда разрабатывает приложение для умного дома. Нужно периодически выполнять задачи (например, проверять температуру каждые 10 секунд). Какой пул выбрать?

Подсказка: Существует пул, поддерживающий задачи с задержкой и периодичностью.

**Ответ:**

- `WorkStealingPool`
- **`ScheduledThreadPool`**
- `FixedThreadPool`
- `CachedThreadPool`

## Шаг 05:

### Миссия 5

Петя хочет тонко настраивать количество потоков, очередь и обработчик отклонённых задач. Какой класс ему подойдёт?

Подсказка: Этот класс — основа всех фабричных методов `Executor`.

**Ответ:**

- **`ThreadPoolExecutor`**
- `Executors`
- `FutureTask`
- `CompletionService`

## Шаг 06:

### Миссия 6

Виктор настраивает `ThreadPoolExecutor`. Какой параметр определяет минимальное количество потоков, которые всегда будут активны?

Подсказка: Этот параметр остаётся даже без задач.

**Ответ:**

- **`corePoolSize`**
- `maximumPoolSize`
- `keepAliveTime`
- `workQueue`

## Шаг 07:

### Миссия 7

Разработчик Сергей заметил, что его `ThreadPoolExecutor` иногда создаёт больше потоков, чем он ожидал. Какой параметр ограничивает максимальное количество потоков?

Подсказка: Этот параметр — верхняя граница для пула.

**Ответ:**

- `corePoolSize`
- **`maximumPoolSize`**
- `keepAliveTime`
- `RejectedExecutionHandler`

## 🔗 Шаг 08:

### Миссия 8

Сопоставьте параметры `ThreadPoolExecutor` с их назначением.

Подсказка: Один параметр отвечает за ядро потоков, другой за максимум, третий — за таймаут, четвёртый — за очередь.

**Ответ:**

- **`corePoolSize`** — Минимальное количество активных потоков
- **`maximumPoolSize`** — Максимальное количество потоков
- **`keepAliveTime`** — Время ожидания перед завершением лишних потоков
- **`workQueue`** — Очередь для хранения задач

## 🔗 Шаг 09:

### Миссия 9

Андрей пишет сервис бронирования билетов. Сопоставьте тип очереди с её характеристикой.

Подсказка: `LinkedBlockingQueue` — почти безразмерна, `SynchronousQueue` — без хранения, `ArrayBlockingQueue` — фиксирована.

**Ответ:**

- **`LinkedBlockingQueue`** — Очередь с почти неограниченной ёмкостью
- **`ArrayBlockingQueue`** — Очередь фиксированного размера
- **`SynchronousQueue`** — Очередь без хранения, каждая задача передаётся напрямую потоку
- **`PriorityBlockingQueue`** — Очередь с упорядочиванием задач по приоритету

## 🔗 Шаг 10:

### Миссия 10

Сопоставьте `RejectedExecutionHandler` с его поведением.

Подсказка: Каждый обработчик решает по-своему, что делать с отклонённой задачей.

**Ответ:**

- **`AbortPolicy`** — Выбрасывает `RejectedExecutionException`
- **`CallerRunsPolicy`** — Задача выполняется в вызывающем потоке
- **`DiscardPolicy`** — Задача просто отбрасывается без ошибки
- **`DiscardOldestPolicy`** — Удаляется самая старая задача из очереди

## 🔗 Шаг 11:

### Миссия 11

Программист Маша тестирует разные виды `ExecutorService`. Сопоставьте метод `Executors` с создаваемым пулом.

Подсказка: Вспомните методы: `newFixedThreadPool`, `newCachedThreadPool`, `newSingleThreadExecutor`, `newScheduledThreadPool`.

**Ответ:**

- **`newFixedThreadPool`** — Пул фиксированного размера
- **`newCachedThreadPool`** — Пул с динамическим созданием потоков
- **`newSingleThreadExecutor`** — Пул из одного потока
- **`newScheduledThreadPool`** — Пул с поддержкой периодических задач

## 🔗 Шаг 12:

### Миссия 12

Сопоставьте метод `ThreadPoolExecutor` с его назначением.

Подсказка: `shutdown()` и `shutdownNow()` отличаются, `awaitTermination` ждёт завершения, `execute()` запускает задачу.

**Ответ:**

- **`execute()`** — Передача задачи на выполнение
- **`shutdown()`** — Останавливает приём новых задач, но завершает старые
- **`shutdownNow()`** — Прерывает все активные задачи и очищает очередь
- **`awaitTermination()`** — Ожидание завершения работы пула

## 🔗 Шаг 13:

### Миссия 13

Сопоставьте ситуации с подходящим пулом потоков.

Подсказка: Для долгих задач подходит фиксированный пул, для коротких — кэшируемый, для последовательных — один поток, для периодических — планировщик.

**Ответ:**

- **Обработка тысяч коротких задач** — `CachedThreadPool`
- **Сервер с ограниченным числом ресурсов** — `FixedThreadPool`
- **Последовательная обработка файлов** — `SingleThreadExecutor`
- **Запуск задачи каждые 5 секунд** — `ScheduledThreadPool`

## 🔗 Шаг 14:

### Миссия 14

Расположите шаги создания `ThreadPoolExecutor` в правильном порядке.

Подсказка: Сначала задаём параметры, затем создаём объект, потом отправляем задачи, а в конце завершаем работу.

**Ответ:**

- Определить `corePoolSize`, `maximumPoolSize` и очередь
- Создать объект `ThreadPoolExecutor`
- Отправить задачи на выполнение
- Вызвать `shutdown()` для завершения

## 🔗 Шаг 15:

### Миссия 15

Расположите действия `RejectedExecutionHandler.AbortPolicy` в порядке их выполнения.

Подсказка: Сначала задача поступает, очередь заполнена, нет потоков, и только после этого выбрасывается исключение.

**Ответ:**

- Задача поступает в `Executor`
- Очередь задач заполнена
- Нет свободных потоков для обработки
- Бросается `RejectedExecutionException`

## Шаг 16:

### Миссия 16

Разработчик настраивает ThreadPoolExecutor. Расположите действия при завершении работы в правильном порядке.

Подсказка: Сначала вызываем shutdown(), потом ждём завершения, и если долго — используем shutdownNow().

#### Ответ:

- Вызвать shutdown()
- Вызвать awaitTermination()
- **Если время истекло** — вызвать shutdownNow()
- Завершение работы пула

## Урок 2 - Fixed, Cached и Scheduled Thread Pools

## Шаг 01:

### Миссия 1

Стас пишет сервис для анализа логов. Он хочет, чтобы одновременно обрабатывалось фиксированное число задач, без роста числа потоков. Какой пул выбрать?

Подсказка: Вспомните пул, где количество потоков строго задано при создании.

#### Ответ:

- **FixedThreadPool**
- CachedThreadPool
- ScheduledThreadPool
- SingleThreadExecutor

## Шаг 02:

### Миссия 2

Аня пишет бота, которому нужно быстро реагировать на непредсказуемое количество входящих сообщений. Какой пул потоков ей подойдёт?

Подсказка: Этот пул создаёт новые потоки при нагрузке и уничтожает их при простое.

#### Ответ:

- FixedThreadPool
- **CachedThreadPool**
- ScheduledThreadPool
- WorkStealingPool

## Шаг 03:

### Миссия 3

Маша пишет систему умного освещения. Лампы должны включаться каждые 10 секунд. Какой пул лучше использовать?

Подсказка: Подумайте о пуле, который поддерживает задачи с задержкой и повторением.

#### Ответ:

- FixedThreadPool
- CachedThreadPool
- **ScheduledThreadPool**

- `SingleThreadExecutor`

#### 🔮 Шаг 04:

##### Миссия 4

Иван заметил, что его `CachedThreadPool` создаёт слишком много потоков. В чём опасность?

Подсказка: Вспомните, что `CachedThreadPool` не ограничивает количество потоков.

**Ответ:**

- Потоки никогда не завершаются
- **Можно быстро исчерпать ресурсы системы**
- Задачи всегда выполняются последовательно
- Невозможно выполнять задачи по расписанию

#### 🔮 Шаг 05:

##### Миссия 5

Олег пишет задачу для проверки статуса сервера каждые 30 секунд. Какой метод `ScheduledExecutorService` подойдёт?

Подсказка: Один метод запускает с задержкой, а другой — периодически.

**Ответ:**

- `schedule()`
- **`scheduleAtFixedRate()`**
- `submit()`
- `invokeAll()`

#### 🔮 Шаг 06:

##### Миссия 6

Лена хочет, чтобы задачи выполнялись через равные интервалы, независимо от времени выполнения предыдущей. Какой метод использовать?

Подсказка: Разница между `scheduleAtFixedRate` и `scheduleWithFixedDelay` в том, когда стартует следующая задача.

**Ответ:**

- **`scheduleAtFixedRate()`**
- `scheduleWithFixedDelay()`
- `schedule()`
- `submit()`

#### 🔮 Шаг 07:

##### Миссия 7

Сопоставьте тип пула с его характеристикой.

Подсказка: Один фиксированный, другой гибкий, третий с поддержкой расписания.

**Ответ:**

- **`FixedThreadPool`** — Фиксированное число потоков
- **`CachedThreadPool`** — Динамическое создание потоков по мере необходимости
- **`ScheduledThreadPool`** — Выполнение задач по расписанию
- **`SingleThreadExecutor`** — Последовательное выполнение задач одним потоком

## 🔧 Шаг 08:

### Миссия 8

Сопоставьте метод `ScheduledExecutorService` с его назначением.

Подсказка: `schedule` — разово, `scheduleAtFixedRate` — равные интервалы, `scheduleWithFixedDelay` — пауза после выполнения.

**Ответ:**

- **`schedule()`** — Выполнение задачи один раз после задержки
- **`scheduleAtFixedRate()`** — Запуск задачи через равные промежутки времени
- **`scheduleWithFixedDelay()`** — Запуск задачи с паузой после завершения предыдущей
- **`submit()`** — Обычная передача задачи на выполнение

## 🔧 Шаг 09:

### Миссия 9

Сопоставьте ситуацию с правильным пулом.

Подсказка: Подумайте, что лучше для коротких, долгих и регулярных задач.

**Ответ:**

- **Сервис с постоянной высокой нагрузкой** — `FixedThreadPool`
- **Всплески краткосрочных задач** — `CachedThreadPool`
- **Регулярные задачи мониторинга** — `ScheduledThreadPool`
- **Нужна строго последовательная обработка** — `SingleThreadExecutor`

## 🔧 Шаг 10:

### Миссия 10

Сопоставьте метод `Executors` с создаваемым пулом.

Подсказка: Все методы начинаются с `new...()`

**Ответ:**

- **`newFixedThreadPool()`** — Фиксированный пул потоков
- **`newCachedThreadPool()`** — Пул с динамическим ростом потоков
- **`newScheduledThreadPool()`** — Пул для планирования задач
- **`newSingleThreadExecutor()`** — Один поток для последовательного выполнения

## 🔧 Шаг 11:

### Миссия 11

Сопоставьте название метода с его поведением.

Подсказка: `shutdown` завершает приём задач, `shutdownNow` — прерывает сразу, `isShutdown` — проверка статуса.

**Ответ:**

- **`shutdown()`** — Останавливает приём новых задач, завершает старые
- **`shutdownNow()`** — Прерывает все задачи и очищает очередь
- **`isShutdown()`** — Возвращает `true`, если вызван `shutdown`
- **`awaitTermination()`** — Ждёт завершения всех задач

## 🔧 Шаг 12:

### Миссия 12

Сопоставьте разработчиков с правильным пулом потоков.

Подсказка: Подумайте, что лучше подходит: фиксированный, гибкий или расписание.

**Ответ:**

- **Иван запускает систему мониторинга сервера** — `ScheduledThreadPool`
- **Аня обрабатывает внезапные пики нагрузки** — `CachedThreadPool`
- **Стас обрабатывает логи в параллельных потоках** — `FixedThreadPool`
- **Маша запускает скрипт только одним потоком** — `SingleThreadExecutor`

### Шаг 13:

#### **Миссия 13**

Расположите шаги работы `FixedThreadPool` в правильном порядке.

Подсказка: Сначала задачи попадают в очередь, потом распределяются потокам, выполняются и завершаются.

**Ответ:**

- Задачи поступают в очередь
- Потоки берут задачи из очереди
- Задачи выполняются
- Результаты возвращаются пользователю

### Шаг 14:

#### **Миссия 15**

Расположите действия при использовании `CachedThreadPool`.

Подсказка: Вспомните: создаётся поток, выполняется задача, поток сохраняется, завершается при простое.

**Ответ:**

- Создаётся новый поток при необходимости
- Задача выполняется
- Поток хранится для повторного использования
- Поток завершается при долгом простое

### Шаг 15:

#### **Миссия 15**

Расположите шаги планирования задачи в `ScheduledThreadPool`.

Подсказка: Сначала указываем задержку или интервал, затем задача помещается в очередь, выполняется и повторяется.

**Ответ:**

- Задача регистрируется с задержкой/интервалом
- Задача помещается в очередь
- Задача выполняется потоком
- Задача повторяется при следующем интервале

### Шаг 16:

#### **Миссия 16**

Расположите шаги завершения работы `ExecutorService`.

Подсказка: Сначала вызов shutdown, потом ожидание завершения, при необходимости — shutdownNow, и в конце все потоки закрываются.

**Ответ:**

- Вызов shutdown()
- Ожидание завершения задач
- При необходимости вызов shutdownNow()
- Все потоки завершают работу

## Урок 3 - Future и Callable

### Шаг 01:

#### Миссия 1

Разработчик Артём написал задачу, которая должна возвращать результат вычислений в будущем. Какой интерфейс в Java ему стоит использовать?

Подсказка: Этот интерфейс похож на Runnable, но с возможностью вернуть результат.

**Ответ:**

- **Callable**
- Runnable
- Future
- Thread

### Шаг 02:

#### Миссия 2

Программист Оля запускает задачу с помощью ExecutorService и получает объект Future. Что может сделать Future?

Подсказка: Он не выполняет задачу сам, а хранит ссылку на её результат.

**Ответ:**

- **Получить результат выполнения**
- **Отменить задачу**
- Запустить новый поток
- Автоматически повторить задачу при сбое

### Шаг 03:

#### Миссия 3

Стас вызвал метод future.get() . Что произойдет, если результат ещё не готов?

Подсказка: Подумайте о том, что делает поток, когда данных ещё нет.

**Ответ:**

- **Текущий поток будет заблокирован до готовности результата**
- Будет возвращено null
- Метод выбросит исключение
- Future завершится с пустым значением

### Шаг 04:

#### Миссия 4

Даша хочет отменить выполнение долгой задачи. Какой метод у Future ей поможет?

Подсказка: Он возвращает true, если задача ещё не выполнена и её можно остановить.

**Ответ:**

- **cancel(true)**
- shutdown()
- stop()
- interrupt()

## Шаг 05:

### Миссия 5

Разработчик Никита решил использовать Callable вместо Runnable. Какое ключевое отличие?

Подсказка: Вспомните про возвращаемое значение и обработку ошибок.

**Ответ:**

- **Callable возвращает результат и может выбрасывать исключения**
- Callable запускается напрямую через new Thread()
- Runnable может возвращать значение
- Callable всегда выполняется быстрее Runnable

## Шаг 06:

### Миссия 6

Сопоставьте методы Future с их назначением.

Подсказка: Один метод блокирует поток, другой — отменяет выполнение.

**Ответ:**

- **get()** — Ожидает результат задачи
- **cancel(true)** — Пытается отменить выполнение
- **isDone()** — Проверяет, завершена ли задача
- **isCancelled()** — Показывает, была ли задача отменена

## Шаг 07:

### Миссия 7

Андрей работает с Callable. Сопоставьте интерфейс или метод с правильным описанием.

Подсказка: Один возвращает значение, другой управляет завершением.

**Ответ:**

- **Callable** — Интерфейс для задач с возвращаемым результатом
- **Runnable** — Интерфейс для задач без результата
- **Future** — Объект, управляющий результатом задачи
- **ExecutorService.submit()** — Запускает задачу и возвращает Future

## Шаг 08:

### Миссия 8

Сопоставьте исключение с ситуацией при работе с Future.

Подсказка: Вспомните, какие ошибки бывают при отмене или таймауте.

**Ответ:**

- **InterruptedException** — Текущий поток был прерван при ожидании
- **ExecutionException** — Задача завершилась с ошибкой

- **TimeoutException** — Истекло время ожидания результата
- **CancellationException** — Задача была отменена

## 🔗 Шаг 09:

### Миссия 9

Архитектор София объясняет новичкам разницу между методами `ExecutorService`. Сопоставьте метод с его функцией.

Подсказка: Обратите внимание, возвращает ли метод `Future` или нет.

**Ответ:**

- **`execute(Runnable)`** — Запускает задачу без возврата результата
- **`submit(Runnable)`** — Запускает задачу и возвращает `Future` без значения
- **`submit(Callable)`** — Запускает задачу и возвращает `Future` с результатом
- **`invokeAll()`** — Запускает коллекцию задач и возвращает список `Future`

## 🔗 Шаг 10:

### Миссия 10

Сопоставьте действие программиста с методом `Future`, который ему нужен.

Подсказка: Если нужно проверить статус — используйте методы с `is...`

**Ответ:**

- **Проверить, завершилась ли задача** — `isDone()`
- **Проверить, отменена ли задача** — `isCancelled()`
- **Получить результат** — `get()`
- **Остановить выполнение** — `cancel(true)`

## 🔗 Шаг 11:

### Миссия 11

Расположите шаги работы программиста с `Callable` и `Future` в правильном порядке.

Подсказка: Сначала создаём задачу, потом передаём в сервис, далее получаем `Future` и, наконец, результат.

**Ответ:**

- Создать класс, реализующий `Callable`
- Передать задачу в `ExecutorService` через `submit()`
- Получить объект `Future`
- Вызвать `get()` для получения результата

## 🔗 Шаг 12:

### Миссия 12

Расставьте по порядку действия `Future` при вызове `cancel(true)`.

Подсказка: Подумайте, как работает отмена: попытка остановить, затем отметка статуса.

**Ответ:**

- Попытка прервать задачу
- **Если задача уже выполнена** — отмена невозможна
- Пометить `Future` как отменённый
- Методы `isCancelled()` будут возвращать `true`

## 🔮 Шаг 13:

### Миссия 13

Расположите этапы обработки исключений при вызове `future.get()`.

Подсказка: Сначала ждём, потом могут возникнуть разные ошибки.

**Ответ:**

- Ожидание завершения задачи
- Проверка, не был ли поток прерван (`InterruptedException`)
- **Если задача выбросила ошибку** — `ExecutionException`
- **Если результат не получен вовремя** — `TimeoutException`

## 🔮 Шаг 14:

### Миссия 14

Расположите шаги при использовании `invokeAll()`.

Подсказка: Этот метод принимает коллекцию задач и возвращает список `Future`.

**Ответ:**

- Создать список `Callable` задач
- Передать список в `invokeAll()`
- Получить список `Future`
- Обработать результаты через `get()`

## 🔮 Шаг 15:

### Миссия 15

Расположите шаги при работе с `invokeAny()`.

Подсказка: Этот метод возвращает результат первой успешно завершённой задачи.

**Ответ:**

- Создать коллекцию `Callable`
- Вызвать `invokeAny()`
- Дождаться завершения одной задачи
- Получить результат этой задачи

## 🔮 Шаг 16:

### Миссия 16

Упорядочьте действия программиста, который хочет обработать таймаут при вызове `future.get(timeout, unit)`.

Подсказка: Если результат не успел вернуться, выбрасывается исключение.

**Ответ:**

- Вызвать `get(timeout, unit)`
- Ожидать результат указанное время
- **Если результат готов** — вернуть значение
- **Если время истекло** — выбросить `TimeoutException`

## Урок 4 - CompletionService для обработки результатов потоков

### Шаг 01:

#### Миссия 1

Разработчик Артём использует CompletionService для обработки задач. Что делает метод take() ?

Подсказка: Подумайте: метод ждёт, пока хотя бы одна задача завершится, и только потом возвращает результат.

**Ответ:**

- **Возвращает Future завершённой задачи, ожидая при необходимости**
- Запускает новую задачу в пуле
- Отменяет все невыполненные задачи
- Сразу возвращает null, если нет завершённых задач

### Шаг 02:

#### Миссия 2

Программист Лена использует CompletionService. Чем отличается метод poll() от take() ?

Подсказка: Вспомните: один из методов возвращает результат немедленно, а другой может ждать завершения задачи.

**Ответ:**

- **poll() не блокирует, а take() ждёт результат**
- poll() отменяет задачу, а take() завершает
- **poll() возвращает список всех результатов, а take() — только один**
- poll() запускает задачу, а take() проверяет статус

### Шаг 03:

#### Миссия 3

Команда разработчиков обрабатывает заказы. Какое преимущество даёт использование CompletionService по сравнению с обычным ExecutorService?

Подсказка: Подумайте о том, в каком порядке возвращаются результаты.

**Ответ:**

- **Результаты можно обрабатывать по мере готовности, а не в порядке отправки**
- CompletionService автоматически масштабирует пул потоков
- CompletionService гарантирует одинаковое время выполнения задач
- CompletionService всегда запускает задачи строго по очереди

### Шаг 04:

#### Миссия 4

Студент Игорь тестирует работу CompletionService. Что вернёт метод poll(timeout, unit) , если за указанное время ни одна задача не завершилась?

Подсказка: Этот метод похож на обычный poll(), но умеет ждать ограниченное время.

**Ответ:**

- **null**
- Пустой список
- Исключение TimeoutException
- Future с пустым результатом

## 🔗 Шаг 05:

### Миссия 5

Когда программист Коля использует `CompletionService`, он замечает: результаты приходят не по порядку запуска. Почему это нормально?

Подсказка: Вспомните: некоторые задачи завершаются быстрее других.

Ответ:

- Потому что **`CompletionService` отдаёт задачи по мере их завершения**
- Потому что задачи всегда сортируются случайным образом
- Потому что `ExecutorService` меняет порядок задач
- Потому что результаты сначала кэшируются, а потом перемешиваются

## 🔗 Шаг 06:

### Миссия 6

Разработчица Света пишет систему уведомлений. Она хочет обработать первые готовые результаты и не ждать остальные. Как это проще всего сделать с `CompletionService`?

Подсказка: Подумайте, какие методы позволяют получать результаты по одному.

Ответ:

- **Использовать цикл с `take()` или `poll()` и обрабатывать результаты по мере готовности**
- Использовать метод `invokeAll()`
- Ждать завершения пула потоков методом `shutdown()`
- Использовать метод `submit()` и вручную проверять `isDone()`

## 🔗 Шаг 07:

### Миссия 7

📝 Сопоставьте методы `CompletionService` с их назначением.

Подсказка: Вспомните разницу между `poll()`, `take()` и `submit()`.

Ответ:

- **`submit()`** — Отправка `Callable` или `Runnable` задачи на выполнение
- **`take()`** — Блокирующее получение завершённой задачи
- **`poll()`** — Неблокирующее получение задачи, если она завершена
- **`poll(timeout, unit)`** — Ожидание задачи ограниченное время

## 🔗 Шаг 08:

### Миссия 8

Сопоставьте понятие с правильным описанием в контексте `CompletionService`.

Подсказка: Подумайте о том, кто хранит и отдаёт результаты.

Ответ:

- **`Executor`** — Исполняет переданные задачи
- **`CompletionService`** — Организует очередь завершённых задач
- **`Future`** — Представляет результат асинхронной операции
- **`Callable`** — Задача, возвращающая значение

## 🔗 Шаг 09:

### Миссия 9

Разработчик Андрей хочет понять, как лучше собирать результаты. Сопоставьте метод с его характеристикой.

Подсказка: Вспомните: блокирующий метод ждёт, а неблокирующий — нет.

**Ответ:**

- **take()** — Ждёт, пока появится завершённая задача
- **poll()** — Сразу возвращает результат или null
- **poll(timeout)** — Ждёт ограниченное время
- **submit()** — Добавляет задачу на выполнение

## 🔗 Шаг 10:

### Миссия 10

📖 Сопоставьте использование CompletionService с описанием сценария.

Подсказка: Вспомните ситуации: быстрые задачи, долгие задачи и параллельные отчёты.

**Ответ:**

- **Быстрый поиск в БД** — Результаты можно получить сразу через poll()
- **Долгий рендер видео** — Лучше использовать take(), чтобы дождаться
- **Несколько отчётов** — Обработка по мере готовности через CompletionService
- **Параллельные уведомления** — Можно завершать по мере появления результатов

## 🔗 Шаг 11:

### Миссия 11

Разработчица Маша изучает разницу классов. Сопоставьте класс с его ролью.

Подсказка: Один класс — интерфейс, другой — конкретная реализация.

**Ответ:**

- **CompletionService** — Интерфейс для работы с завершёнными задачами
- **ExecutorCompletionService** — Стандартная реализация CompletionService
- **ExecutorService** — Запускает задачи и управляет пулом
- **FutureTask** — Обёртка, представляющая выполнение Callable или Runnable

## 🔗 Шаг 12:

### Миссия 12

📖 Сопоставьте ситуацию с действием программиста.

Подсказка: Подумайте: как быстро получить результат, если он готов, и что делать, если надо дождаться.

**Ответ:**

- **Хотим мгновенно проверить результат** — Используем poll()
- **Нужно обязательно дождаться** — Используем take()
- **Готовы ждать, но не вечно** — Используем poll(timeout)
- **Запуск новой задачи** — Используем submit()

## 🔗 Шаг 13:

### Миссия 13

Разработчик Петя хочет обработать результаты задач с `CompletionService`. Расставьте шаги в правильном порядке.

Подсказка: Сначала задачи отправляются, потом собираются результаты.

**Ответ:**

- Создать `ExecutorCompletionService`
- Отправить задачи через `submit()`
- Забирать результаты через `take()`
- Обрабатывать полученные данные

#### Шаг 14:

##### **Миссия 14**

Команда запускает несколько долгих задач. Какой порядок действий правильный?

Подсказка: Сначала создаём сервис, потом добавляем задачи, затем обрабатываем результаты.

**Ответ:**

- Создание `CompletionService`
- Отправка задач
- Получение результатов
- Анализ результатов

#### Шаг 15:

##### **Миссия 15**

 Расположите этапы работы `CompletionService` с учётом очереди завершённых задач.

Подсказка: Результаты приходят не по порядку запуска, а по мере готовности.

**Ответ:**

- Запуск задач в `ExecutorCompletionService`
- Завершение отдельных задач
- Помещение результатов в очередь
- Извлечение результатов через `poll/take`

#### Шаг 16:

##### **Миссия 16**

Разработчик Саша решил использовать `CompletionService`. В каком порядке выполняются действия?

Подсказка: Вспомните: `submit` — это запуск, `take` — это получение результата.

**Ответ:**

- Создать пул потоков
- Обёрнуть его в `ExecutorCompletionService`
- Вызвать `submit()` для задач
- Использовать `take()` для получения результатов

#### Шаг 17:

##### **Миссия 17**

Команда студентов пишет систему мониторинга. Расставьте шаги использования `CompletionService` в правильном порядке.

Подсказка: Логика проста: сначала создаём сервис, потом задачи, потом обрабатываем результаты.

**Ответ:**

- Создание `ExecutorCompletionService`
- Добавление задач через `submit()`
- Ожидание завершения через `take()`
- Анализ результатов

## **Модуль 5 - CompletableFuture и асинхронность**

### **Урок 1 - Основы CompletableFuture**

#### **Шаг 01:**

##### **Миссия 1**

Программист Вика впервые пробует `CompletableFuture`. Что это за класс?

Подсказка: Он связан с `Future`, но расширяет его возможности.

**Ответ:**

- **Класс для асинхронных вычислений с возможностью цепочек действий**
- Интерфейс для потоков ввода-вывода
- Абстракция над `ScheduledExecutorService`
- Специальный класс только для работы с базами данных

#### **Шаг 02:**

##### **Миссия 2**

Разработчик Саша запускает долгую операцию с помощью `CompletableFuture.supplyAsync()`. Чем этот метод отличается от `runAsync()` ?

Подсказка: Один метод возвращает результат, другой — нет.

**Ответ:**

- **`supplyAsync()` принимает `Supplier` и возвращает результат, `runAsync()` — только `Runnable` без результата**
- `runAsync()` всегда быстрее `supplyAsync()`
- `supplyAsync()` используется только в `main`-потоке
- `runAsync()` работает только в `single-thread executor`

#### **Шаг 03:**

##### **Миссия 3**

Студент Игорь хочет обработать результат асинхронной задачи. Какой метод позволяет добавить действие после её завершения?

Подсказка: Этот метод начинается на «`then...`».

**Ответ:**

- **`thenApply` или `thenAccept`**
- `get()`
- `cancel()`
- `invokeAll()`

## 🔗 Шаг 04:

### Миссия 4

Программист Дима хочет дождаться результата `CompletableFuture`. Что произойдет при вызове `get()` ?

Подсказка: Метод `get()` ведёт себя похоже на `Future.get()`.

**Ответ:**

- **Текущий поток будет заблокирован до получения результата**
- Возвращается всегда `null`
- Задача немедленно отменяется
- Метод асинхронно продолжает выполнение без ожидания

## 🔗 Шаг 05:

### Миссия 5

Разработчица Настя использует `thenCombine()` . Для чего этот метод нужен?

Подсказка: Вспомни, что `combine` — это объединение.

**Ответ:**

- **Для объединения результатов двух `CompletableFuture`**
- Для отмены задачи при ошибке
- Для запуска задачи в отдельном процессе
- Для объединения нескольких потоков в один

## 🔗 Шаг 06:

### Миссия 6

Команда разработчиков хочет выполнить действие после завершения всех асинхронных задач. Какой метод использовать?

Подсказка: Слово «all» подскажет правильный метод.

**Ответ:**

- **`CompletableFuture.allOf()`**
- `CompletableFuture.anyOf()`
- `CompletableFuture.runAsync()`
- `CompletableFuture.supplyAsync()`

## 🔗 Шаг 07:

### Миссия 7

Сопоставьте метод `CompletableFuture` с его описанием.

Подсказка: Подумайте, какие методы работают с результатом, а какие — без него.

**Ответ:**

- **`thenApply`** — Преобразует результат и возвращает новый
- **`thenAccept`** — Принимает результат, но ничего не возвращает
- **`thenRun`** — Запускает действие без параметров и результата
- **`get`** — Блокирует поток до получения результата

## 🔗 Шаг 08:

### Миссия 8

 Сопоставьте методы комбинирования задач.

Подсказка: Один метод ждёт все задачи, другой — только первую завершённую.

**Ответ:**

- **allOf** — Ждёт завершения всех задач
- **anyOf** — Завершается при первой завершённой задаче
- **thenCombine** — Объединяет результаты двух задач
- **thenCompose** — Использует результат одной задачи для запуска другой

## Шаг 09:

### Миссия 9

Разработчик Андрей изучает обработку ошибок. Сопоставьте методы с их поведением.

Подсказка: error-handling — важная часть асинхронного кода.

**Ответ:**

- **exceptionally** — Позволяет вернуть значение при ошибке
- **handle** — Позволяет обработать результат и ошибку
- **join** — Как get(), но без checked-исключений
- **cancel** — Отменяет выполнение задачи

## Шаг 10:

### Миссия 10

 Сопоставьте ситуацию с правильным методом.

Подсказка: Вспомните, что использовать для отмены, ожидания и объединения.

**Ответ:**

- **Хотим объединить два результата** — Используем thenCombine
- **Хотим запустить вторую задачу после первой** — Используем thenCompose
- **Хотим дождаться всех задач** — Используем allOf
- **Хотим отменить задачу** — Используем cancel

## Шаг 11:

### Миссия 11

Программистка Маша хочет разобраться с API. Сопоставьте класс или интерфейс с его назначением.

Подсказка: Future и CompletableFuture тесно связаны.

**Ответ:**

- **Future** — Представляет асинхронный результат
- **CompletableFuture** — Расширяет Future и позволяет строить цепочки
- **Supplier** — Функциональный интерфейс для поставки значения
- **Runnable** — Функциональный интерфейс без возвращаемого значения

## Шаг 12:

### Миссия 12

Сопоставьте методы запуска задач с их особенностями.

Подсказка: supplyAsync и runAsync различаются по возвращаемому значению.

**Ответ:**

- **runAsync** — Запускает Runnable без результата
- **supplyAsync** — Запускает Supplier и возвращает результат
- **allOf** — Комбинирует несколько CompletableFuture
- **anyOf** — Завершается при первой готовой задаче

### Шаг 13:

#### **Миссия 13**

Разработчик Петя хочет использовать CompletableFuture для загрузки данных. Расставьте шаги по порядку.

Подсказка: Сначала создаём задачу, потом обрабатываем результат.

**Ответ:**

- Создать CompletableFuture с supplyAsync
- Добавить обработчик через thenApply
- Вызвать join() для получения результата
- Вывести результат в консоль

### Шаг 14:

#### **Миссия 14**

Команда разработчиков пишет сервис для API-запросов. Расставьте действия.

Подсказка: Сначала запрос, потом обработка.

**Ответ:**

- Запустить supplyAsync с запросом
- Использовать thenCompose для цепочки
- Применить exceptionally для ошибок
- Вывести успешный результат

### Шаг 15:

#### **Миссия 15**

Студент Коля изучает объединение задач. Расставьте шаги.

Подсказка: thenCombine объединяет два результата.

**Ответ:**

- Создать первую задачу supplyAsync
- Создать вторую задачу supplyAsync
- Объединить результаты через thenCombine
- Вывести результат

### Шаг 16:

#### **Миссия 16**

Программист Антон отлаживает асинхронный код. Какой порядок действий правильный?

Подсказка: Помни про exceptionally и handle.

**Ответ:**

- Создать задачу supplyAsync
- Добавить обработчик результата через thenApply

- Добавить обработчик ошибок через `exceptionally`
- Вызвать `join()` для завершения

### Шаг 17:

#### Миссия 17

Команда студентов пишет систему логирования. Расставьте шаги работы с `CompletableFuture`.

Подсказка: Обрати внимание на порядок: запуск, обработка, ошибки, вывод.

**Ответ:**

- Запустить `runAsync` для фоновой задачи
- Добавить `thenRun` для дополнительного действия
- Добавить `handle` для обработки ошибок
- Вывести результат в лог

## Урок 2 - `thenApply`, `thenAccept`, `thenCompose`

### Шаг 01:

#### Миссия 1

Программист Алексей создает `CompletableFuture` для получения курса валют из API. Какой метод позволит ему обработать результат и вернуть преобразованное значение?

Подсказка: Подумайте, какой метод позволяет трансформировать результат без изменения исходного `CompletableFuture`.

**Ответ:**

- **`thenApply`**
- `thenAccept`
- `thenCompose`
- `join`

### Шаг 02:

#### Миссия 2

Екатерина запускает фоновую задачу и хочет вывести результат в консоль, но не менять сам результат. Какой метод `CompletableFuture` лучше использовать?

Подсказка: Здесь важна обработка результата без его трансформации.

**Ответ:**

- **`thenAccept`**
- `thenApply`
- `thenCompose`
- `get`

### Шаг 03:

#### Миссия 3

Михаил хочет объединить два `CompletableFuture`, один загружает данные пользователей, другой — их заказы. Какой метод подойдет для асинхронного объединения?

Подсказка: Метод должен возвращать новый `CompletableFuture`, основанный на результатах предыдущих.

**Ответ:**

- **thenCompose**
- thenApply
- thenAccept
- runAsync

 **Шаг 04:**

#### **Миссия 4**

Анна пишет метод для преобразования данных с сервера. Ей нужно выполнить функцию синхронно после завершения `CompletableFuture`. Какой метод лучше выбрать?

Подсказка: Нужно вернуть результат, изменив его внутри цепочки.

**Ответ:**

- **thenApply**
- thenAccept
- thenCompose
- complete

 **Шаг 05:**

#### **Миссия 5**

Сопоставьте методы `CompletableFuture` с их назначением:

**Ответ:**

- thenApply
  - thenAccept
  - thenCompose
  - runAsync
1. Выполняет функцию с результатом и возвращает новый `CompletableFuture`
  2. Обрабатывает результат без возврата нового значения
  3. Объединяет несколько `CompletableFuture` в один
  4. Запускает задачу асинхронно без результата

 **Подсказка:**

Подумайте, какой метод нужен для трансформации, какой — для действий с побочным эффектом.

- **thenApply** — Выполняет функцию с результатом и возвращает новый `CompletableFuture`
- **thenAccept** — Обрабатывает результат без возврата нового значения
- **thenCompose** — Объединяет несколько `CompletableFuture` в один
- **runAsync** — Запускает задачу асинхронно без результата

 **Шаг 06:**

#### **Миссия 6**

Сопоставьте термин с примером использования:

**Ответ:**

- thenApply
- thenAccept
- thenCompose

- join

1. Преобразование строки JSON в объект
2. Вывод результата загрузки данных в консоль
3. Объединение загрузки профиля и заказов пользователя
4. Ожидание завершения CompletableFuture и получение результата

#### Подсказка:

Обратите внимание, какой метод возвращает новый результат, а какой — просто выполняет действие.

- **thenApply** — Преобразование строки JSON в объект
- **thenAccept** — Вывод результата загрузки данных в консоль
- **thenCompose** — Объединение загрузки профиля и заказов пользователя
- **join** — Ожидание завершения CompletableFuture и получение результата

#### Шаг 07:

##### Миссия 7

Сопоставьте этапы цепочки CompletableFuture с их ролью:

##### Ответ:

- Создание CompletableFuture
  - thenApply
  - thenAccept
  - thenCompose
1. Инициализация асинхронной задачи
  2. Преобразование результата
  3. Действие с результатом без возврата
  4. Связывание нескольких асинхронных задач

#### Подсказка:

Последовательность операций часто идет от создания к преобразованию и обработке результатов.

- **Создание CompletableFuture** — Инициализация асинхронной задачи
- **thenApply** — Преобразование результата
- **thenAccept** — Действие с результатом без возврата
- **thenCompose** — Связывание нескольких асинхронных задач

#### Шаг 08:

##### Миссия 8

Сопоставьте сценарий с правильным методом CompletableFuture:

##### Ответ:

- Форматирование данных из API
  - Логирование результата в консоль
  - Получение профиля и заказов пользователя
  - Запуск фоновой задачи без результата
1. thenApply
  2. thenAccept
  3. thenCompose

#### 4. runAsync

#### 🔗 Подсказка:

Подумайте, что возвращает метод и есть ли необходимость объединять задачи.

- **Форматирование данных из API** — thenApply
- **Логирование результата в консоль** — thenAccept
- **Получение профиля и заказов пользователя** — thenCompose
- **Запуск фоновой задачи без результата** — runAsync

#### 🔗 Шаг 09:

##### Миссия 9

Расположите этапы работы с CompletableFuture для асинхронной загрузки и обработки данных:

1. Обработка результата (логирование или преобразование)
2. Создание CompletableFuture с задачей загрузки данных
3. Ожидание завершения и получение результата
4. Связывание нескольких задач с thenCompose

#### 🔗 Подсказка:

Сначала создается задача, потом она может быть преобразована или объединена, и в конце ожидается результат.

#### Ответ:

- Создание CompletableFuture с задачей загрузки данных
- Обработка результата (логирование или преобразование)
- Связывание нескольких задач с thenCompose
- Ожидание завершения и получение результата

#### 🔗 Шаг 10:

##### Миссия 10

Расположите последовательность действий программиста при использовании thenApply:

1. Возврат преобразованного значения
2. Написание функции для преобразования результата
3. Применение thenApply к CompletableFuture
4. Получение результата

#### 🔗 Подсказка:

Подумайте о логике цепочки: сначала пишем функцию, потом применяем к будущему, затем получаем результат.

#### Ответ:

- Написание функции для преобразования результата
- Применение thenApply к CompletableFuture
- Возврат преобразованного значения
- Получение результата

#### 🔗 Шаг 11:

##### Миссия 11

Упорядочьте шаги объединения двух `CompletableFuture` с помощью `thenCompose`:

1. Определение первой задачи
2. Создание второй задачи, зависящей от первой
3. Применение `thenCompose` для связывания задач
4. Получение итогового результата

 **Подсказка:**

Помните: `thenCompose` нужен, чтобы последовательные задачи выполнялись одна за другой.

**Ответ:**

- Определение первой задачи
- Создание второй задачи, зависящей от первой
- Применение `thenCompose` для связывания задач
- Получение итогового результата

 **Шаг 12:**

**Миссия 12**

Упорядочьте шаги, если программист хочет сначала обработать результат, а потом вывести его:

1. `thenAccept` для вывода результата
2. Создание функции обработки данных
3. `thenApply` для обработки результата
4. Запуск `CompletableFuture`

 **Подсказка:**

Сначала создаем и запускаем задачу, потом преобразуем данные, и в конце выводим результат.

**Ответ:**

- Запуск `CompletableFuture`
- Создание функции обработки данных
- `thenApply` для обработки результата
- `thenAccept` для вывода результата

 **Шаг 13:**

**Миссия 13**

Игорь создает цепочку `CompletableFuture` и замечает, что ему нужно дождаться результата сразу. Какой метод лучше использовать для синхронного получения результата?

Подсказка: Обратите внимание на методы, которые блокируют поток.

**Ответ:**

- `join`
- `thenApply`
- `thenAccept`
- `thenCompose`

 **Шаг 14:**

**Миссия 14**

Дмитрий хочет обработать результат `CompletableFuture` только после того, как завершатся сразу несколько других задач. Какой метод позволит объединить их в один будущий результат?

Подсказка: Подумайте о методах для объединения нескольких `CompletableFuture` одновременно.

**Ответ:**

- **thenCompose**
- thenApply
- thenAccept
- runAsync

🔗 **Шаг 15:**

### **Миссия 15**

Сопоставьте типичные сценарии использования `CompletableFuture` с подходящими методами:

**Ответ:**

- Трансформация результата запроса API
- Логирование без изменения результата
- Последовательная загрузка данных профиля и заказов
- Запуск фоновой задачи без результата

1. thenApply
2. thenAccept
3. thenCompose
4. runAsync

🔗 **Подсказка:**

Сосредоточьтесь на том, что возвращает метод и есть ли необходимость объединять результаты.

- **Трансформация результата запроса API** — thenApply
- **Логирование без изменения результата** — thenAccept
- **Последовательная загрузка данных профиля и заказов** — thenCompose
- **Запуск фоновой задачи без результата** — runAsync

🔗 **Шаг 16:**

### **Миссия 16**

Расположите шаги построения сложной цепочки `CompletableFuture` с преобразованием и выводом результатов:

1. Создание функции обработки данных
2. Запуск первой задачи с `CompletableFuture`
3. Применение thenApply для трансформации результата
4. Применение thenAccept для вывода результата
5. Объединение с другой задачей через thenCompose

🔗 **Подсказка:**

Цепочка должна идти: создаем задачу → трансформация → действие с результатом → объединение с другими задачами.

**Ответ:**

- Запуск первой задачи с `CompletableFuture`
- Создание функции обработки данных
- Применение thenApply для трансформации результата
- Применение thenAccept для вывода результата
- Объединение с другой задачей через thenCompose

## Урок 3 - Обработка ошибок и таймаутов

### Шаг 01:

#### Миссия 1

Саша запускает цепочку `CompletableFuture`, но один из этапов бросает исключение. Как лучше всего обработать ошибку, чтобы не разрушить всю цепочку?

Подсказка: В `CompletableFuture` есть методы, которые ловят исключения и позволяют восстановить работу.

Ответ:

- **Использовать `.exceptionally` или `.handle`, чтобы вернуть fallback значение при ошибке**
- Игнорировать исключение и завершить цепочку штатно
- Бросить `RuntimeException` наружу без обработки
- В том же `thenApply` ловить `try-catch` и подавлять ошибку

### Шаг 02:

#### Миссия 2

Ирина хочет запустить асинхронную задачу с таймаутом: если `CompletableFuture` не завершится за заданное время — отменить её и вернуть запасной результат. Какой подход правильный?

Подсказка: `CompletableFuture` не поддерживает таймауты «из коробки», но можно комбинировать с другими средствами.

Ответ:

- **Запустить `CompletableFuture` вместе с `ScheduledExecutorService`, который завершит её через `timeout`**
- Ожидать `future.get(timeout)` внутри `thenApply`
- Бросить `InterruptedException` в `thenCompose` и ждать бесконечно
- **Ничего не делать** — пусть блокируется

### Шаг 03:

#### Миссия 3

Пётр строит сложную цепочку из `thenCompose`, `thenApply` и обрабатывает ошибку где-то в середине. Что произойдет, если на этапе `thenApply` возникнет исключение, и он не поймёт его локально?

Подсказка: Исключения «пропускаются» дальше, если не обработаны.

Ответ:

- **Исключение передастся дальше и весь `CompletableFuture` завершится `exceptionally`**
- Цепочка продолжится как будто ничего не случилось
- `thenCompose` «проглотит» исключение и вернёт `null`
- Программа зависнет

### Шаг 04:

#### Миссия 4

Ольга хочет логировать ошибки, но не менять значение результата цепочки. Что она должна использовать?

Подсказка: Есть методы, которые дают доступ к исключению, не меняя результат.

**Ответ:**

- **Использовать `.whenComplete` и логировать исключение, а затем возвращать исходное значение**
- Вставить печать в `thenApply` и ловить исключение там
- Использовать `get` и ловить `ExecutionException`
- **Ничего не делать** — исключение подавится автоматически

 **Шаг 05:**

### Миссия 5

Василий хочет сразу несколько независимых задач запустить и дождаться результата первой успешной, игнорируя те, что упали или не успели. Какой метод подойдет?

Подсказка: Можно комбинировать `CompletableFuture`, чтобы «гоняться» за первым завершившимся.

**Ответ:**

- **Использовать `CompletableFuture.anyOf` и фильтровать исключения**
- Использовать `allOf` и брать первый элемент списка
- Запустить `get` на каждом и выбрать первый не исключение
- Использовать `thenCompose` последовательно

 **Шаг 06:**

### Миссия 6

Анна хочет комбинировать две асинхронные задачи, но, если одна из них не успела завершиться в пределах таймаута, использовать запасной результат. Как поступить?

Подсказка: Можно применить `timeout` внутри каждой задачи и потом комбинировать.

**Ответ:**

- **Обернуть каждую `CompletableFuture` в таймаут-обёртку и затем использовать `thenCombine`**
- Просто вызвать `thenCombine`, а потом `.get` с `timeout`
- Выполнить одну задачу внутри другой через `thenCompose`
- Запустить все задачи `sync`-методами

 **Шаг 07:**

### Миссия 7

Кирилл хочет, чтобы цепочка автоматически ретрилась при временной ошибке (`timeout`, `соединение`), но не при любой `RuntimeException`. Что он должен сделать?

Подсказка: `.handle` или `.exceptionally` возвращают значение, но чтобы ретрай — нужна логика внутри.

**Ответ:**

- **В `.handle` проверять тип исключения и при нужном** — запустить повторно через новую цепочку
- В `.exceptionally` возвращать `null` и запускать `retry` в `thenApply`
- Ловить все `RuntimeException` и запускать `retry` без фильтрации
- **Ничего** — пусть вызывающий `retry`» вручную

 **Шаг 08:**

### Миссия 8

Сопоставьте методы `CompletableFuture` с их основным назначением в контексте обработки ошибок и таймаутов.

Методы:

**Ответ:**

- `exceptionally`
- `handle`
- `whenComplete`
- `completeOnTimeout`

Назначения:

1. Возвращает fallback значение при исключении
2. Позволяет вернуть либо результат, либо ошибку как значение
3. Позволяет выполнить побочный код (логирование) без изменения результата
4. Автоматически завершает future запасным значением при таймауте

Подсказка: Задача — вспомнить, как каждый метод влияет на результат цепочки.

- **`exceptionally`** — Возвращает fallback значение при исключении
- **`handle`** — Позволяет вернуть либо результат, либо ошибку как значение
- **`whenComplete`** — Позволяет выполнить побочный код (логирование) без изменения результата
- **`completeOnTimeout`** — Автоматически завершает future запасным значением при таймауте

## Шаг 09:

### Миссия 9

Сопоставьте ошибки/ситуации с подходящим способом обработки в `CompletableFuture`.

Ситуации:

**Ответ:**

- Исключение соединения, хочется `retry`
- Операция превышает таймаут, вернуть запасной ответ
- Исключение, но нужно логировать и продолжить с дефолтом
- **Завершение без исключения** — просто проследить

Способы:

1. `handle` + условный `retry`
2. `completeOnTimeout` или `orTimeout`
3. `exceptionally` + логирование внутри
4. `whenComplete` для мониторинга

Подсказка: Для каждой ситуации подумайте, нужно ли изменять результат, или просто выполнить побочную логику.

- **Исключение соединения, хочется `retry`** — `handle` + условный `retry`
- **Операция превышает таймаут, вернуть запасной ответ** — `completeOnTimeout` или `orTimeout`
- **Исключение, но нужно логировать и продолжить с дефолтом** — `exceptionally` + логирование внутри
- **Завершение без исключения** — просто проследить — `whenComplete` для мониторинга

## 🔗 Шаг 10:

### Миссия 10

Сопоставьте комбинации типов методов с ожидаемым поведением цепочки:

Комбинации:

**Ответ:**

- `thenApply` → `exceptionally`
- `thenCompose` → `handle`
- `whenComplete` → `thenApply`
- `completeOnTimeout` → `thenApply`

Поведение:

1. Результат преобразуется, ошибки заменяются `fallback`
2. **Если при составлении вложенного `future` ошибка** — можно восстановиться
3. На завершении логика, затем трансформация результата
4. **Если `timeout`** — обрабатываем и трансформируем результат

Подсказка: Смотрите, на каком этапе вставляется обработка ошибок или `timeout`.

- **`thenApply` → `exceptionally`** — Результат преобразуется, ошибки заменяются `fallback`
- **`thenCompose` → `handle`** — Если при составлении вложенного `future` ошибка — можно восстановиться
- **`whenComplete` → `thenApply`** — На завершении логика, затем трансформация результата
- **`completeOnTimeout` → `thenApply`** — Если `timeout` — обрабатываем и трансформируем результат

## 🔗 Шаг 11:

### Миссия 11

Расположите шаги создания таймаут-обёртки над `CompletableFuture` в правильном порядке:

1. Создать `ScheduledExecutorService` для отложенного действия
2. Запустить оригинальный `CompletableFuture` задачи
3. С помощью `scheduler` через заданное время вызвать `future.complete` или `completeExceptionally`
4. Комбинировать (например, `applyToEither`) две `future`: оригинал и таймаутный

Подсказка: Сначала запускаем задачу, затем готовим таймаут, а потом комбинируем результаты.

**Ответ:**

- Создать `ScheduledExecutorService` для отложенного действия
- Запустить оригинальный `CompletableFuture` задачи
- С помощью `scheduler` через заданное время вызвать `future.complete` или `completeExceptionally`
- Комбинировать (например, `applyToEither`) две `future`: оригинал и таймаутный

## 🔗 Шаг 12:

### Миссия 12

Расположите этапы обработки `retry`-механизма в цепочке:

1. Поймать ошибку, проверить её тип

2. **Если ошибка** — запустить новую попытку через новый `CompletableFuture`
3. Ограничить максимальное число попыток
4. **Если успех** — продолжить основную цепочку

Подсказка: Не стоит просто бесконечно перезапускать без ограничений.

**Ответ:**

- Поймать ошибку, проверить её тип
- **Если ошибка** — запустить новую попытку через новый `CompletableFuture`
- Ограничить максимальное число попыток
- **Если успех** — продолжить основную цепочку

### Шаг 13:

#### Миссия 13

Расположите по порядку обработку зависимых задач с таймаутом и fallback:

1. Запустить первую задачу с таймаутом-обёрткой
2. На её основе запустить второй `thenCompose` / `thenApply`
3. **Если первая завершилась с ошибкой или таймаутом** — использовать запасной результат
4. Обрабатывать ошибки второго этапа через `exceptionally` или `handle`

Подсказка: Сначала гарантия выполнения или fallback для первого, затем построение цепочки дальше.

**Ответ:**

- Запустить первую задачу с таймаутом-обёрткой
- На её основе запустить второй `thenCompose` / `thenApply`
- **Если первая завершилась с ошибкой или таймаутом** — использовать запасной результат
- Обрабатывать ошибки второго этапа через `exceptionally` или `handle`

### Шаг 14:

#### Миссия 14

Расположите шаги комплексной задачи: множество параллельных futures, таймауты и агрегация результатов:

1. Оборачиваем каждый future таймаут-механизмом
2. Запускаем все futures параллельно
3. Собираем результаты через `allOf` / `anyOf`
4. Обрабатываем ошибки / таймауты через `handle` / `exceptionally`

Подсказка: Таймаут индивидуально на каждой задаче, потом ждём все вместе.

**Ответ:**

- Оборачиваем каждый future таймаут-механизмом
- Запускаем все futures параллельно
- Собираем результаты через `allOf` / `anyOf`
- Обрабатываем ошибки / таймауты через `handle` / `exceptionally`

### Шаг 15:

#### Миссия 15

Расположите этапы смешанной стратегии: таймаут + retry + fallback:

1. Запустить задачу с таймаутом-обёрткой и fallback внутри
2. **Если fallback неудач** — инициировать retry
3. Ограничить число повторов / задержку между ними
4. **После успешного выполнения** — вернуть результат

Подсказка: Таймаут и fallback на внешнем уровне, retry — внутри, но с контролем.

**Ответ:**

- Запустить задачу с таймаутом-обёрткой и fallback внутри
- **Если fallback неудач** — инициировать retry
- Ограничить число повторов / задержку между ними
- **После успешного выполнения** — вернуть результат

## Шаг 16:

### Миссия 16

Расположите стратегию обработки «любых первых успешных» с таймаутом:

1. Оборачиваем каждый исходный future в таймаутную обёртку
2. Запускаем все параллельно
3. Используем аnyOf, чтобы выбрать первый завершившийся
4. Обрабатываем возможную ошибку или таймаут окончательного выбранного

Подсказка: Для каждого — таймаут, потом выбор первого, потом валидация результата.

**Ответ:**

- Оборачиваем каждый исходный future в таймаутную обёртку
- Запускаем все параллельно
- Используем аnyOf, чтобы выбрать первый завершившийся
- Обрабатываем возможную ошибку или таймаут окончательного выбранного

## Шаг 17:

### Миссия 17

Расположите цепочку действий при комбинированной обработке ошибок и таймаутов для сложной задачи:

1. Запустить задачу A с таймаутной обёрткой и fallback A'
2. thenCompose к задаче B (которая зависит от результата A или fallback)
3. Для B тоже обернуть в таймаут и fallback B'
4. Обработать ошибки и исключения на всем пути через handle / exceptionally

Подсказка: Важно в каждом этапе гарантировать, что неудача не сломает весь поток.

**Ответ:**

- Запустить задачу A с таймаутной обёрткой и fallback A'
- thenCompose к задаче B (которая зависит от результата A или fallback)
- Для B тоже обернуть в таймаут и fallback B'
- Обработать ошибки и исключения на всем пути через handle / exceptionally

## Урок 4 - Combine и allOf для параллельных операций

### Шаг 01:

#### Миссия 1

Алёна хочет выполнить две задачи параллельно и получить результат первой, которая завершится успешно. Какой метод `CompletableFuture` использовать?

Подсказка: `oneOf` композиции ищет первый завершившийся future.

Ответ:

- **`CompletableFuture.anyOf(...)`**
- `CompletableFuture.allOf(...)`
- `thenCombine`
- `thenCompose`

### Шаг 02:

#### Миссия 2

Борис хочет запустить три независимые асинхронные операции и дождаться, когда **все** завершатся, неважно — успешно или с ошибкой. Как ему действовать?

Подсказка: `allOf` блокирует на все futures, но обработка ошибок — на вас.

Ответ:

- **Использовать `CompletableFuture.allOf(...)` и потом проверять статус каждого**
- Использовать `anyOf` и игнорировать ошибки
- Запустить `oneOf` и брать первый завершившийся
- Вызывать `get()` на каждом future вручную

### Шаг 03:

#### Миссия 3

Катя хочет объединить результаты двух futures (A и B) в одно значение (например, суммировать). Какой метод лучше всего использовать?

Подсказка: `combine` позволяет объединять значения при параллелизме.

Ответ:

- **`thenCombine(A, B, (a,b)->...)`**
- `allOf` и потом брать значения из A и B
- `anyOf` и комбинировать результат
- `thenCompose` внутри A на B

### Шаг 04:

#### Миссия 4

Дмитрий запускает три future и хочет собрать их значения в список, как только все завершатся. Что он сделает?

Подсказка: `allOf` не возвращает результаты сам по себе.

Ответ:

- **`allOf`, а затем из каждой исходной future взять `get` или `join`**
- `anyOf` и собрать результаты
- Сделать `sequential thenCompose`
- Выполнить futures синхронно в потоке `main`

## Шаг 05:

### Миссия 5

Евгения хочет запустить futures A и B, и если B завершится раньше A — подождать A, но если A завершится раньше — сразу вернуть A + B (если готово) или ждать B. Какой подход нужен?

Подсказка: Можно комбинировать `combine` и `anyOf`: выбор давления времени.

Ответ:

- Запустить **`thenCombine`**, но при любом завершении одного — ожидать второго через `allOf/anyOf` логику
- Использовать `anyOf` просто и игнорировать второй
- Использовать `allOf` и ждать оба всегда
- Сделать `nested thenCompose` вручную

## Шаг 06:

### Миссия 6

Сопоставьте методы с их роли при параллельной композиции:

Методы:

Ответ:

- `thenCombine`
- `thenAcceptBoth`
- `allOf`
- `anyOf`

Роли:

1. Ожидать оба результата и вернуть новое значение
2. Ожидать оба результата и выполнить действие без возвращения значения
3. Ожидать, что все futures завершатся
4. Возвращать результат первым завершившегося future

Подсказка: Разница между `thenCombine` и `thenAcceptBoth` — в возвращаемом значении.

- **`thenCombine`** — Ожидать оба результата и вернуть новое значение
- **`thenAcceptBoth`** — Ожидать оба результата и выполнить действие без возвращения значения
- **`allOf`** — Ожидать, что все futures завершатся
- **`anyOf`** — Возвращать результат первым завершившегося future

## Шаг 07:

### Миссия 7

Сопоставьте сценарий использования с комбинирующим паттерном:

Сценарии:

Ответ:

- Нужно получить первую успешную цену от нескольких API
- Нужно дождаться всех API и усреднить результаты
- Нужно дождаться двух API и сложить значения
- Нужно дождаться двух API и выполнить побочное действие (лог)

Паттерны:

1. anyOf + фильтрация ошибок
2. allOf + извлечение результатов
3. thenCombine
4. thenAcceptBoth

Подсказка: Подумайте, где нужен результат, а где лишь эффект.

- **Нужно получить первую успешную цену от нескольких API** — anyOf + фильтрация ошибок
- **Нужно дождаться всех API и усреднить результаты** — allOf + извлечение результатов
- **Нужно дождаться двух API и сложить значения** — thenCombine
- **Нужно дождаться двух API и выполнить побочное действие (лог)** — thenAcceptBoth

## Шаг 08:

### Миссия 8

Сопоставьте методы композиции с их поведением, когда один из futures завершится исключением:

Методы:

**Ответ:**

- thenCombine
- allOf
- anyOf
- thenAcceptBoth

Поведение при исключении:

1. **Если один future исключается** — композиция завершится с ошибкой
2. В allOf исключение отменяет весь allOf
3. В anyOf можно выбрать другой, если он успешен
4. thenAcceptBoth ведёт себя как thenCombine, но без возвращаемого значения

Подсказка: anyOf допускает альтернативу, другие — жёстче.

- **thenCombine** — Если один future исключается — композиция завершится с ошибкой
- **allOf** — В allOf исключение отменяет весь allOf
- **anyOf** — В anyOf можно выбрать другой, если он успешен
- **thenAcceptBoth** — thenAcceptBoth ведёт себя как thenCombine, но без возвращаемого значения

## Шаг 09:

### Миссия 9

Сопоставьте подход к обработке с комбинацией futures:

Подходы:

**Ответ:**

- Обёртка каждого future try/catch + thenCombine
- Запустить allOf, затем фильтровать исключения
- Использовать anyOf для выбора первой успешной
- Использовать thenAcceptBoth для логирования результата двух futures

Сценарии:

1. Нужно безопасно комбинировать два результата, даже если один fails
2. Нужно дождаться всех и убрать упавшие  
Нужно получить первую успешную ответ
3. Нужно дождаться две операции и поддерживать side-эффект

Подсказка: При комбинировании важно, как обрабатываются исключения в каждом future.

- **Обёртка каждого future try/catch + thenCombine** — Нужно безопасно комбинировать два результата, даже если один fails
- **Запустить allOf, затем фильтровать исключения** — Нужно дождаться всех и убрать упавшие
- **Использовать anyOf для выбора первой успешной** — Нужно получить первую успешную ответ
- **Использовать thenAcceptBoth для логирования результата двух futures** — Нужно дождаться две операции и поддерживать side-эффект

## Шаг 10:

### Миссия 10

Расположите шаги сборки результата с allOf и thenApply:

1. Запустить все futures параллельно
2. Вызвать CompletableFuture.allOf(...) для их объединения
3. После завершения allOf извлечь значения через join/get
4. С помощью thenApply/thenCompose собрать итоговый объект

Подсказка: allOf лишь ждёт, но не возвращает данные внутри.

**Ответ:**

- Запустить все futures параллельно
- Вызвать CompletableFuture.allOf(...) для их объединения
- После завершения allOf извлечь значения через join/get
- С помощью thenApply/thenCompose собрать итоговый объект

## Шаг 11:

### Миссия 11

Расположите шаги использования anyOf для выбора первого успешного:

1. Запустить все futures параллельно
2. Вызвать CompletableFuture.anyOf(...) на них
3. На получившемся future проверить, было ли исключение
4. **Если исключение** — дождаться следующего успешного вручную либо повторить anyOf

Подсказка: anyOf возвращает первый завершившийся, успешный или нет.

**Ответ:**

- Запустить все futures параллельно
- Вызвать CompletableFuture.anyOf(...) на них
- На получившемся future проверить, было ли исключение
- **Если исключение** — дождаться следующего успешного вручную либо повторить anyOf

## Шаг 12:

### Миссия 12

Расположите этапы безопасной комбинации двух futures с fallback:

1. Каждый future оборачивают в handle/exceptionally для fallback
2. Затем применяют thenCombine к уже безопасным futures
3. В итоговом thenApply проверяют, какой из fallback использован
4. Вернуть агрегированный результат или дефолт

Подсказка: Лучше обрабатывать риски заранее, до combine.

**Ответ:**

- Каждый future оборачивают в handle/exceptionally для fallback
- Затем применяют thenCombine к уже безопасным futures
- В итоговом thenApply проверяют, какой из fallback использован
- Вернуть агрегированный результат или дефолт

## Шаг 13:

### Миссия 13

Расположите стратегию, когда нужно дождаться трёх futures, но взять лишь сумму двух лучших (наибольших):

1. Запустить три future параллельно
2. Использовать allOf, дождаться всех
3. Извлечь все три результата
4. Отсортировать и сложить две наибольшие

Подсказка: После allOf легко получить все результаты.

**Ответ:**

- Запустить три future параллельно
- Использовать allOf, дождаться всех
- Извлечь все три результата
- Отсортировать и сложить две наибольшие

## Шаг 14:

### Миссия 14

Расположите шаги стратегии «первый успешный из группы, но ждать минимум N результатов»:

1. Запустить все futures параллельно
2. Использовать anyOf, чтобы поймать первый удачный
3. Продолжать ждать остальные до достижения нужного числа N через allOf / комбинирование
4. Собрать итоговый результат из первых N успешных

Подсказка: Сочетание anyOf и allOf может быть полезно.

**Ответ:**

- Запустить все futures параллельно
- Использовать anyOf, чтобы поймать первый удачный
- Продолжать ждать остальные до достижения нужного числа N через allOf / комбинирование

- Собрать итоговый результат из первых N успешных

## Шаг 15:

### Миссия 15

Расположите шаги сложной композиции с fallback, retry и combine:

1. Оборачиваем каждый future в fallback/retry защиту
2. Запускаем все параллельно
3. Собираем результаты с thenCombine / allOf
4. Обрабатываем окончательные ошибки и default

Подсказка: Безопасность на границах, комбинирование в центре.

#### Ответ:

- Оборачиваем каждый future в fallback/retry защиту
- Запускаем все параллельно
- Собираем результаты с thenCombine / allOf
- Обрабатываем окончательные ошибки и default

## Шаг 16:

### Миссия 16

Расположите шаги при комбинировании nested групп futures (группа внутри группы):

1. Запустить внутренние группы через allOf / anyOf
2. Получить результаты внутренних групп
3. Комбинировать группы через thenCombine / allOf
4. Обработать ошибки на всех уровнях

Подсказка: Сначала внутренняя агрегация, потом внешняя.

#### Ответ:

- Запустить внутренние группы через allOf / anyOf
- Получить результаты внутренних групп
- Комбинировать группы через thenCombine / allOf
- Обработать ошибки на всех уровнях

## Модуль 6 - Паттерны многопоточности

### Урок 1 - Producer-Consumer

## Шаг 01:

### Миссия 1

Анна разрабатывает приложение с паттерном Producer-Consumer. Что из ниже перечисленного является основной задачей Producer?

Подсказка: Producer обычно отвечает за создание или получение данных.

#### Ответ:

- **Создавать и помещать элементы в очередь**
- Потреблять элементы из очереди
- Синхронизировать потоки
- Управлять доступом к базе данных

## Шаг 02:

### Миссия 2

Максим столкнулся с проблемой, что Consumer потребляет данные быстрее, чем Producer их производит. Что может помочь сгладить такую ситуацию?

Подсказка: Посмотрите на структуру хранения данных между потоками.

**Ответ:**

- **Использовать буферизированную очередь**
- Увеличить число Consumer
- Удалить синхронизацию
- Понизить приоритет Producer

## Шаг 03:

### Миссия 3

Игорь реализует Producer-Consumer с помощью wait и notify. Что из перечисленного верно для wait?

Подсказка: wait освобождает монитор, позволяя другим потокам работать.

**Ответ:**

- **Останавливает текущий поток и освобождает монитор до notify**
- Останавливает поток без освобождения монитора
- Вызывает исключение при вызове без синхронизации
- Автоматически возобновляет работу через 5 секунд

## Шаг 04:

### Миссия 4

Екатерина использует BlockingQueue для реализации Producer-Consumer. Что происходит, если очередь заполнена, а Producer пытается добавить элемент?

Подсказка: BlockingQueue умеет блокировать потоки при необходимости.

**Ответ:**

- **Producer блокируется до освобождения места**
- Происходит ошибка времени выполнения
- Элемент удаляется из конца очереди
- Producer пропускает добавление

## Шаг 05:

### Миссия 5

Вадим проектирует систему, где несколько Producer и несколько Consumer работают одновременно. Что важнее всего для корректной работы очереди?

Подсказка: Подумайте о потокобезопасности.

**Ответ:**

- **Использовать потокобезопасную очередь**
- Выделить отдельный поток для синхронизации
- Ограничить число Producer до одного
- Использовать обычный список

## Шаг 06:

### Миссия 6

 Сопоставьте термины из паттерна Producer-Consumer с их описаниями.

Термины:

Ответ:

- Producer
- Consumer
- Буфер
- Deadlock

Определения:

1. Поток, который создаёт данные и помещает их в очередь.
2. Поток, который забирает данные из очереди для обработки.
3. Область памяти, где хранятся данные для передачи между потоками.
4. Ситуация, когда потоки навсегда блокируют друг друга.

Подсказка: Термины связаны с ролями и состояниями потоков.

- **Producer** — Поток, который создаёт данные и помещает их в очередь.
- **Consumer** — Поток, который забирает данные из очереди для обработки.
- **Буфер** — Область памяти, где хранятся данные для передачи между потоками.
- **Deadlock** — Ситуация, когда потоки навсегда блокируют друг друга.

## Шаг 07:

### Миссия 7

Алина столкнулась с ситуацией, когда Consumer блокируется, ожидая элемент в пустой очереди. Какое поведение Producer может избежать этой проблемы?

Подсказка: Producer должен гарантировать, что элементы появляются в очереди вовремя.

Ответ:

- **Регулярно создавать и помещать элементы в очередь**
- Игнорировать очередь
- Удалять элементы из очереди
- Запускать Consumer с задержкой

## Шаг 08:

### Миссия 8

Павел проектирует систему Producer-Consumer, используя синхронизированные блоки. Что он должен учитывать, чтобы избежать проблем с производительностью?

Подсказка: Слишком долгий захват блокировки тормозит работу других потоков.

Ответ:

- **Минимизировать время удержания блокировки**
- Использовать одну глобальную блокировку для всего
- Игнорировать блокировки
- Использовать только wait без notify

## Шаг 09:

### Миссия 9

Наташа использует Semaphore для контроля доступа к буферу в паттерне Producer-Consumer. Какой основной смысл у Semaphore в этом контексте?

Подсказка: Semaphore управляет числом потоков, одновременно работающих с ресурсом.

**Ответ:**

- **Ограничить количество потоков, которые могут одновременно получить доступ к буферу**
- Разрешить всем потокам работать одновременно
- Блокировать всех Consumer
- Автоматически увеличивать размер очереди

## Шаг 10:

### Миссия 10

 Сопоставьте основные проблемы паттерна Producer-Consumer с их описаниями.

Проблемы:

**Ответ:**

- Race Condition
- Deadlock
- Starvation
- Buffer Overflow

Описание:

1. Одновременный доступ потоков приводит к некорректному состоянию данных.
2. Потоки навсегда блокируют друг друга, не позволяя программе продолжать работу.
3. Один поток постоянно блокируется, не получая ресурсы из-за других.
4. Производитель добавляет данные в уже полный буфер.

Подсказка: Обратите внимание, как взаимодействуют потоки и ресурсы.

- **Race Condition** — Одновременный доступ потоков приводит к некорректному состоянию данных.
- **Deadlock** — Потоки навсегда блокируют друг друга, не позволяя программе продолжать работу.
- **Starvation** — Один поток постоянно блокируется, не получая ресурсы из-за других.
- **Buffer Overflow** — Производитель добавляет данные в уже полный буфер.

## Шаг 11:

### Миссия 11

 Сопоставьте методы из `java.util.concurrent` с их назначением в Producer-Consumer:

Методы:

**Ответ:**

- `wait()`
- `notify()`
- `put()`
- `take()`

Назначение:

1. Освобождение заблокированного потока для продолжения работы.
2. Ожидание события, освобождая монитор.
3. Добавление элемента в очередь, блокируя при заполнении.
4. Получение элемента из очереди, блокируя при пустоте.

Подсказка: Помните, что `wait` и `notify` работают с монитором, а `put/take` с очередью.

- **`wait()`** — Ожидание события, освобождая монитор.
- **`notify()`** — Освобождение заблокированного потока для продолжения работы.
- **`put()`** — Добавление элемента в очередь, блокируя при заполнении.
- **`take()`** — Получение элемента из очереди, блокируя при пустоте.

## Шаг 12:

### Миссия 12

 Сопоставьте причины возможных ошибок с их проявлениями в паттерне `Producer-Consumer`.

Причины:

**Ответ:**

- Отсутствие синхронизации
- Неправильное использование `wait/notify`
- Неправильный размер буфера
- Перекрестная блокировка

Проявления:

1. Несогласованные данные и ошибки в очереди.
2. Поток может никогда не проснуться.
3. Потеря данных или частая блокировка потоков.
4. Все потоки навсегда заблокированы друг другом.

Подсказка: Каждая проблема связана с особенностями многопоточности и управления ресурсами.

- **Отсутствие синхронизации** — Несогласованные данные и ошибки в очереди.
- **Неправильное использование `wait/notify`** — Поток может никогда не проснуться.
- **Неправильный размер буфера** — Потеря данных или частая блокировка потоков.
- **Перекрестная блокировка** — Все потоки навсегда заблокированы друг другом.

## Шаг 13:

### Миссия 13

Расположите этапы отладки проблемы блокировки в реализации `Producer-Consumer` в правильном порядке:

1. Проанализировать точки захвата и освобождения блокировок
2. Воспроизвести проблему на тестовом окружении
3. Внедрить логирование потоков и состояний буфера
4. Исправить некорректное использование `wait/notify`
5. Проверить исправления и повторить тесты

Подсказка: Сначала нужно понять проблему, потом собрать данные, после - исправлять и проверять.

**Ответ:**

- Воспроизвести проблему на тестовом окружении
- Внедрить логирование потоков и состояний буфера
- Проанализировать точки захвата и освобождения блокировок
- Исправить некорректное использование wait/notify
- Проверить исправления и повторить тесты

#### Шаг 14:

##### **Миссия 14**

Какой механизм лучше всего использовать для предотвращения состояния гонки в Producer-Consumer?

Подсказка: Защита критической секции важна для корректной работы.

**Ответ:**

- **Использовать блокировки (locks) или synchronized**
- Игнорировать проблему
- Запускать несколько Producers без синхронизации
- Удалять элементы из очереди в любом порядке

#### Шаг 15:

##### **Миссия 15**

 Сопоставьте термины с их определениями в контексте паттерна Producer-Consumer:

Термины:

**Ответ:**

- BlockingQueue
- Deadlock
- Semaphore
- Wait/Notify

Определения:

1. Структура данных, которая блокирует потоки при попытке взять или положить элемент, если очередь пуста или заполнена.
2. Состояние, при котором два или более потока навсегда ожидают друг друга, не освобождая ресурсы.
3. Механизм, ограничивающий количество потоков, одновременно имеющих доступ к ресурсу.
4. Методы для управления ожиданием и пробуждением потоков в Java.

Подсказка: Подумайте, как синхронизация и управление ресурсами реализуются на практике.

- **BlockingQueue** — Структура данных, которая блокирует потоки при попытке взять или положить элемент, если очередь пуста или заполнена.
- **Deadlock** — Состояние, при котором два или более потока навсегда ожидают друг друга, не освобождая ресурсы.
- **Semaphore** — Механизм, ограничивающий количество потоков, одновременно имеющих доступ к ресурсу.
- **Wait/Notify** — Методы для управления ожиданием и пробуждением потоков в Java.

## Шаг 16:

### Миссия 16

Расположите шаги для безопасной реализации паттерна Producer-Consumer с использованием BlockingQueue в правильном порядке:

1. Создать BlockingQueue с ограниченным размером
2. В Producer вызвать put() для добавления элемента (блокируется при заполнении)
3. В Consumer вызвать take() для получения элемента (блокируется при пустой очереди)
4. Обработать полученный элемент в Consumer
5. Запустить потоки Producer и Consumer

Подсказка: Очередь должна быть создана до использования, а методы put/take обеспечивают потокобезопасность.

#### Ответ:

- Создать BlockingQueue с ограниченным размером
- Запустить потоки Producer и Consumer
- В Producer вызвать put() для добавления элемента (блокируется при заполнении)
- В Consumer вызвать take() для получения элемента (блокируется при пустой очереди)
- Обработать полученный элемент в Consumer

## Урок 2 - Reader-Writer

## Шаг 01:

### Миссия 1

Анна пишет систему учёта посещений читателей в библиотеке. Доступ к данным осуществляется через разделяемый ресурс. Какой подход наиболее безопасен для чтения и записи одновременно?

Подсказка: Подумайте, что делать, если много читателей, мало писателей.

#### Ответ:

- **Использовать механизм reader-writer lock**
- Синхронизировать весь метод на одном мониторе
- Запретить одновременно записи вовсе

## Шаг 02:

### Миссия 2

Борис реализует reader-writer решение. Что он должен обеспечить дополнительно помимо блокировки целостности данных?

Подсказка: Подумайте о справедливости между читателями и писателями.

#### Ответ:

- **Избежать голодания писателей**
- Допускать только одного читателя
- Запретить чтение при любой записи независимо от контекста

## Шаг 03:

### Миссия 3

Цель Ксении — оптимизировать throughput системы с высокой нагрузкой на чтение. Какой вариант стоит выбрать?

Подсказка: Большинство операций — чтения, записи немного.

**Ответ:**

- **Дать приоритет читателям**
- Всегда давать приоритет писателям
- Не использовать приоритет вообще

#### 🔮 Шаг 04:

##### Миссия 4

Иван настраивает базу, и читатели часто ждут, пока писатель закончит. Что он мог сделать неправильно?

Подсказка: Возможно, вы заблокировали слишком широко.

**Ответ:**

- **Использовал эксклюзивный lock для чтения**
- Неявно синхронизировал только часть кода
- Неправильно выставил флаг разрешения записи

#### 🔮 Шаг 05:

##### Миссия 5

Ольга внедряет тайм-аут для получения writer lock, чтобы избежать зависания. Что происходит, если тайм-аут истекает?

Подсказка: Не забывайте об обработке ошибок.

**Ответ:**

- **Бросается исключение или возвращается ошибка**
- Ждёт бесконечно
- Прерывает поток заставляя писать в другой ресурс

#### 🔮 Шаг 06:

##### Миссия 6

Сопоставьте понятие и его описание в контексте reader-writer:

Понятия:

**Ответ:**

- Reader-first политика
- Writer-first политика
- Starvation (голодание)
- Reentrancy (реэнтрантность)

Описания:

1. Когда писатели постоянно ждут, потому что читатели занимают ресурс
2. Когда поток может повторно войти в тот же lock
3. Читатели получают доступ раньше писателей
4. Запись имеет приоритет перед чтением

Подсказка: Подумайте, какие политики подавляют писателей и когда возможны повторные захваты.

- **Reader-first политика** — Читатели получают доступ раньше писателей
- **Writer-first политика** — Запись имеет приоритет перед чтением

- **Starvation (голодание)** — Когда писатели постоянно ждут, потому что читатели занимают ресурс
- **Reentrancy (реэнтрантность)** — Когда поток может повторно войти в тот же lock

## 🔮 Шаг 07:

### Миссия 7

Сопоставьте методы reader-writer класса с их описаниями:

Методы:

**Ответ:**

- `acquireReadLock()`
- `releaseReadLock()`
- `acquireWriteLock()`
- `releaseWriteLock()`

Описания:

1. Освобождение блокировки для записи
2. Получение блокировки для чтения
3. Получение блокировки для записи
4. Освобождение блокировки для чтения

Подсказка: Помните: чтение — shared, запись — exclusive, и каждый захват должен соответствовать освобождению.

- **`acquireReadLock()`** — Получение блокировки для чтения
- **`releaseReadLock()`** — Освобождение блокировки для чтения
- **`acquireWriteLock()`** — Получение блокировки для записи
- **`releaseWriteLock()`** — Освобождение блокировки для записи

## 🔮 Шаг 08:

### Миссия 8

Сопоставьте концептуальные узлы Reader-Writer с их ролью:

Узлы:

**Ответ:**

- Shared counter
- Write priority queue
- Read hold count
- Writer queue

Роли:

1. Счётчик активных читателей
2. Список ожидающих писателей
3. Очередь писателей, ожидающих write lock
4. Механизм предпочтения писателям

Подсказка: Кто ведёт счёт читателей? Что ставит в очередь писателей?

- **Shared counter** — Счётчик активных читателей
- **Write priority queue** — Механизм предпочтения писателям
- **Read hold count** — Список ожидающих писателей

- **Writer queue** — Очередь писателей, ожидающих write lock

## Шаг 09:

### Миссия 9

Расположите этапы обработки запроса на запись в правильном порядке:

1. Освободить write lock
2. Ждать пока все читатели отпустят read lock
3. Получить write lock
4. Обновить данные

Подсказка: Сначала подготовка, затем обновление, затем освобождение.

#### Ответ:

- Ждать пока все читатели отпустят read lock
- Получить write lock
- Обновить данные
- Освободить write lock

## Шаг 10:

### Миссия 10

Расположите шаги при запросе на чтение в контексте reader-writer:

1. Выполнить чтение данных
2. Освободить read lock
3. Получить read lock
4. Вернуть результат читателю

Подсказка: Захват → чтение → отпуск → возврат результата.

#### Ответ:

- Получить read lock
- Выполнить чтение данных
- Освободить read lock
- Вернуть результат читателю

## Шаг 11:

### Миссия 11

Расположите шаги «fair» подхода, когда и читатели, и писатели ожидают:

1. Проверить очередь писателей
2. **Если нет писателей** — дать читать
3. **Иначе** — дать первому писателю писать
4. Обработка освобождения lock и переход к следующему ожиданию

Подсказка: Fair означает учёт очереди писателей прежде, чем допускать новых читателей.

#### Ответ:

- Проверить очередь писателей
- **Если нет писателей** — дать читать
- **Иначе** — дать первому писателю писать
- Обработка освобождения lock и переход к следующему ожиданию

## 🔗 Шаг 12:

### Миссия 12

Расположите последовательность событий, если writer-запрос пришёл во время активных читателей (fair-подход):

1. Writer попадает в очередь ожидания
2. Когда все читатели отпущены, writer получает lock
3. Допускаются новые читатели только после писателя
4. Писатель обновляет данные и освобождает lock

Подсказка: Заметьте, что writer ждёт, пока читатели завершатся, и затем имеет преимущество перед новыми читателями.

**Ответ:**

- Writer попадает в очередь ожидания
- Когда все читатели отпущены, writer получает lock
- Писатель обновляет данные и освобождает lock
- Допускаются новые читатели только после писателя

## 🔗 Шаг 13:

### Миссия 13

Расположите следующие подходы от простейшего к наиболее сложному реализации:

1. Простой synchronized-блок без приоритетов
2. Reader-writer lock с reader-first
3. Reader-writer lock с writer-first
4. Reader-writer lock с fair-политикой и тайм-аутами

Чем больше вариаций — тем более сложна реализация.

**Ответ:**

- Простой synchronized-блок без приоритетов
- Reader-writer lock с reader-first
- Reader-writer lock с writer-first
- Reader-writer lock с fair-политикой и тайм-аутами

## 🔗 Шаг 14:

### Миссия 14

Расположите стадии отказа и recovery в reader-writer реализации, при сбое во время записи:

1. Освободить write lock (если захвачен)
2. Откатить изменения транзакции
3. Уведомить ожидающих потоков
4. Записать лог ошибки

Подсказка: Даже при сбое нужно корректно отпустить ресурсы и дать шанс другим.

**Ответ:**

- Откатить изменения транзакции
- Освободить write lock (если захвачен)
- Записать лог ошибки
- Уведомить ожидающих потоков

## Шаг 15:

### Миссия 15

Расположите фазы сложной reader-writer системы с очередями и тайм-аутами:

1. Проверка очереди писателей и установка приоритета
2. Попытка захвата read или write lock с тайм-аутом  
Ожидание разрешения в очереди
3. Обновление или чтение ресурса
4. Освобождение соответствующей блокировки и уведомление

Подсказка: Сначала политика, потом попытка, затем ожидание, потом действие, наконец освобождение.

#### Ответ:

- Проверка очереди писателей и установка приоритета
- Попытка захвата read или write lock с тайм-аутом
- Ожидание разрешения в очереди
- Обновление или чтение ресурса
- Освобождение соответствующей блокировки и уведомление

## Урок 3 - Thread-safe Singleton

## Шаг 01:

### Миссия 1

Олег хочет сделать класс ConfigurationManager с single instance. Какой базовый способ гарантирует единственный экземпляр при однопоточном исполнении?

Подсказка: Начинайте с простого варианта.

#### Ответ:

- **Создать приватный конструктор и статическое поле-синглтон**
- Создавать новый экземпляр при каждом вызове getInstance()
- Использовать public-конструктор и один глобальный объект

## Шаг 02:

### Миссия 2

Катя запускает приложение в многопоточном окружении. Какой риск при простом варианте singleton без синхронизации?

Подсказка: Подумайте о гонке при инициализации.

#### Ответ:

- **Создание двух экземпляров singleton одновременно**
- NullPointerException при getInstance()
- Блокировка при каждом методе класса

## Шаг 03:

### Миссия 3

Лев внедряет synchronized в метод getInstance(). Что он должен сделать, чтобы избежать лишней блокировки после инициализации?

Подсказка: Подумайте о проверке перед входом в synchronized.

**Ответ:**

- **Сделать двойную проверку (double-checked locking)**
- Синхронизировать всю инициализацию
- Не проверять перед synchronized

🔮 **Шаг 04:**

#### **Миссия 4**

Мария читает про volatile в double-checked locking. Почему поле instance объявляют volatile?

Подсказка: Подумайте о видимости и reordering.

**Ответ:**

- **Чтобы предотвратить полуручно сконструированный объект (reordering) и обеспечить видимость другим потокам**
- Чтобы автоматически сериализовать доступ
- Чтобы ускорить чтение поля

🔮 **Шаг 05:**

#### **Миссия 5**

Игорь задумал ленивую инициализацию с synchronized. Какой вариант предпочтительнее для производительности?

Подсказка: Используйте специальную конструкцию из Java.

**Ответ:**

- **Использовать Initialization-on-demand holder идиому**
- Использовать eager initialization всегда
- Синхронизировать каждый вызов getInstance()

🔮 **Шаг 06:**

#### **Миссия 6**

Сопоставьте термины и объяснения, связанные с thread-safe singleton:

Термины:

**Ответ:**

- Eager initialization
- Lazy initialization
- Double-checked locking
- Holder pattern

Объяснения:

1. Инициализация при загрузке класса
2. Отложенная инициализация при первом вызове getInstance()
3. Паттерн, использующий вложенный static-класс
4. Синхронизированная проверка с двойным условием

Подсказка: Подумайте, как и когда создаётся экземпляр в каждом подходе.

- **Eager initialization** — Инициализация при загрузке класса
- **Lazy initialization** — Отложенная инициализация при первом вызове getInstance()
- **Holder pattern** — Паттерн, использующий вложенный static-класс
- **Double-checked locking** — Синхронизированная проверка с двойным условием

## 🔗 Шаг 07:

### Миссия 7

Сопоставьте преимущества и недостатки подходов singleton:

Подход:

Ответ:

- Eager initialization
- Lazy synchronized getInstance()
- Double-checked locking
- Holder pattern

Характеристика:

1. Простой, но создаёт экземпляр сразу
2. Низкая производительность при частых вызовах
3. Сложнее реализовать корректно, но эффективен после инициализации
4. Беспроblemen, безопасен и эффективен

Подсказка: Подумайте про время создания и нагрузку на синхронизацию.

- **Eager initialization** — Простой, но создаёт экземпляр сразу
- **Lazy synchronized getInstance()** — Низкая производительность при частых вызовах
- **Double-checked locking** — Сложнее реализовать корректно, но эффективен после инициализации
- **Holder pattern** — Беспроblemen, безопасен и эффективен

## 🔗 Шаг 08:

### Миссия 8

Сопоставьте сценарии и подходящие методы создания singleton:

Сценарий:

Ответ:

- Класс нужен сразу при старте приложения
- Класс нужен редко, но многопоточно
- Необходима высокая производительность после инициализации
- Нужна безопасность без явных synchronized

Подход:

1. Eager initialization
2. Lazy synchronized getInstance()
3. Double-checked locking
4. Holder pattern

Подсказка: Подумайте, когда экземпляр создаётся и как избежать блокировки.

- **Класс нужен сразу при старте приложения** — Eager initialization
- **Класс нужен редко, но многопоточно** — Lazy synchronized getInstance()
- **Необходима высокая производительность после инициализации** — Double-checked locking
- **Нужна безопасность без явных synchronized** — Holder pattern

## 🔧 Шаг 09:

### Миссия 9

Сопоставьте проблемы, которые могут возникнуть, и методы защиты в singleton:

Проблема:

Ответ:

- Reordering создаёт неправильно сконструированный объект
- Несколько потоков одновременно инициализируют
- Низкая производительность из-за synchronized
- Отсутствие ленивости, когда объект редко нужен

Защита:

1. volatile + DCL
2. synchronized блокировка
3. Holder pattern / static inner
4. Eager initialization

Подсказка: Подумайте, что нейтрализует reordering, что даёт безопасность и что даёт простоту.

- **Reordering создаёт неправильно сконструированный объект** — volatile + DCL
- **Несколько потоков одновременно инициализируют** — synchronized блокировка
- **Низкая производительность из-за synchronized** — Holder pattern / static inner
- **Отсутствие ленивости, когда объект редко нужен** — Eager initialization

## 🔧 Шаг 10:

### Миссия 10

Расположите шаги при реализации singleton с double-checked locking:

1. Проверка instance != null без блокировки
2. Вход в synchronized блок
3. Повторная проверка внутри synchronized
4. Создание экземпляра
5. Возврат instance

Подсказка: Сначала быстрая проверка, затем безопасная инициализация.

Ответ:

- Проверка instance != null без блокировки
- Вход в synchronized блок
- Повторная проверка внутри synchronized
- Создание экземпляра
- Возврат instance

## 🔧 Шаг 11:

### Миссия 11

Расположите шаги использования holder pattern:

1. Ссылка на static inner класс с INSTANCE
2. Класс загружается только при первом обращении
3. Инициализация INSTANCE в inner классе
4. getInstance возвращает INSTANCE

Подсказка: Всё основано на механизме загрузки классов в Java.

**Ответ:**

- Ссылка на static inner класс с INSTANCE
- Класс загружается только при первом обращении
- Инициализация INSTANCE в inner классе
- getInstance возвращает INSTANCE

🔮 **Шаг 12:**

### Миссия 12

Расположите подходы singleton от простейшего к наиболее сложному по безопасности и ленивости:

1. Простой static field без синхронизации
2. Lazy synchronized getInstance()
3. Double-checked locking с volatile
4. Holder pattern
5. Enum singleton

Подсказка: Учитывайте сложность, безопасность и оптимальность.

**Ответ:**

- Простой static field без синхронизации
- Lazy synchronized getInstance()
- Double-checked locking с volatile
- Holder pattern
- Enum singleton

🔮 **Шаг 13:**

### Миссия 13

Расположите стадии, если enum singleton используется:

1. Загрузка enum класса
2. Инициализация единственного элемента enum
3. Вызов INSTANCE.method()
4. Возврат enum INSTANCE

Подсказка: Enum-singleton — самый безопасный от атак, загружается JVM.

**Ответ:**

- Загрузка enum класса
- Инициализация единственного элемента enum
- Вызов INSTANCE.method()
- Возврат enum INSTANCE

🔮 **Шаг 14:**

### Миссия 14

Расположите шаги предотвращения атаки reflection на singleton:

1. Проверить, не инициализирован ли INSTANCE в конструкторе
2. Бросить исключение, если попытка создания вторичного экземпляра
3. Сделать конструктор private

#### 4. Обработать возможные ошибки безопасности

Подсказка: Reflection может обойти private, надо защититься внутри.

**Ответ:**

- Проверить, не инициализирован ли INSTANCE в конструкторе
- Бросить исключение, если попытка создания вторичного экземпляра
- Сделать конструктор private
- Обработать возможные ошибки безопасности

### Шаг 15:

#### Миссия 15

Расположите шаги защиты от десериализации создания нового объекта singleton:

1. Реализовать method readResolve()
2. Вернуть INSTANCE в readResolve()
3. Десериализация пытается создать объект
4. Чтение потока вызывает readResolve, замещающий объект

Подсказка: readResolve часто используется для безопасного singleton.

**Ответ:**

- Реализовать method readResolve()
- Вернуть INSTANCE в readResolve()
- Десериализация пытается создать объект
- Чтение потока вызывает readResolve, замещающий объект

### Шаг 16:

#### Миссия 16

Расположите этапы полной защиты singleton от reflection и десериализации:

1. Сделать конструктор private с проверкой
2. Использовать enum или holder pattern
3. Имплементировать readResolve (если не enum)
4. Защититься от reflection через исключение в конструкторе
5. Обеспечить thread-safe инициализацию

Подсказка: Защитных мер может быть несколько, важно последовательность.

**Ответ:**

- Сделать конструктор private с проверкой
- Использовать enum или holder pattern
- Имплементировать readResolve (если не enum)
- Защититься от reflection через исключение в конструкторе
- Обеспечить thread-safe инициализацию

### Шаг 17:

#### Миссия 17

Расположите уровни зрелости реализации thread-safe singleton от базового к максимально защищённому:

1. Простой static field без синхронизации

2. Lazy synchronized getInstance()
3. Double-checked locking с volatile
4. Holder pattern
5. Enum singleton с защитой от reflection и десериализации

Подсказка: Каждая следующая версия добавляет новый уровень защиты.

**Ответ:**

- Простой static field без синхронизации
- Lazy synchronized getInstance()
- Double-checked locking с volatile
- Holder pattern
- Enum singleton с защитой от reflection и десериализации

## Урок 4 - Future и Promise паттерны

### Шаг 01:

#### Миссия 1

Анна запускает задачу на исполнение в другом потоке с помощью Future<Integer>, но забывает вызвать метод get(). Что произойдёт?

Подсказка: Future без get() — задача всё равно запустится, но вы не получите результат.

**Ответ:**

- **Задача выполнится, но результат никогда не будет использован**
- Задача не запустится
- Получится исключение сразу при запуске

### Шаг 02:

#### Миссия 2

Борис хочет отменить задачу, запущенную через Future, до её завершения. Какой метод он должен вызвать?

Подсказка: В API Future предусмотрен способ попытаться прервать задачу.

**Ответ:**

- **cancel(boolean mayInterruptIfRunning)**
- abort()
- stop()

### Шаг 03:

#### Миссия 3

Катя использует CompletableFuture, и хочет, чтобы после завершения первой задачи автоматически запустилась вторая. Какой метод ей нужен?

Подсказка: CompletableFuture поддерживает методы цепочки, например thenApply, thenAccept и др.

**Ответ:**

- **thenApply(...) или thenAccept(...) (в зависимости от возвращаемого типа)**
- whenComplete(...) сразу
- complete(...)

## 🔮 Шаг 04:

### Миссия 4

Иван хочет создать «обещание» (Promise) вручную и передать в него результат позже. Какой класс в Java стандартной библиотеки ближе всего к Promise?

Подсказка: Это тот же CompletableFuture, только в роли «контроллера» вы вручную завершаете.

**Ответ:**

- **CompletableFuture (вручную complete / completeExceptionally)**
- FutureTask
- Promise (нет такого класса в стандартной библиотеке)

## 🔮 Шаг 05:

### Миссия 5

Мария хочет объединить два независимых CompletableFuture<Integer> в одно, которое даёт сумму их результатов. Какой подход она выберет?

Подсказка: Есть метод, который принимает второй Future и функцию объединения.

**Ответ:**

- **thenCombine(second, (a, b) -> a + b)**
- thenMerge(second, sum)
- allOf(...)

## 🔮 Шаг 06:

### Миссия 6

Олег хочет дождаться завершения всех из списка CompletableFuture<Void>. Какой метод правильно использовать?

Подсказка: Для объединения многих futures есть вспомогательная функция.

**Ответ:**

- **CompletableFuture.allOf(...)**
- CompletableFuture.anyOf(...)
- CompletableFuture.joinAll(...)

## 🔮 Шаг 07:

### Миссия 7

Сопоставьте термин и определение, относящееся к шаблону Future / Promise.

Термины:

**Ответ:**

- Future
- Promise
- thenApply
- completeExceptionally

Определения:

1. Метод, позволяющий трансформировать результат Future
2. Объект, который вручную завершается либо успешно, либо с ошибкой
3. Интерфейс, представляющий результат асинхронной операции
4. Метод, завершает «обещание» с исключением

Подсказка: Promise — это активная сторона, Future — пассивная.

- **Future** — Интерфейс, представляющий результат асинхронной операции
- **Promise** — Объект, который вручную завершается либо успешно, либо с ошибкой
- **thenApply** — Метод, позволяющий трансформировать результат Future
- **completeExceptionally** — Метод, завершает «обещание» с исключением

## 🔗 Шаг 08:

### Миссия 8

Сопоставьте ситуации с методами CompletableFuture:

Ситуации:

**Ответ:**

- Обработать исключение, если future неудачно завершён
- Запустить действие после завершения, без возвращаемого значения
- Объединить два Future
- Запустить код независимо от успеха или ошибки

Методы:

1. handle()
2. thenAccept()
3. thenCombine()
4. whenComplete()

Подсказка: Обратите внимание: handle даёт возможность вернуть значение даже в случае исключения.

- **Обработать исключение, если future неудачно завершён** — handle()
- **Запустить действие после завершения, без возвращаемого значения** — thenAccept()
- **Объединить два Future** — thenCombine()
- **Запустить код независимо от успеха или ошибки** — whenComplete()

## 🔗 Шаг 09:

### Миссия 9

Сопоставьте сценарий и результат:

Сценарии:

**Ответ:**

- Promise завершился успешно до вызова get()
- Promise никогда не завершился и никто не вызвал complete или completeExceptionally
- Promise завершился исключением
- Future был отменён через cancel()

Результаты get()/join():

1. Вернёт значение или бросит ExecutionException
2. Зависание (блокировка) навсегда
3. Выбросит ExecutionException, оборачивая 원ную ошибку
4. Выбросит CancellationException

Подсказка: Что делает метод get(), если задача отменена или исключение было выброшено?

- **Promise завершился успешно до вызова get()** — Вернёт значение или бросит ExecutionException
- **Promise никогда не завершился и никто не вызвал complete или completeExceptionally** — Зависание (блокировка) навсегда
- **Promise завершился исключением** — Выбросит ExecutionException, оборачивая 원ную ошибку
- **Future был отменён через cancel()** — Выбросит CancellationException

## 🔮 Шаг 10:

### Миссия 10

Сопоставьте персонажа и сценарий использования Future/Promise:

Персонажи:

**Ответ:**

- Сергей, вызывающий API и ожидающий ответ
- Наташа, у которой есть результат заранее и она хочет дать его обещанию
- Дмитрий, который хочет запустить цепочку задач в цепочку
- Лена, которая хочет ждать любого из множества задач

Сценарии:

1. CompletableFuture.completedFuture(...) или immediate completion
2. thenCompose / thenApply цепочка
3. CompletableFuture.anyOf(...)
4. Запуск запроса и ожидание get()

Подсказка: Подумайте, кто что хочет сделать — ждать, сразу вернуть, объединить.

- **Сергей, вызывающий API и ожидающий ответ** — Запуск запроса и ожидание get()
- **Наташа, у которой есть результат заранее и она хочет дать его обещанию** — CompletableFuture.completedFuture(...) или immediate completion
- **Дмитрий, который хочет запустить цепочку задач в цепочку** — thenCompose / thenApply цепочка
- **Лена, которая хочет ждать любого из множества задач** — CompletableFuture.anyOf(...)

## 🔮 Шаг 11:

### Миссия 11

Сопоставьте методы и характер поведения:

Методы:

**Ответ:**

- join()
- get()
- get(long timeout, TimeUnit u)
- exceptionally()

Поведение:

1. Бросает unchecked исключение CompletionException при ошибке
2. Бросает checked ExecutionException и InterruptedException
3. Ожидает не более указанного времени или бросает TimeoutException
4. Позволяет вернуть альтернативный результат в случае исключения

Подсказка: Различие между join и get — checked vs unchecked исключения.

- **join()** — Бросает unchecked исключение CompletionException при ошибке
- **get()** — Бросает checked ExecutionException и InterruptedException
- **get(long timeout, TimeUnit u)** — Ожидает не более указанного времени или бросает TimeoutException
- **exceptionally()** — Позволяет вернуть альтернативный результат в случае исключения

## 🔧 Шаг 12:

### Миссия 12

Расположите этапы обработки асинхронной задачи с Promise / CompletableFuture:

1. Вручную вызвать complete или completeExceptionally на Promise
2. Клиент вызывает get / join на Future
3. Запустить задачу в отдельном потоке
4. Установить цепочку методов thenApply / thenCompose и др.

Подсказка: Сначала запускается задача, затем выстраиваются цепочки, затем завершается Promise, потом ждут результата.

#### Ответ:

- Запустить задачу в отдельном потоке
- Установить цепочку методов thenApply / thenCompose и др.
- Вручную вызвать complete или completeExceptionally на Promise
- Клиент вызывает get / join на Future

## 🔧 Шаг 13:

### Миссия 13

Расположите шаги, если вы хотите объединить три асинхронные задачи и получить результат, когда все завершатся:

1. Создать массив CompletableFuture<T>
2. Вызвать allOf(...) на массиве
3. После завершения allOf выполнить thenApply / thenCompose для получения списка результатов
4. Запустить каждую отдельную задачу и сохранить их в массив

Подсказка: Сначала вы готовите futures, потом запускаете, потом ждёте allOf, затем обрабатываете.

#### Ответ:

- Создать массив CompletableFuture
- Запустить каждую отдельную задачу и сохранить их в массив
- Вызвать allOf(...) на массиве
- После завершения allOf выполнить thenApply / thenCompose для получения списка результатов

## 🔧 Шаг 14:

### Миссия 14

Расположите методы в порядке повышения степени возможности обработки исключений (от наименее гибкого к наиболее гибкому):

1. thenApply

2. `whenComplete`
3. `exceptionally`
4. `handle`

Подсказка: `thenApply` не обрабатывает ошибки, `handle` даёт полную гибкость.

**Ответ:**

- `thenApply`
- `whenComplete`
- `exceptionally`
- `handle`

## Шаг 15:

### Миссия 15

Расположите стратегические этапы создания собственного Promise-оболочки:

1. Создать экземпляр `CompletableFuture` как внутренний «промис»
2. Предоставить внешний интерфейс, возвращающий `Future` клиентам
3. Внутри логики вычисления при успехе вызвать `complete`
4. Внутри логики при ошибке вызвать `completeExceptionally`
5. Запустить вычисление в отдельном потоке или пуле

Подсказка: Сначала создаётся контейнер, затем запускается вычисление, затем завершается.

**Ответ:**

- Создать экземпляр `CompletableFuture` как внутренний «промис»
- Предоставить внешний интерфейс, возвращающий `Future` клиентам
- Запустить вычисление в отдельном потоке или пуле
- Внутри логики вычисления при успехе вызвать `complete`
- Внутри логики при ошибке вызвать `completeExceptionally`

## Шаг 16:

### Миссия 16

Расположите этапы обработки ситуации `timeout` для `Future` с `get(timeout)`:

1. `get(timeout)` выбросит `TimeoutException`
2. Можно обработать исключение с помощью `exceptionally` / `handle`
3. Можно попытаться отменить задачу (`cancel`)
4. Можно запустить альтернативную ветку (`fallback`)

Подсказка: `Timeout` может быть пойман и обработан прежде, чем вы отмените или запустите `fallback`.

**Ответ:**

- `get(timeout)` выбросит `TimeoutException`
- Можно обработать исключение с помощью `exceptionally` / `handle`
- Можно попытаться отменить задачу (`cancel`)
- Можно запустить альтернативную ветку (`fallback`)

## Шаг 17:

### Миссия 17

Расположите логические шаги для реализации сценария: запустить N задач, дождаться любой первой завершившейся, использовать её результат, остальные отменить.

1. Вызвать `CompletableFuture.anyOf(...)` на массиве задач
2. Когда `anyOf` завершится, вызвать `cancel` на остальных futures
3. Запустить каждую задачу и собрать futures
4. После завершения `anyOf` получить результат и использовать его

Подсказка: `anyOf` даёт первую завершившуюся задачу, затем вы знаете, кого отменить.

**Ответ:**

- Запустить каждую задачу и собрать futures
- Вызвать `CompletableFuture.anyOf(...)` на массиве задач
- После завершения `anyOf` получить результат и использовать его
- Когда `anyOf` завершится, вызвать `cancel` на остальных futures

## Урок 5 - Thread Pool Pattern и Task Queue

### Шаг 01:

#### Миссия 1

Женя хочет ограничить число одновременно выполняющихся задач, чтобы не перегрузить систему. Какой паттерн она использует?

Подсказка: Подумайте об ограниченном пуле потоков.

**Ответ:**

- **Thread Pool (пул потоков)**
- Callback
- Observer

### Шаг 02:

#### Миссия 2

Игорь хочет, чтобы задачи, когда все потоки заняты, ставились в очередь на выполнение. Как называется эта очередь?

Подсказка: В пуле потоков используется структура, куда помещаются ожидающие задачи.

**Ответ:**

- **Task Queue (очередь задач)**
- Callback Queue
- Signal Queue

### Шаг 03:

#### Миссия 3

Света использует `Executors.newFixedThreadPool(5)`. Что это даёт?

Подсказка: Фиксированный пул гарантирует ограниченное число одновременных потоков.

**Ответ:**

- **Пул из ровно 5 потоков, остальные задачи ждут в очереди**
- Создаётся 5 очередей задач
- Пул с динамическим ростом до 5 потоков

## 🔮 Шаг 04:

### Миссия 4

Максим хочет пул, который сам увеличится при нагрузке и уменьшится при простое. Какой тип пула подойдёт?

Подсказка: Существует пул, который адаптируется.

**Ответ:**

- **Cached Thread Pool (пул с динамическим ростом)**
- Fixed Thread Pool
- Single Thread Executor

## 🔮 Шаг 05:

### Миссия 5

Оксана хочет, чтобы задачи сверх лимита порождали исключение, а не ждали. Какой RejectedExecutionHandler подойдёт?

Подсказка: Один из вариантов — выброс исключения при переполнении.

**Ответ:**

- **new ThreadPoolExecutor.AbortPolicy()**
- CallerRunsPolicy
- DiscardPolicy

## 🔮 Шаг 06:

### Миссия 6

Виктория использует ThreadPoolExecutor с SynchronousQueue как очередь. Что это значит?

Подсказка: SynchronousQueue не хранит задачи, она передаёт их напрямую потоку.

**Ответ:**

- **Каждая задача передаётся напрямую потоку, нет буфера**
- Очередь с неограниченным буфером
- Очередь с фиксированной ёмкостью

## 🔮 Шаг 07:

### Миссия 7

Сопоставьте термин пула и его характеристику:

Термины:

**Ответ:**

- FixedThreadPool
- CachedThreadPool
- ScheduledThreadPool
- SingleThreadExecutor

Характеристики:

1. Всегда один поток
2. Пул с фиксированным числом потоков
3. Пул, создающий потоки по потребности
4. Пул для планирования задач с задержкой или периодичностью

Подсказка: Подумайте, что делают разные factory-методы Executors.

- **FixedThreadPool** — Пул с фиксированным числом потоков
- **CachedThreadPool** — Пул, создающий потоки по потребности
- **ScheduledThreadPool** — Пул для планирования задач с задержкой или периодичностью
- **SingleThreadExecutor** — Всегда один поток

## 🔧 Шаг 08:

### Миссия 8

Сопоставьте элемент пула и его роль:

Элементы:

**Ответ:**

- corePoolSize
- maximumPoolSize
- workQueue
- keepAliveTime

Роли:

1. Максимальное число потоков в пуле
2. Время ожидания перед завершением лишнего потока
3. Минимальное устойчивое число потоков
4. Очередь задач, ожидающих исполнение

Подсказка: corePoolSize удерживается постоянно, а maximumPoolSize — предел.

- **corePoolSize** — Минимальное устойчивое число потоков
- **maximumPoolSize** — Максимальное число потоков в пуле
- **workQueue** — Очередь задач, ожидающих исполнение
- **keepAliveTime** — Время ожидания перед завершением лишнего потока

## 🔧 Шаг 09:

### Миссия 9

Сопоставьте политику отклонения и её поведение:

Политики:

**Ответ:**

- AbortPolicy
- CallerRunsPolicy
- DiscardPolicy
- DiscardOldestPolicy

Поведение при переполнении:

1. Выбросить RejectedExecutionException
2. Заставить вызывающий поток выполнять задачу
3. Отбросить новую задачу без уведомления
4. Отбросить старую задачу в очереди и вставить новую

Подсказка: Есть политика, что вызывает задачу в том же потоке.

- **AbortPolicy** — Выбросить RejectedExecutionException

- **CallerRunsPolicy** — Заставить вызывающий поток выполнять задачу
- **DiscardPolicy** — Отбросить новую задачу без уведомления
- **DiscardOldestPolicy** — Отбросить старую задачу в очереди и вставить новую

## 🔮 Шаг 10:

### Миссия 10

Сопоставьте сценарий и подход:

Сценарии:

**Ответ:**

- Нужно запретить рост пула сверх лимита
- Нужно, чтобы в пике создавались дополнительные потоки
- Нужно, чтобы потоки закрывались при простое
- Нужно сбалансировать очередь и запуск

Подходы:

1. Установить `fixedPool` и `bounded queue`
2. Использовать `cachedPool`
3. Использовать `keepAliveTime` & `allowCoreThreadTimeOut`
4. Подбирать `corePoolSize`, `maximumPoolSize` и размер `workQueue`

Подсказка: Некоторые потоки можно завершать при простое, другие — ограничивать.

- **Нужно запретить рост пула сверх лимита** — Установить `fixedPool` и `bounded queue`
- **Нужно, чтобы в пике создавались дополнительные потоки** — Использовать `cachedPool`
- **Нужно, чтобы потоки закрывались при простое** — Использовать `keepAliveTime` & `allowCoreThreadTimeOut`
- **Нужно сбалансировать очередь и запуск** — Подбирать `corePoolSize`, `maximumPoolSize` и размер `workQueue`

## 🔮 Шаг 11:

### Миссия 11

Расположите шаги настройки `ThreadPoolExecutor` вручную:

1. Выбор `corePoolSize` и `maximumPoolSize`
2. Выбор `workQueue` (типа очереди и ёмкости)
3. Указание `RejectedExecutionHandler`
4. Установка `keepAliveTime` и `TimeUnit`
5. Создание экземпляра `ThreadPoolExecutor`

Подсказка: Сначала параметры, потом построение.

**Ответ:**

- Выбор `corePoolSize` и `maximumPoolSize`
- Выбор `workQueue` (типа очереди и ёмкости)
- Указание `RejectedExecutionHandler`
- Установка `keepAliveTime` и `TimeUnit`
- Создание экземпляра `ThreadPoolExecutor`

## Шаг 12:

### Миссия 12

Расположите действия менеджера пула при поступлении задачи:

1. Если число работающих потоков  $< \text{corePoolSize}$ , создать новый поток для этой задачи
2. Иначе, попытаться положить задачу в `workQueue`
3. Если очередь полна и число потоков  $< \text{maximumPoolSize}$ , создать новый поток
4. Иначе, применить `RejectedExecutionHandler`

Подсказка: Логика из `ThreadPoolExecutor.execute` именно так проверяется.

**Ответ:**

- Если число работающих потоков  $< \text{corePoolSize}$ , создать новый поток для этой задачи
- Иначе, попытаться положить задачу в `workQueue`
- Если очередь полна и число потоков  $< \text{maximumPoolSize}$ , создать новый поток
- Иначе, применить `RejectedExecutionHandler`

## Шаг 13:

### Миссия 13

Расположите фазы жизненного цикла пула после `idle`-периода:

1. Пул остаётся активным `corePoolSize` потоков
2. Лишние потоки выше `corePoolSize` ждут время `keepAliveTime`
3. Потоки, простоявшие дольше `keepAliveTime`, завершаются
4. Если `allowCoreThreadTimeOut = true`, `core`-потоки тоже могут завершаться

Подсказка: Сперва проверяют высшие потоки, а потом — `core`, если разрешено.

**Ответ:**

- Пул остаётся активным `corePoolSize` потоков
- Лишние потоки выше `corePoolSize` ждут время `keepAliveTime`
- Потоки, простоявшие дольше `keepAliveTime`, завершаются
- Если `allowCoreThreadTimeOut = true`, `core`-потоки тоже могут завершаться

## Шаг 14:

### Миссия 14

Расположите шаги работы с очередью задач вручную:

1. Поставить задачу в очередь, если поток занят
2. Если очередь полна, принять решение по политике отклонения
3. Когда пул освобождается, взять задачу из очереди
4. Выполнить задачу в освободившемся потоке

Подсказка: Очередь используется только тогда, когда нет свободного потока.

**Ответ:**

- Поставить задачу в очередь, если поток занят
- Если очередь полна, принять решение по политике отклонения
- Когда пул освобождается, взять задачу из очереди
- Выполнить задачу в освободившемся потоке

## Шаг 15:

### Миссия 15

Расположите шаги эскалации реплик задачи при перегрузке:

1. Обнаружить, что очередь заполнена
2. Проверить возможность добавить поток до `maximumPoolSize`
3. **Если нельзя** — применить `RejectedExecutionHandler`
4. **Если можно** — создать новый поток и выполнить задачу

Подсказка: Пул сначала пытается масштабироваться вверх, прежде чем отклонить.

**Ответ:**

- Обнаружить, что очередь заполнена
- Проверить возможность добавить поток до `maximumPoolSize`
- **Если нельзя** — применить `RejectedExecutionHandler`
- **Если можно** — создать новый поток и выполнить задачу

## Шаг 16:

### Миссия 16

Расположите шаги завершения пула:

1. Вызвать `shutdown()`, чтобы больше не принимать задачи
2. Завершить выполнение уже принятых задач
3. Ожидать завершения всех задач (`awaitTermination`)
4. **Если не завершился** — вызвать `shutdownNow()` для принудительного завершения

Подсказка: Сначала мягкое завершение, потом форсирование.

**Ответ:**

- Вызвать `shutdown()`, чтобы больше не принимать задачи
- Завершить выполнение уже принятых задач
- Ожидать завершения всех задач (`awaitTermination`)
- **Если не завершился** — вызвать `shutdownNow()` для принудительного завершения

## Модуль 7 - Продвинутая синхронизация

### Урок 1 - Deadlock detection и prevention

## Шаг 01:

### Миссия 1

Аня и Борис работают над приложением банка. Один поток захватывает ресурс А, затем пытается захватить ресурс В. Другой поток — наоборот: В, потом А. Что происходит?

Подсказка: Два потока ждут освобождения ресурсов, которые каждый уже удерживает.

**Ответ:**

- Происходит **deadlock (взаимная блокировка)**
- Несколько потоков продолжают работать параллельно без проблем
- Происходит livelock
- Один из потоков бросает исключение

## 🔗 Шаг 02:

### Миссия 2

В проекте Тимур и Катя заметили, что потоки иногда блокируются навсегда. Какой подход поможет обнаружить deadlock динамически?

Подсказка: Нужно что-то, что анализирует состояние блокировок во время выполнения.

**Ответ:**

- **Использовать ThreadMXBean для выявления циклов в графе блокировок**
- Добавить таймауты ко всем synchronized-блокам
- Писать логи всех вызовов методов
- Увеличить количество потоков

## 🔗 Шаг 03:

### Миссия 3

Разработчики Мария и Иван обсуждают профилактику deadlock. Какой из вариантов предотвращения наиболее гарантированно работает?

Подсказка: Определите фиксированный порядок захвата ресурсов.

**Ответ:**

- **Всегда захватывать ресурсы в установленном порядке**
- Использовать меньше потоков чем ресурсов
- Не использовать synchronized вообще
- Отключить блокировки

## 🔗 Шаг 04:

### Миссия 4

Максим написал метод, где два ресурса блокируются через synchronized, но иногда deadlock не происходит. Что стоит добавить, чтобы повысить безопасность?

Подсказка: Подумайте про таймауты и возможности отката.

**Ответ:**

- **Использовать tryLock с таймаутом, а при неудаче отпустить ранее захваченные ресурсы**
- **Проверять, свободны ли ресурсы перед захватом**
- Добавить больше synchronized блоков
- Разрешить только один поток в системе

## 🔗 Шаг 05:

### Миссия 5

Сергей имеет три ресурса: R1, R2, R3. Поток А захватывает R1, затем R2. Поток В — R2, затем R3. Поток С — R3, затем R1. Что это за ситуация?

Подсказка: Если выследить круг захватов, видно замкнутую цепочку.

**Ответ:**

- **Троичный цикл deadlock между А, В и С**
- Race condition
- Livelock
- Starvation одного из потоков

## 🔧 Шаг 06:

### Миссия 6

Даша использует ReentrantLock без справедливости по умолчанию. Как это может повлиять на deadlock или блокировки?

Подсказка: Есть отличия между starvation и deadlock.

**Ответ:**

- **Повышается риск starvation, но deadlock напрямую не устраняется**
- Deadlock автоматически предотвращается
- Блокировки становятся fair
- Потоки не получают доступ к ресурсу никогда

## 🔧 Шаг 07:

### Миссия 7

📖 Задача: Сопоставьте название техники с её описанием в контексте предотвращения deadlock.

Название техники:

**Ответ:**

- Иерархический порядок захвата ресурсов
- Timed locks / таймауты
- Avoid nested synchronized
- Detect & recover

Описание:

1. Устанавливать фиксированную иерархию ресурсов, захватывать их по порядку
2. **Если ресурс не получен за заданное время** — освобождаются уже захваченные и пробуются позже
3. Минимизировать вложенные synchronized-блоки, чтобы меньше зависимостей
4. Обнаружение состояния, когда deadlock уже случился, и корректировка / рестарт

**Подсказка: Подумайте о каждой стратегии: что она делает до, во время, или после возможного deadlock.**

- **Иерархический порядок захвата ресурсов** — Устанавливать фиксированную иерархию ресурсов, захватывать их по порядку
- **Timed locks / таймауты** — Если ресурс не получен за заданное время — освобождаются уже захваченные и пробуются позже
- **Avoid nested synchronized** — Минимизировать вложенные synchronized-блоки, чтобы меньше зависимостей
- **Detect & recover** — Обнаружение состояния, когда deadlock уже случился, и корректировка / рестарт

## 🔧 Шаг 08:

### Миссия 8

📖 Задача: Сопоставьте тип deadlock'a с примером из истории программиста.

Типы:

**Ответ:**

- Static deadlock
- Dynamic deadlock

- Resource deadlock
- Livelock

Примеры:

1. Два потока используют synchronized на двух постоянных объектах в одном порядке всегда
2. Потоки захватывают объекты на основе входных данных, порядок меняется во время выполнения
3. Когда очередь заданий постоянно переупорядочивается, и потоки ничего не делают (меняется активность, но нет прогресса)
4. Ограниченный набор ресурсов, который может быть захвачен, потоки ждут освобождения

**Подсказка: Заметьте разницу между статической структурой и поведением во время исполнения.**

- **Static deadlock** — Два потока используют synchronized на двух постоянных объектах в одном порядке всегда
- **Dynamic deadlock** — Потоки захватывают объекты на основе входных данных, порядок меняется во время выполнения
- **Livelock** — Когда очередь заданий постоянно переупорядочивается, и потоки ничего не делают (меняется активность, но нет прогресса)
- **Resource deadlock** — Ограниченный набор ресурсов, который может быть захвачен, потоки ждут освобождения

## Шаг 09:

### Миссия 9

 Задача: Сопоставьте жизненную ситуацию с техникой разрешения deadlock.

Ситуация:

**Ответ:**

- Система зависает, когда два процесса ждут друг друга на постоянных объектах
- **Поток В держит ресурс и ждёт А, но А уже ждёт В** — обнаружили цикл во время исполнения
- Несколько потоков по-разному захватывают ресурсы, иногда deadlock, иногда нет
- **Поток захватывает один ресурс, не получает второй** — нужно откатиться

Техника:

1. Fixed locking order
2. Detect using JVM tools and restart
3. Consistent resource acquisition strategy
4. Rollback on failure / tryLock with timeout

**Подсказка: Каждая техника соответствует ситуации: до события, во время, после.**

- **Система зависает, когда два процесса ждут друг друга на постоянных объектах** — Fixed locking order
- **Поток В держит ресурс и ждёт А, но А уже ждёт В** — обнаружили цикл во время исполнения — Detect using JVM tools and restart
- **Несколько потоков по-разному захватывают ресурсы, иногда deadlock, иногда нет** — Consistent resource acquisition strategy
- **Поток захватывает один ресурс, не получает второй** — нужно откатиться — Rollback on failure / tryLock with timeout

## 🔔 Шаг 10:

### Миссия 10

📖 Задача: Сопоставьте термин безопасности многопоточности с примером deadlock/prevention.

Термины:

Ответ:

- Fair lock
- Non-fair lock
- Deadlock prevention
- Deadlock detection

Примеры:

1. Каждый поток получает ресурс в порядке очереди
2. Потоки могут получить ресурс в произвольном порядке, возможна нечестность
3. Система организована так, что цикл захвата ресурсов невозможен
4. Анализировать состояние и реагировать после возникновения deadlock

**Подсказка: Fair vs non-fair — про порядок, prevention vs detection — про до или после.**

- **Fair lock** — Каждый поток получает ресурс в порядке очереди
- **Non-fair lock** — Потоки могут получить ресурс в произвольном порядке, возможна нечестность
- **Deadlock prevention** — Система организована так, что цикл захвата ресурсов невозможен
- **Deadlock detection** — Анализировать состояние и реагировать после возникновения deadlock

## 🔔 Шаг 11:

### Миссия 11

Упорядочите шаги отладки deadlock'а, как если бы вы были программистом, обнаружившим зависание системы.

1. Собрать стек вызовов всех потоков
2. Идентифицировать ресурсы, которые удерживаются и ждутся
3. Определить циклическую зависимость между потоками
4. Использовать инструменты JVM (ThreadMXBean и др.) для анализа блокировок
5. Внедрить превентивную стратегию (фиксированный порядок, таймауты и др.)

**Подсказка: Сперва понять, что происходит, затем анализ, затем решение на будущее.**

Ответ:

- Собрать стек вызовов всех потоков
- Идентифицировать ресурсы, которые удерживаются и ждутся
- Определить циклическую зависимость между потоками
- Использовать инструменты JVM (ThreadMXBean и др.) для анализа блокировок
- Внедрить превентивную стратегию (фиксированный порядок, таймауты и др.)

## 🔔 Шаг 12:

### Миссия 12

От Софьи поступил код, в котором возможен deadlock. Расположите исправления по приоритету, от наиболее критичного к менее критичному.

1. Добавить таймауты к блокировкам
2. Обеспечить захват ресурсов в едином порядке во всех потоках
3. Провести код-ревью и обозначить потенциально опасные synchronized-блоки
4. Логировать попытки блокировок и живые потоки
5. Минимизировать зону, где захватывается более чем один ресурс

**Подсказка: Что можно сделать сразу, что требует плана и анализа.**

**Ответ:**

- Обеспечить захват ресурсов в едином порядке во всех потоках
- Минимизировать зону, где захватывается более чем один ресурс
- Добавить таймауты к блокировкам
- Провести код-ревью и обозначить потенциально опасные synchronized-блоки
- Логировать попытки блокировок и живые потоки

### Шаг 13:

#### **Миссия 13**

Представьте, что у вас есть три потока и два ресурса. Вы хотите применить стратегию предотвращения deadlock при выполнении операций. Упорядочите действия, которые должен предпринимать поток при попытке захватить ресурсы.

1. Попытаться захватить первый ресурс с таймаутом
2. Если не удалось, отпустить всё и попробовать позже
3. При удаче захватить второй ресурс с таймаутом
4. После выполнения работы отпустить оба ресурса
5. Документировать порядок захвата ресурсов заранее

**Подсказка: Сначала подготовка (документирование), потом попытки, потом откат, потом освобождение.**

**Ответ:**

- Документировать порядок захвата ресурсов заранее
- Попытаться захватить первый ресурс с таймаутом
- Если не удалось, отпустить всё и попробовать позже
- При удаче захватить второй ресурс с таймаутом
- После выполнения работы отпустить оба ресурса

### Шаг 14:

#### **Миссия 14**

Команда WS считает, что в их системе rare deadlock'ы случаются из-за смешанных стилей блокировки. Упорядочите мероприятия по улучшению архитектуры, чтобы устранить такие случаи.

1. Определить все объекты, которые используются как монитор
2. Ввести общий интерфейс или абстракцию доступа к ресурсам
3. Заменить synchronized на ReentrantLock с возможностью таймаута и справедливости там, где нужно
4. Обеспечить, что захват ресурсов происходит в едином порядке по интерфейсу/абстракции
5. Провести нагрузочное тестирование и симуляцию deadlock'ов

**Подсказка: Сначала выявление, затем изменение API, затем проверка в тестах.**

**Ответ:**

- Определить все объекты, которые используются как монитор
- Ввести общий интерфейс или абстракцию доступа к ресурсам
- Обеспечить, что захват ресурсов происходит в едином порядке по интерфейсу/абстракции
- Заменить synchronized на ReentrantLock с возможностью таймаута и справедливости там, где нужно
- Провести нагрузочное тестирование и симуляцию deadlock'ов

## Шаг 15:

### **Миссия 15**

Когда вы пишете библиотеку, вам важно предотвращать deadlock в API пользователя. Расположите стадии ревью API в порядке внедрения безопасности.

1. Определить порядок захвата ресурсов, который библиотека будет соблюдать
2. Документировать правила захвата ресурсов для пользователей
3. Предоставить версии методов с таймаутом / tryLock
4. Обеспечить справедливую блокировку, если ожидаются конкурирующие потоки
5. Написать тесты, имитирующие deadlock'ы и их отсутствие при правильном использовании

**Подсказка: Безопасность API зиждется на контракте, возможности и проверках.**

**Ответ:**

- Определить порядок захвата ресурсов, который библиотека будет соблюдать
- Документировать правила захвата ресурсов для пользователей
- Предоставить версии методов с таймаутом / tryLock
- Обеспечить справедливую блокировку, если ожидаются конкурирующие потоки
- Написать тесты, имитирующие deadlock'ы и их отсутствие при правильном использовании

## Шаг 16:

### **Миссия 16**

Вы — архитектор крупного проекта. Упорядочите внедрение политики предотвращения deadlock в жизненном цикле проекта.

1. Аудит существующих модулей на предмет потенциальных deadlock'ов
2. Обучение команды принципам deadlock prevention and detection
3. Введение соглашений о порядке захвата ресурсов и использовании lock'ов
4. Интеграция таймаутов и проверки tryLock где возможно
5. Мониторинг и логирование блокировок в проде
6. Регулярные тесты нестабильности и сценариев deadlock

**Подсказка: Процесс: сначала оценка, затем правила, затем инструменты и проверки, затем мониторинг.**

**Ответ:**

- Аудит существующих модулей на предмет потенциальных deadlock'ов
- Обучение команды принципам deadlock prevention and detection
- Введение соглашений о порядке захвата ресурсов и использовании lock'ов
- Интеграция таймаутов и проверки tryLock где возможно
- Мониторинг и логирование блокировок в проде
- Регулярные тесты нестабильности и сценариев deadlock

## Урок 2 - Livelock и Starvation

### Шаг 01:

#### Миссия 1

Никита пишет сервис, где два потока обмениваются флагами: если один уступит, другой снова уступает, и оба продолжают крутиться без прогресса. Что это за ситуация?

Подсказка: Оба потока активны, но ни один не делает полезной работы.

**Ответ:**

- **Livelock (живая блокировка)**
- Deadlock (взаимная блокировка)
- Starvation (голодание)
- Race condition

### Шаг 02:

#### Миссия 2

В коде Алисы один поток постоянно получает доступ к ресурсу, а другой почти никогда. Как называется такая проблема?

Подсказка: Один поток «забывает» ресурс, другой не может его получить.

**Ответ:**

- **Starvation (голодание)**
- Livelock
- Deadlock
- Priority inversion

### Шаг 03:

#### Миссия 3

В проекте Олега два потока передают мячик: когда один делает шаг, другой делает шаг, и так они бесконечно передают мяч без продвижения задачи. Что это?

Подсказка: Они активно взаимодействуют, но задача не продвигается.

**Ответ:**

- **Livelock**
- Starvation
- Deadlock
- Busy waiting

### Шаг 04:

#### Миссия 4

В команде Лизы поток с низким приоритетом почти никогда не получает CPU, хотя ресурсы свободны. Что это за ситуация?

Подсказка: Это не deadlock — один поток просто не получает шанс.

**Ответ:**

- **Starvation (голодание)**
- Livelock
- Deadlock
- Priority inversion

## 🔗 Шаг 05:

### Миссия 5

Игорь наблюдает, что два потока бесконечно выполняют обратные уступки: каждый раз, когда один предлагает уступить, другой предлагает уступить. Что может помочь избежать такого поведения?

Подсказка: Нужно ограничить бесконечный обмен уступками или ввести твердую логику.

**Ответ:**

- **Добавить рандомные задержки или backoff стратегию**
- Увеличить приоритет обоих потоков
- Перевести на synchronized
- Убрать все блокировки

## 🔗 Шаг 06:

### Миссия 6

📖 Задача: Сопоставьте поведение с правильным термином (livelock, starvation или deadlock).

Поведение:

**Ответ:**

- Потоки активно передают управление друг другу, но никаких задач не выполняют
- Один поток никогда не получает ресурс из-за постоянной конкуренции
- Два потока ждут друг друга навсегда
- Потоки иногда блокируются, иногда продолжают работу

Термины:

1. Livelock
2. Starvation
3. Deadlock
4. Переход между режимами (mixed behavior)

**Подсказка: Подумайте, активны ли потоки, и делают ли они прогресс.**

- **Потоки активно передают управление друг другу, но никаких задач не выполняют** — Livelock
- **Один поток никогда не получает ресурс из-за постоянной конкуренции** — Starvation
- **Два потока ждут друг друга навсегда** — Deadlock
- **Потоки иногда блокируются, иногда продолжают работу** — Переход между режимами (mixed behavior)

## 🔗 Шаг 07:

### Миссия 7

📖 Задача: Сопоставьте механизмы с тем, как они помогают уменьшить риск starvation или livelock.

Механизмы:

**Ответ:**

- Приоритетное вращение (priority aging)
- Backoff / exponential delay
- Fair locks / очереди ожидания
- Ограничение числа уступок (yield) подряд

Эффект:

1. Поток с низким приоритетом со временем повышает свой приоритет
2. После неудачи ждать немного и пробовать снова, уменьшая шанс livelock
3. Гарантировать, что все потоки получают ресурс по очереди
4. Не позволять одному потоку бесконечно уступать другому

**Подсказка: Механизм должен делать жизнь «голодного» потока лучше или мешать бесконечной уступке.**

- **Приоритетное вращение (priority aging)** — Поток с низким приоритетом со временем повышает свой приоритет
- **Backoff / exponential delay** — После неудачи ждать немного и пробовать снова, уменьшая шанс livelock
- **Fair locks / очереди ожидания** — Гарантировать, что все потоки получают ресурс по очереди
- **Ограничение числа уступок (yield) подряд** — Не позволять одному потоку бесконечно уступать другому

## 🔗 Шаг 08:

### Миссия 8

📖 Задача: Сопоставьте Java-паттерн / подход с тем, как он помогает избежать starvation или livelock.

Подход:

**Ответ:**

- ReentrantLock с fair = true
- tryLock с таймаутом
- Backoff стратегия (задержка при конфликте)
- Priority aging (увеличение приоритета)

Роль:

1. Гарантирует очередность доступа
2. Позволяет избежать бесконечных уступок
3. Уменьшает вероятность livelock, давая время
4. Позволяет «голодным» потокам постепенно получить ресурс

**Подсказка: Подумайте, что делает каждый подход: справедливость, откат, пауза, компенсация приоритета.**

- **ReentrantLock с fair = true** — Гарантирует очередность доступа
- **tryLock с таймаутом** — Позволяет избежать бесконечных уступок
- **Backoff стратегия (задержка при конфликте)** — Уменьшает вероятность livelock, давая время
- **Priority aging (увеличение приоритета)** — Позволяет «голодным» потокам постепенно получить ресурс

## 🔗 Шаг 09:

### Миссия 9

Упорядочите шаги расследования ситуации, когда в приложении появляется livelock:

1. Наблюдать поведение потоков и идентифицировать, что они не делают прогресс
2. Собрать трассировки и логи взаимных уступок / yield / backoff

3. Проанализировать последовательность уступок между потоками  
Внедрить backoff или ограничение уступок
4. Мониторить систему, чтобы убедиться, что livelock больше не повторяется

**Подсказка: Сначала нужно увидеть проблему, затем изучить, потом исправить и проверить.**

**Ответ:**

- Наблюдать поведение потоков и идентифицировать, что они не делают прогресс
- Собрать трассировки и логи взаимных уступок / yield / backoff
- Проанализировать последовательность уступок между потоками
- Внедрить backoff или ограничение уступок
- Мониторить систему, чтобы убедиться, что livelock больше не повторяется

## Шаг 10:

### **Миссия 10**

Разработчики столкнулись и с starvation, и с livelock. Упорядочите мероприятия по приоритетности внедрения защиты:

1. Внедрить fair-блокировки там, где возможно
2. Добавить backoff-стратегии / задержки при неудаче
3. Ограничить число yield / уступок подряд
4. Использовать приоритетное старение (priority aging) для потоков
5. Мониторить и логировать случаи длительного ожидания

**Подсказка: Начните с простых, надежных решений, затем к более адаптивным, и не забудьте тестирование и мониторинг.**

**Ответ:**

- Внедрить fair-блокировки там, где возможно
- Добавить backoff-стратегии / задержки при неудаче
- Ограничить число yield / уступок подряд
- Использовать приоритетное старение (priority aging) для потоков
- Мониторить и логировать случаи длительного ожидания

## Шаг 11:

### **Миссия 11**

Вы хотите улучшить систему с риском starvation. Упорядочите шаги от анализа до действия:

1. Проанализировать логи о времени ожидания потоков
2. Выявить потоки, которые ждут чересчур долго
3. Внедрить priority aging или справедливые очереди
4. Тестировать систему на нагрузки и выявлять новые случаи starvation
5. Мониторить и уведомлять об отклонениях ожидания

**Подсказка: Без анализа нет понимания, какие потоки страдают — начните с метрик и логов.**

**Ответ:**

- Проанализировать логи о времени ожидания потоков
- Выявить потоки, которые ждут чересчур долго
- Внедрить priority aging или справедливые очереди
- Тестировать систему на нагрузки и выявлять новые случаи starvation

- Мониторить и уведомлять об отклонениях ожидания

## 🔗 Шаг 12:

### Миссия 12

Команда внедряет три решения: fair-блокировки, backoff, priority aging. Упорядочите, в каком порядке вы бы применили их на практике.

1. Сначала заменить основные блокировки на fair-варианты
2. Затем добавить backoff/рандомные задержки при конфликте
3. Наконец, применить priority aging для потоков, которые остаются голодными

**Подсказка: Начинайте с гарантированных свойств, затем позволить адаптацию, потом компенсировать долгосрочную несправедливость.**

**Ответ:**

- Сначала заменить основные блокировки на fair-варианты
- Затем добавить backoff/рандомные задержки при конфликте
- Наконец, применить priority aging для потоков, которые остаются голодными

## 🔗 Шаг 13:

### Миссия 13

Для случая смешанных livelock & starvation упорядочите интегрированные шаги решения:

1. Мониторинг и логирование обеих проблем (задержки и уступки)
2. Внедрение fair / очередей для блокировок
3. Добавление backoff / задержек при конфликтах
4. Применение priority aging для «голодных» потоков
5. Тестирование на сценарии, когда одновременно появляются и livelock, и starvation

**Подсказка: Решение должно охватить все виды проблем — сначала наблюдать, затем обеспечить справедливость, затем адаптировать, затем тестировать.**

**Ответ:**

- Мониторинг и логирование обеих проблем (задержки и уступки)
- Внедрение fair / очередей для блокировок
- Добавление backoff / задержек при конфликтах
- Применение priority aging для «голодных» потоков
- Тестирование на сценарии, когда одновременно появляются и livelock, и starvation

## 🔗 Шаг 14:

### Миссия 14

Когда вы проектируете систему, где конкуренция потоков важна, упорядочите лучшие практики против starvation и livelock:

1. Использование fair-блокировок по умолчанию там, где возможно
2. Предоставление API с tryLock и таймаутами
3. Реализация backoff-стратегий при повторных попытках захвата
4. Priority aging для долгождующих потоков
5. Мониторинг и алерты при аномально длительном ожидании

**Подсказка: При проектировании защит у вас должна быть многоуровневая защита: от простого к адаптивному.**

**Ответ:**

- Использование fair-блокировок по умолчанию там, где возможно
- Предоставление API с tryLock и таймаутами
- Реализация backoff-стратегий при повторных попытках захвата
- Priority aging для долгождущих потоков
- Мониторинг и алерты при аномально длительном ожидании

 **Шаг 15:**

**Миссия 15**

Костя внедряет защиту от starvation и livelock в существующую систему. Упорядочите этапы внедрения:

1. Добавить метрики и логирование ожиданий и количества уступок
2. Проанализировать где действительно проявляются проблемы
3. Внедрить fair-блокировки или очереди в критичных местах
4. Добавить backoff / задержки при konflikтах
5. Использовать priority aging там, где после предыдущих мер есть голодание

**Подсказка: Всегда начните с измерения, затем постепенное улучшение, а не всё сразу.**

**Ответ:**

- Добавить метрики и логирование ожиданий и количества уступок
- Проанализировать где действительно проявляются проблемы
- Внедрить fair-блокировки или очереди в критичных местах
- Добавить backoff / задержки при konflikтах
- Использовать priority aging там, где после предыдущих мер есть голодание

 **Урок 3 - Phasers, CountdownLatch, CyclicBarrier**

 **Шаг 01:**

**Миссия 1**

Павел пишет простую кооперативную игру: все игроки должны нажать «готов», прежде чем начнётся следующий раунд. Какой синхронизатор из Java подходит лучше всего?

Подсказка: Нужно, чтобы потоки ждали друг друга ровно один раз за раунд.

**Ответ:**

- **CyclicBarrier**
- CountdownLatch
- Semaphore
- ReentrantLock

 **Шаг 02:**

**Миссия 2**

Ольга запускает серию фоновых задач и хочет дождаться, что все они завершатся, прежде чем продолжить. Какой синхронизатор ей выбрать?

Подсказка: Нужно, чтобы основной поток ждал только один раз, когда все задачи закончатся.

**Ответ:**

- **CountDownLatch**
- CyclicBarrier

- Exchanger
- SynchronousQueue

### 🔧 Шаг 03:

#### Миссия 3

Анна использует `CyclicBarrier`, но один из потоков вышел с исключением до `barrier.await()`. Что произойдёт?

Подсказка: Как `CyclicBarrier` реагирует, если один участник не доходит до барьера.

**Ответ:**

- **Остальные потоки заблокируются навсегда, если не отменить `barrier`**
- Барьер автоматически сбросится
- Все потоки продолжат работу
- Возникнет deadlock

### 🔧 Шаг 04:

#### Миссия 4

📖 Задача: Сопоставьте синхронизатор с его свойством, характерным именно для него.

Синхронизатор:

**Ответ:**

- `CountDownLatch`
- `CyclicBarrier`
- `Phaser`

Свойство:

1. Одноразовое ожидание всех участников
2. Барьер, который можно сбрасывать и использовать повторно
3. Поддержка динамического присоединения и отписки участников к фазам

**Подсказка: Подумайте, какой синхронизатор позволяет менять число участников во время работы.**

- **`CountDownLatch`** — Одноразовое ожидание всех участников
- **`CyclicBarrier`** — Барьер, который можно сбрасывать и использовать повторно
- **`Phaser`** — Поддержка динамического присоединения и отписки участников к фазам

### 🔧 Шаг 05:

#### Миссия 5

📖 Задача: Сопоставьте участника (роль) с тем, как он взаимодействует с `Phaser`.

Роль:

**Ответ:**

- Главный поток (coordinator)
- Рабочий поток
- Новый поток, присоединяющийся на лету

Действие:

1. Вызывает `arriveAndAwaitAdvance()` для синхронизации фаз
2. Вначале регистрируется через `register()` после создания `Phaser`

3. После завершения работы вызывает `arriveAndDeregister()`

**Подсказка: Подумайте, что делает координатор, что делают рабочие, и что если поток добавляется динамически.**

- **Главный поток (coordinator)** — Вызывает `arriveAndAwaitAdvance()` для синхронизации фаз
- **Рабочий поток** — После завершения работы вызывает `arriveAndDeregister()`
- **Новый поток, присоединяющийся на лету** — Вначале регистрируется через `register()` после создания `Phaser`

## Шаг 06:

### Миссия 6

 Задача: Сопоставьте класс / интерфейс с его ролью при работе с `CyclicBarrier` или `Phaser`.

Класс / интерфейс:

**Ответ:**

- `Callable / Runnable`
- `BrokenBarrierException`
- `Phaser`
- `BarrierAction` (`Runnable` для `CyclicBarrier`)

Роль:

1. Исключение, сигнализирующее, что барьер нарушен
2. Код, который выполняется после достижения барьера всеми участниками
3. Класс, управляющий фазами и регистрацией участников
4. **Рабочая задача** — участник синхронизации

**Подсказка: Подумайте, что делает каждый элемент: кто участвует, кто управляет, кто обрабатывает ошибку, кто запускает действие после барьера.**

- **`Callable / Runnable`** — Рабочая задача — участник синхронизации
- **`BrokenBarrierException`** — Исключение, сигнализирующее, что барьер нарушен
- **`Phaser`** — Класс, управляющий фазами и регистрацией участников
- **`BarrierAction` (`Runnable` для `CyclicBarrier`)** — Код, который выполняется после достижения барьера всеми участниками

## Шаг 07:

### Миссия 7

 Задача: Сопоставьте сценарий с предпочтительным синхронизатором.

Сценарий:

**Ответ:**

- Запустить N задач и ждать окончания, но не повторять
- Синхронизировать фазовый алгоритм с динамическим числом участников
- Синхронизировать N потоков по шагам, многократно

Предпочтительный синхронизатор:

1. `CountDownLatch`
2. `Phaser`
3. `CyclicBarrier`

**Подсказка: Посмотрите, нужен ли многократный барьер или фиксированный одноразовый счётчик.**

- Запустить N задач и ждать окончания, но не повторять — `CountDownLatch`
- Синхронизировать фазовый алгоритм с динамическим числом участников — `Phaser`
- Синхронизировать N потоков по шагам, многократно — `CyclicBarrier`

## Шаг 08:

### Миссия 8

Упорядочите этапы, как программист диагностирует проблему с `CyclicBarrier`, который «застрял»:

1. Проверить, все ли потоки вызвали `await()`
2. Проверить, не было ли исключения (`BrokenBarrierException`) в каком-то потоке
3. Логировать время ожидания на `barrier.await()`
4. Добавить таймаут в `await()` вызовы
5. При необходимости сбросить `barrier (reset)` или отменить его

**Подсказка: Сначала понять, что происходит, затем смягчить, затем восстановить работу.**

**Ответ:**

- Проверить, все ли потоки вызвали `await()`
- Проверить, не было ли исключения (`BrokenBarrierException`) в каком потоке
- Логировать время ожидания на `barrier.await()`
- Добавить таймаут в `await()` вызовы
- При необходимости сбросить `barrier (reset)` или отменить его

## Шаг 09:

### Миссия 9

Упорядочите шаги при отладке `Phaser`, если фазы не продвигаются:

1. Проверить, зарегистрированы ли все ожидаемые участники (`registered parties`)
2. Логировать вызовы `arrive()` и `awaitAdvance()`
3. Выяснить, не произошло ли `deregister()` раньше времени
4. Проверить, не заблокирован ли какой-то поток внутри фазы
5. Если нужно, вызвать `forceTermination()` или вручную продвинуть `Phaser`

**Подсказка: Сначала проверьте конфигурацию участников, затем логи, затем вмешательство.**

**Ответ:**

- Проверить, зарегистрированы ли все ожидаемые участники (`registered parties`)
- Логировать вызовы `arrive()` и `awaitAdvance()`
- Выяснить, не произошло ли `deregister()` раньше времени
- Проверить, не заблокирован ли какой-то поток внутри фазы
- Если нужно, вызвать `forceTermination()` или вручную продвинуть `Phaser`

## Шаг 10:

### Миссия 10

Команда интеграции тестирует модуль с Phaser и CyclicBarrier. Упорядочите подходы, как минимизировать риски ошибок:

1. Писать юнит-тесты для фаз (arrive, await) с малым числом участников
2. Добавить стресс-тесты с большим числом потоков и случайными задержками
3. Логировать участки задержек и времени ожидания
4. Пробовать крайние случаи: один поток сбрасывает фазу / отписывается
5. Запустить тест в реальном окружении с нагрузкой

**Подсказка: Начинайте с простого, затем проверяйте крайние случаи, затем нагрузочные сценарии.**

**Ответ:**

- Писать юнит-тесты для фаз (arrive, await) с малым числом участников
- Добавить стресс-тесты с большим числом потоков и случайными задержками
- Логировать участки задержек и времени ожидания
- Пробовать крайние случаи: один поток сбрасывает фазу / отписывается
- Запустить тест в реальном окружении с нагрузкой

### Шаг 11:

#### **Миссия 11**

Когда в модуле используется Phaser с динамическим числом участников, упорядочите шаги, как безопасно отписать поток:

1. Поток вызывает arriveAndDeregister() вместо await
2. Поток завершает свою работу после вызова arriveAndDeregister()
3. Остальные потоки продолжают во второй фазе без учёта отписавшегося
4. Phaser автоматически уменьшит число parties для дальнейших фаз
5. Никаких дополнительных действий не требуется от других потоков

**Подсказка: Отписка должна быть аккуратной: сначала сигнал фазы, затем исключение из числа участников.**

**Ответ:**

- Поток вызывает arriveAndDeregister() вместо await
- Поток завершает свою работу после вызова arriveAndDeregister()
- Остальные потоки продолжают во второй фазе без учёта отписавшегося
- Phaser автоматически уменьшит число parties для дальнейших фаз
- Никаких дополнительных действий не требуется от других потоков

### Шаг 12:

#### **Миссия 12**

В крупном сервисе применяется комбинация CountdownLatch и Phaser. Упорядочите, как выстроить инициализацию и этапы:

1. Создать CountdownLatch, равный числу начальных задач
2. Запустить потоки, каждая задача при старте вызывает latch.countDown()
3. Основной поток ждёт latch.await(), затем инициализирует Phaser для фазовой работы
4. Потоки выполняют фазы при помощи Phaser
5. После фазовой работы можно снова перейти на latch для новой порции задач

**Подсказка: Сначала дождаться запуска всех задач, затем перейти в фазовый режим.**

**Ответ:**

- Создать CountdownLatch, равный числу начальных задач
- Запустить потоки, каждая задача при старте вызывает latch.countDown()
- Основной поток ждёт latch.await(), затем инициализирует Phaser для фазовой работы
- Потоки выполняют фазы при помощи Phaser
- После фазовой работы можно снова перейти на latch для новой порции задач

 **Шаг 13:**

**Миссия 13**

В распределённой тестовой платформе используются циклы с CyclicBarrier и динамическое подключение с Phaser. Упорядочите архитектурные этапы интеграции:

1. Внедрить базовый механизм с CyclicBarrier для синхронизации шагов
2. Добавить регистрирующиеся потоки через Phaser для дополнительных модулей
3. Обеспечить, что новые модули регистрируются на фазы правильно
4. **Обрабатывать случаи, когда модуль отключается** — deregister()
5. Обеспечить сброс CyclicBarrier (reset) для начала нового цикла при необходимости
6. Логировать и проверять нарушения синхронизации на границах фаз

**Подсказка: Сначала запустите простую модель, затем гибкость, затем устойчивость и мониторинг.**

**Ответ:**

- Внедрить базовый механизм с CyclicBarrier для синхронизации шагов
- Добавить регистрирующиеся потоки через Phaser для дополнительных модулей
- Обеспечить, что новые модули регистрируются на фазы правильно
- **Обрабатывать случаи, когда модуль отключается** — deregister()
- Обеспечить сброс CyclicBarrier (reset) для начала нового цикла при необходимости
- Логировать и проверять нарушения синхронизации на границах фаз

 **Шаг 14:**

**Миссия 14**

Проект использует Phaser для координатора задач и CyclicBarrier для групповых шагов внутри фазы. Упорядочите проверку инициализации:

1. Создать Phaser с начальными parties = 1 (координатор)
2. Для каждого модуля вызвать phaser.register()
3. Внутри фазы использовать CyclicBarrier для синхронизации подзадач модуля
4. По завершении модуля вызвать arriveAndDeregister()
5. Координатор вызывает arriveAndAwaitAdvance() для каждой фазы

**Подсказка: Убедитесь, что регистрация участников происходит до начала фазы, и отписка после завершения.**

**Ответ:**

- Создать Phaser с начальными parties = 1 (координатор)
- Для каждого модуля вызвать phaser.register()
- Внутри фазы использовать CyclicBarrier для синхронизации подзадач модуля
- По завершении модуля вызвать arriveAndDeregister()
- Координатор вызывает arriveAndAwaitAdvance() для каждой фазы

## Шаг 15:

### Миссия 15

Вы — архитектор системы обработки потоковых данных и хотите комбинировать Phaser, CountdownLatch и CyclicBarrier. Упорядочите стратегию внедрения шаг за шагом:

1. Использовать CountdownLatch для инициализации сервисов
2. После запуска сервисов зарегистрировать их в Phaser
3. В фазах обработки данных модули синхронизируются через CyclicBarrier
4. По завершении фазы модули вызывают arriveAndDeregister у Phaser
5. CountdownLatch используется только один раз при старте
6. Phaser и CyclicBarrier используются многократно для циклов обработки

**Подсказка: Стратегия: инициализация → регистрация → фазы → завершение → многократное использование.**

**Ответ:**

- Использовать CountdownLatch для инициализации сервисов
- После запуска сервисов зарегистрировать их в Phaser
- В фазах обработки данных модули синхронизируются через CyclicBarrier
- По завершении фазы модули вызывают arriveAndDeregister у Phaser
- CountdownLatch используется только один раз при старте
- Phaser и CyclicBarrier используются многократно для циклов обработки

## Урок 4 - Semaphore для контроля доступа

## Шаг 01:

### Миссия 1

Аня пишет многопоточное приложение, в котором несколько потоков должны читать данные, но только один поток может записывать. Какой семафор ей подойдёт для обеспечения этого поведения?

Подсказка: Семафоры могут контролировать количество одновременно работающих потоков.

**Ответ:**

- **Semaphore с начальным числом разрешений 1 для записи, и другой семафор с большим количеством разрешений для чтения**
- Semaphore с начальным числом разрешений 0
- Semaphore с начальным числом разрешений 10, все потоки получают доступ

## Шаг 02:

### Миссия 2

Борис запускает 5 потоков, но хочет, чтобы одновременно работало не более 2 потоков при выполнении ресурсоёмкой операции. Какой подход с семафором наиболее подходит?

Подсказка: контроль числа одновременно активных потоков — основная задача семафора.

**Ответ:**

- **Инициализировать Semaphore(2), acquire перед операцией, release после**
- Semaphore(5), acquire внутри, release в конце
- Semaphore(1) для каждого потока отдельно

## Шаг 03:

### Миссия 3

Светлана хочет, чтобы поток, вызвавший `acquire()`, ждал максимум 3 секунды, если нет разрешения, затем выбросился `InterruptedException`. Какой метод семафора ей нужен?

Подсказка: Java Semaphore имеет методы с таймаутом и без.

**Ответ:**

- **`tryAcquire(long timeout, TimeUnit unit)`**
- `acquire()`
- `acquireUninterruptibly()`

## Шаг 04:

### Миссия 4

Дмитрий использует Semaphore с выставленным `fairness=true`. Что это значит для порядка получения разрешений?

Подсказка: Fairness влияет на очередь ожидающих потоков.

**Ответ:**

- **Разрешения выдаются потокам в порядке их запроса (FIFO)**
- Приоритетным получит тот, у кого выше приоритет потока
- Не гарантируется порядок, как в `ConcurrentLinkedQueue`

## Шаг 05:

### Миссия 5

Катя хочет сбросить все разрешения семафора к начальному состоянию автоматически, если приложение рестартуется. Что нужно, чтобы разрешения сбросились?

Подсказка: Состояние семафора хранится в памяти — как его восстановить?

**Ответ:**

- **Создавать новый объект Semaphore при старте приложения**
- Вызывать `release()` столько раз, сколько потоков было
- Установить семафор как `static volatile`

## Шаг 06:

### Миссия 6

Игорь хочет, чтобы в его приложении один поток контролировал ресурс, а другие ждали, но не блокировали интерфейс. Какой метод семафора поможет ему избежать блокировки, если поток прерывается?

Подсказка: есть варианты `acquire`, которые реагируют на прерывание.

**Ответ:**

- **`acquire(long timeout, TimeUnit unit)`**
- `acquireUninterruptibly()`
- `tryAcquire()` без параметров

## Шаг 07:

### Миссия 7

 Сопоставьте примеры использования Semaphore с их назначениями.

Примеры:

**Ответ:**

- Поток ждёт максимум 5 секунд разрешения
- Всего два потока читают, один пишет
- Ограничение до трёх одновременных запросов к БД
- Гарантия порядка справедливого ожидания

Назначения:

1. fairness=true
2. initial permits = 3
3. tryAcquire(timeout, unit)
4. читатель-писатель с двумя семафорами

 **Подсказка:**

Сопоставьте ситуацию с настройкой, которая позволяет её реализовать.

- **Поток ждёт максимум 5 секунд разрешения** — tryAcquire(timeout, unit)
- **Всего два потока читают, один пишет** — читатель-писатель с двумя семафорами
- **Ограничение до трёх одновременных запросов к БД** — initial permits = 3
- **Гарантия порядка справедливого ожидания** — fairness=true

 **Шаг 08:**

**Миссия 8**

 Сопоставьте методы Semaphore с их свойствами.

Методы:

**Ответ:**

- acquire()
- release()
- tryAcquire()
- acquireUninterruptibly()

Свойства:

1. Бросает InterruptedException, если поток прерывается
2. Не бросает InterruptedException, не реагирует на прерывание
3. Пытается получить разрешение без ожидания
4. Освобождает разрешение, увеличивая счётчик

 **Подсказка:**

Подумайте, какие методы блокируют и реагируют на прерывание, а какие — нет.

- **acquire()** — Бросает InterruptedException, если поток прерывается
- **acquireUninterruptibly()** — Не бросает InterruptedException, не реагирует на прерывание
- **tryAcquire()** — Пытается получить разрешение без ожидания
- **release()** — Освобождает разрешение, увеличивая счётчик

 **Шаг 09:**

**Миссия 9**

 Сопоставьте состояния семафора после выполнения операций с описаниями:

Состояния:

**Ответ:**

- Semaphore(3), два acquire(), один release()
- Semaphore(2), три acquire() с fairness=true
- Semaphore(1), tryAcquire() успешно, затем release()
- Semaphore(4), ни одного acquire()

Описания:

1. разрешения = 2
2. поток ждал, но не получил разрешения
3. разрешения = 4
4. разрешения = 1

💡 **Подсказка:** Считайте, что initial permits — это стартовое число, а acquire() уменьшает, release() увеличивает.

- **Semaphore(3), два acquire(), один release()** — разрешения = 2
- **Semaphore(2), три acquire() с fairness=true** — поток ждал, но не получил разрешения
- **Semaphore(1), tryAcquire() успешно, затем release()** — разрешения = 1
- **Semaphore(4), ни одного acquire()** — разрешения = 4

💡 **Шаг 10:**

**Миссия 10**

📖 Сопоставьте ситуации с возможными проблемами без использования семафора:

Ситуации:

**Ответ:**

- 10 потоков пишут одновременно в один файл
- много запросов к базе, нет ограничения количества соединений
- GUI-приложение зависает при ожидании доступа к ресурсу
- потоки ожидают в неопределённом порядке, часто одни и те же

Проблемы:

1. гонка данных (race condition)
2. отказ в обслуживании (resource starvation)
3. блокировка интерфейса (UI freeze)
4. неуправляемый поток ожидания (unfair waiting)

💡 **Подсказка:**

Что может произойти, если нет контроля владения ресурсами?

- **10 потоков пишут одновременно в один файл** — гонка данных (race condition)
- **много запросов к базе, нет ограничения количества соединений** — отказ в обслуживании (resource starvation)
- **GUI-приложение зависает при ожидании доступа к ресурсу** — блокировка интерфейса (UI freeze)
- **потоки ожидают в неопределённом порядке, часто одни и те же** — неуправляемый поток ожидания (unfair waiting)

💡 **Шаг 11:**

**Миссия 11**

📖 Сопоставьте параметры конструктора Semaphore с их реальными эффектами:

Параметры:

**Ответ:**

- `new Semaphore(1)`
- `new Semaphore(5, true)`
- `new Semaphore(0, false)`
- `new Semaphore(2, false)`

Эффекты:

1. никакой поток не получит разрешения до выпуска (release)
2. до пяти потоков могут работать одновременно без очереди FIFO
3. только один поток может работать одновременно без очереди FIFO
4. до двух потоков могут работать одновременно, порядок не гарантирован

 **Подсказка:**

Обратите внимание на число initial permits и параметр fairness.

- **`new Semaphore(1)`** — только один поток может работать одновременно без очереди FIFO
- **`new Semaphore(5, true)`** — до пяти потоков могут работать одновременно без очереди FIFO
- **`new Semaphore(0, false)`** — никакой поток не получит разрешения до выпуска (release)
- **`new Semaphore(2, false)`** — до двух потоков могут работать одновременно, порядок не гарантирован

 **Шаг 12:**

## Миссия 12

 Сопоставьте описания с реальными ошибками, которые помогает предотвратить семафор:

Описания:

**Ответ:**

- сразу несколько потоков записывают в лог-файл
- слишком много соединений с сервером
- несколько пользователей одновременно изменяют один объект данных
- другие потоки ждут без возможности прерывать ожидание

Ошибки:

1. race condition
2. resource exhaustion
3. data inconsistency
4. uninterruptible waiting / блокировка навсегда

 **Подсказка:**

Семафор помогает ограничивать доступ и предотвращать состояние гонки и избыточное потребление ресурсов.

- **сразу несколько потоков записывают в лог-файл** — race condition
- **слишком много соединений с сервером** — resource exhaustion
- **несколько пользователей одновременно изменяют один объект данных** — data inconsistency
- **другие потоки ждут без возможности прерывать ожидание** — uninterruptible waiting / блокировка навсегда

### 🔗 Шаг 13:

#### Миссия 13

Расположите шаги, как правильно использовать Semaphore в коде, чтобы избежать утечек разрешений:

1. вызвать `acquire()` перед доступом к защищённому ресурсу
2. внутри `try-finally` блокировать доступ к ресурсу
3. в `finally` блоке вызвать `release()`
4. обработать `InterruptedException` при попытке `acquire()`

#### 🔗 Подсказка:

Подумайте, в каком порядке делают шаги при работе с ресурсами, особенно когда возможны исключения.

#### Ответ:

- вызвать `acquire()` перед доступом к защищённому ресурсу
- обработать `InterruptedException` при попытке `acquire()`
- внутри `try-finally` блокировать доступ к ресурсу
- в `finally` блоке вызвать `release()`

### 🔗 Шаг 14:

#### Миссия 14

Расположите этапы настройки читателя-писателя с помощью семафоров:

1. семафор для читателей, семафор для писателей
2. читатели могут захватывать разрешения, если нет писателей
3. писатель ждёт, пока все читатели освободят ресурс
4. писатель получает эксклюзивное разрешение на запись

🔗 Подсказка: Рассмотрите, как взаимодействуют читатели и писатели по очереди и по доступу.

#### Ответ:

- семафор для читателей, семафор для писателей
- читатели могут захватывать разрешения, если нет писателей
- писатель ждёт, пока все читатели освободят ресурс
- писатель получает эксклюзивное разрешение на запись

### 🔗 Шаг 15:

#### Миссия 15

Расположите шаги, как безопасно использовать `tryAcquire` с таймаутом:

1. вызвать `tryAcquire(timeout, unit)`
2. **если true** — выполнить операцию с ресурсом
3. **если false** — обработать отсутствие разрешения
4. **если получено разрешение** — в `finally` вызвать `release()`

🔗 Подсказка: Убедитесь, что `release` вызывается только если разрешение получено.

#### Ответ:

- вызвать `tryAcquire(timeout, unit)`
- **если true** — выполнить операцию с ресурсом

- **если получено разрешение** — в finally вызвать `release()`
- **если false** — обработать отсутствие разрешения

#### 🔗 Шаг 16:

##### Миссия 16

Расположите этапы обработки fairness в конструкторе семафора и его влияния:

1. создать `Semaphore(permits, true)`
2. несколько потоков начнут ждать разрешения
3. потоки получают разрешения в порядке запроса
4. `release()` освободит разрешение для следующего в очереди

🔗 **Подсказка: fairness=true влияет на порядок ожидания и выдачи разрешений FIFO.**

**Ответ:**

- создать `Semaphore(permits, true)`
- несколько потоков начнут ждать разрешения
- потоки получают разрешения в порядке запроса
- `release()` освободит разрешение для следующего в очереди

#### 🔗 Шаг 17:

##### Миссия 17

Расположите шаги, как реализовать семафор для пула ограниченного размера:

1. инициализировать `Semaphore` с размером пула
2. в методе `borrow()`: `acquire` разрешение
3. в методе `return()`: `release` разрешение
4. обработать ситуацию, когда пула нет доступных объектов

🔗 **Подсказка: Пул ресурсов: подумайте, что делать, если все ресурсы заняты.**

**Ответ:**

- инициализировать `Semaphore` с размером пула
- в методе `borrow()`: `acquire` разрешение
- в методе `return()`: `release` разрешение
- обработать ситуацию, когда пула нет доступных объектов

## 📁 Модуль 8 - Инструменты мониторинга и отладки

### 📖 Урок 1 - Thread dumps и анализ блокировок

#### 🔗 Шаг 01:

##### Миссия 1

Илья запускает приложение, которое неожиданно виснет. Он делает `thread dump` и видит множество потоков в состоянии `RUNNABLE`, но один поток — в `BLOCKED`. Что это значит?

Подсказка: `BLOCKED` указывает на ожидание монитора кого-то другого.

**Ответ:**

- **Этот поток ожидает вход в синхронизированный блок, занятый другим потоком**
- Поток не стартовал и ожидает запуска
- Поток завершился, но остался в дампе

## Шаг 02:

### Миссия 2

Настя анализирует thread dump и видит состояние WAITING у потока. Что обычно означает WAITING?

Подсказка: WAITING часто появляется при вызове Object.wait() или LockSupport.park().

**Ответ:**

- Поток ничего не делает, ожидая уведомления или сигнала
- Поток занят вычислениями
- Поток завершён, но держит ресурс

## Шаг 03:

### Миссия 3

Максим видит в дампе поток с состоянием TIMED\_WAITING. Как он может быть получен?

Подсказка: Таймаутный wait или sleep приводит к TIMED\_WAITING.

**Ответ:**

- Поток вызвал sleep или wait с таймаутом
- Поток блокирован на синхронизаторе
- Поток завершает run()

## Шаг 04:

### Миссия 4

Ирина хочет быстро найти, какие потоки держат блокировки и какие ждут. Какой инструмент или опция JVM ей поможет?

Подсказка: Используйте флаги JVM или визуальные анализаторы дампов.

**Ответ:**

- Запустить JVM с -XX:+UnlockDiagnosticVMOptions и -XX:+PrintSafepointStatistics либо использовать jstack + анализатор
- Использовать только System.out.println в потоках
- Переименовать потоки по именам

## Шаг 05:

### Миссия 5

 Сопоставьте названия состояний потока в дампе с их описаниями:

Состояния:

**Ответ:**

- RUNNABLE
- BLOCKED
- WAITING
- TIMED\_WAITING

Описание:

1. активно исполняет код на ЦПУ или ожидает ввода/вывода
2. ожидает входа в синхронизированный блок, захваченный другим потоком
3. ожидает уведомления без таймаута
4. ожидает уведомления или сплит-операции с таймаутом или sleep

💡 **Подсказка: RUNNABLE не обязательно исполняет, но готов к исполнению.**

- **RUNNABLE** — активно исполняет код на ЦПУ или ожидает ввода/вывода
- **BLOCKED** — ожидает входа в синхронизированный блок, захваченный другим потоком
- **WAITING** — ожидает уведомления без таймаута
- **TIMED\_WAITING** — ожидает уведомления или сплит-операции с таймаутом или sleep

💡 **Шаг 06:**

### Миссия 6

📄 Сопоставьте виды блокировок из дампа с их угрозами:

Блокировки:

**Ответ:**

- deadlock
- livelock
- starvation
- priority inversion

Угрозы:

1. два потока навсегда ждут друг друга
2. потоки активно переключаются, но не продвигаются дальше
3. некоторые потоки никогда не получают доступ к ресурсу
4. низкоприоритетный поток мешает исполнению высокоприоритетного через посредника

💡 **Подсказка: Подумайте, что именно означает каждая из этих ситуаций.**

- **deadlock** — два потока навсегда ждут друг друга
- **livelock** — потоки активно переключаются, но не продвигаются дальше
- **starvation** — некоторые потоки никогда не получают доступ к ресурсу
- **priority inversion** — низкоприоритетный поток мешает исполнению высокоприоритетного через посредника

💡 **Шаг 07:**

### Миссия 7

📄 Сопоставьте инструменты анализа дампов с их возможностями:

Инструменты:

**Ответ:**

- jstack
- VisualVM
- ThreadMXBean (jconsole)
- jcmd Thread.print

Возможности:

1. вывести дамп потоков на консоль JVM
2. графический просмотр состояний и блокировок
3. инструмент мониторинга в реальном времени через JMX
4. альтернатива jstack встроенного инструмента командной строки

💡 **Подсказка: jcmd и jstack схожи по функционалу, VisualVM даёт GUI, JMX даёт runtime-мониторинг.**

- **jstack** — вывести дампы потоков на консоль JVM
- **VisualVM** — графический просмотр состояний и блокировок
- **ThreadMXBean (jconsole)** — инструмент мониторинга в реальном времени через JMX
- **jcmd Thread.print** — альтернатива jstack встроенного инструмента командной строки

## 🔗 Шаг 08:

### Миссия 8

Расположите этапы анализа thread dump, чтобы найти блокировки:

1. собрать дампы всех потоков
2. выявить потоки в состоянии BLOCKED или WAITING
3. посмотреть, какие мониторы они удерживают или на какие ждут
4. использовать инструменты визуализации для анализа

🔗 **Подсказка: Сначала нужно получить дампы, затем фильтровать по состояниям, потом анализировать мониторы и завершить визуальным анализом.**

**Ответ:**

- собрать дампы всех потоков
- выявить потоки в состоянии BLOCKED или WAITING
- посмотреть, какие мониторы они удерживают или на какие ждут
- использовать инструменты визуализации для анализа

## 🔗 Шаг 09:

### Миссия 9

Расположите шаги, как находить взаимные блокировки (deadlock) в дампе:

1. найти цепочку ожиданий потоков
2. определить, какие мониторы держатся каждым потоком
3. проверить, есть ли циклический обмен мониторами между потоками
4. отметить пары поток-монитор, участвующие в deadlock

🔗 **Подсказка: Deadlock — это цикл ожиданий: А ждёт В, В ждёт А (или более сложные циклы).**

**Ответ:**

- найти цепочку ожиданий потоков
- определить, какие мониторы держатся каждым потоком
- проверить, есть ли циклический обмен мониторами между потоками
- отметить пары поток-монитор, участвующие в deadlock

## 🔗 Шаг 10:

### Миссия 10

Расположите шаги, как строить отчёт блокировок в GUI-анализаторе:

1. импортировать дампы потоков в анализатор
2. отобразить дерево потоков и мониторов
3. фильтровать потоки по состояниям BLOCKED / WAITING
4. настроить подсветку взаимных блокировок и циклов

💡 **Подсказка: GUI-анализатор обычно показывает дерево потоков и позволяет фильтры и подсветки.**

**Ответ:**

- импортировать дампы потоков в анализатор
- отобразить дерево потоков и мониторов
- фильтровать потоки по состояниям BLOCKED / WAITING
- настроить подсветку взаимных блокировок и циклов

💡 **Шаг 11:**

### **Миссия 11**

Расположите шаги, как анализировать блокировки, затрагивающие поток-звезду (hub thread):

1. определить центральный поток, на который ссылаются другие
2. посмотреть, какие мониторы удерживает центральный поток
3. посмотреть, какие потоки ждут этих мониторов
4. восстановить граф ожидания и найти узкие места

💡 **Подсказка: Поток-звезда — это поток, от которого зависит множество других.**

**Ответ:**

- определить центральный поток, на который ссылаются другие
- посмотреть, какие мониторы удерживает центральный поток
- посмотреть, какие потоки ждут этих мониторов
- восстановить граф ожидания и найти узкие места

💡 **Шаг 12:**

### **Миссия 12**

Расположите шаги, как использовать ThreadMXBean для поиска мёртвых блокировок:

1. получить экземпляр ThreadMXBean через ManagementFactory
2. вызвать findDeadlockedThreads()
3. **если найдены идентификаторы потоков** — получить их информацию
4. анализировать, какие мониторы удерживают и на какие ждут

💡 **Подсказка: ThreadMXBean предоставляет API для поиска блокировок без дампа.**

**Ответ:**

- получить экземпляр ThreadMXBean через ManagementFactory
- вызвать findDeadlockedThreads()
- **если найдены идентификаторы потоков** — получить их информацию
- анализировать, какие мониторы удерживают и на какие ждут

💡 **Шаг 13:**

### **Миссия 13**

Расположите шаги, как добавить маркировку в thread dump для своего кода:

1. в коде задать имя потоку через Thread.currentThread().setName()
2. ждать блокировки или делать synchronized в названных потоках
3. собрать дампы с именованными потоками
4. в анализаторе искать по имени потока, а не по идентификатору

💡 **Подсказка: Имя потока облегчает поиск в дампе, вместо адресов или ID.**

**Ответ:**

- в коде задать имя потоку через `Thread.currentThread().setName()`
- ждать блокировки или делать `synchronized` в названных потоках
- собрать дампы с именованными потоками
- в анализаторе искать по имени потока, а не по идентификатору

💡 **Шаг 14:**

#### Миссия 14

Расположите шаги, как обрабатывать ситуацию, если в дампе нет явных deadlock, но есть задержки:

1. оценить длительности состояний BLOCKED или WAITING в дампе
2. отсортировать потоки по времени простоя в ожидании
3. обратить внимание на потоки-«узлы» с большим ожиданием
4. **проанализировать, почему эти потоки ждут** — цепочки блокировок

💡 **Подсказка: Не всегда deadlock явно проявлен — часто это цепочки задержек.**

**Ответ:**

- оценить длительности состояний BLOCKED или WAITING в дампе
- отсортировать потоки по времени простоя в ожидании
- обратить внимание на потоки-«узлы» с большим ожиданием
- **проанализировать, почему эти потоки ждут** — цепочки блокировок

💡 **Шаг 15:**

#### Миссия 15

Расположите шаги, как комбинировать ThreadMXBean и дампы для качественного анализа:

1. сначала вызвать `findDeadlockedThreads()` для быстрой проверки
2. **если есть идентификаторы** — получить дампы или детальные данные по этим потокам
3. импортировать дампы в визуальный анализатор
4. сопоставить данные от ThreadMXBean и визуального анализа для подтверждения гипотез

💡 **Подсказка: ThreadMXBean быстро говорит о потенциальных deadlock, дампы дают контекст.**

**Ответ:**

- сначала вызвать `findDeadlockedThreads()` для быстрой проверки
- **если есть идентификаторы** — получить дампы или детальные данные по этим потокам
- импортировать дампы в визуальный анализатор
- сопоставить данные от ThreadMXBean и визуального анализа для подтверждения гипотез

## 📖 Урок 2 - JConsole и VisualVM для мониторинга

💡 **Шаг 01:**

#### Миссия 1

Анна запустила JConsole и видит вкладку «Threads». Какое состояние потока она увидит, если поток спит внутри `Thread.sleep()`?

Подсказка: В Java потоки, ожидающие с таймаутом, получают особое состояние.

**Ответ:**

- **TIMED\_WAITING**
- RUNNABLE
- BLOCKED

 **Шаг 02:**

### **Миссия 2**

Пётр подключается к JVM через VisualVM. Что он может мониторить в режиме Remote?

Подсказка: VisualVM поддерживает сбор данных о памяти, потоках и CPU.

**Ответ:**

- **Память, потоки, использование CPU**
- Изменение исходного кода приложения
- Удалённое редактирование базы данных

 **Шаг 03:**

### **Миссия 3**

Ольга в JConsole видит, что один поток постоянно находится в состоянии BLOCKED. Что это говорит о приложении?

Подсказка: BLOCKED сигнализирует о конкуренции за монитор.

**Ответ:**

- **Этот поток ждет, чтобы войти в synchronized блок, удерживаемый другим**
- Он занят выполнением длительной задачи
- Он завершился, но остался в списке

 **Шаг 04:**

### **Миссия 4**

В VisualVM на вкладке «Monitor» видно, что использование CPU постоянно около 100 %. Что наиболее вероятно?

Подсказка: Высокий CPU без роста памяти часто сигнал о «горячем» цикле.

**Ответ:**

- **Приложение находится в бесконечном активном цикле**
- Утечка памяти
- Потоки ждут I/O

 **Шаг 05:**

### **Миссия 5**

Никита хочет увидеть, сколько объектов класса MyTask загружены в хип. Какой инструмент в VisualVM он использует?

Подсказка: Используй VisualVM для просмотра heap и детального анализа.

**Ответ:**

- **Heap Dump / Objects view**
- Threads view
- CPU sampler

## 🔗 Шаг 06:

### Миссия 6

📄 Сопоставьте вкладки в VisualVM / JConsole с их назначением:

Вкладки:

**Ответ:**

- Memory / Heap
- Threads
- CPU / Monitor
- Sampler / Profiler

Назначения:

1. отслеживать число потоков и состояния
2. отображать нагрузку на CPU со временем
3. анализировать объекты, GC, утечки памяти
4. профилирование методов и использование ресурсов

🔗 **Подсказка: Каждая вкладка специализируется: память, потоки, процессор, профилирование.**

- **Memory / Heap** — анализировать объекты, GC, утечки памяти
- **Threads** — отслеживать число потоков и состояния
- **CPU / Monitor** — отображать нагрузку на CPU со временем
- **Sampler / Profiler** — профилирование методов и использование ресурсов

## 🔗 Шаг 07:

### Миссия 7

📄 Сопоставьте сценарии с тем, что в VisualVM вы увидите:

Сценарии:

**Ответ:**

- приложение активно аллоцирует объекты
- многие потоки заблокированы на синхронизаторах
- методы вызываются рекурсивно глубоко
- высокая конкуренция за CPU

Что вы увидите:

1. рост heap usage, частые GC
2. многие потоки в BLOCKED/WAITING состоянии в Threads
3. методы, потребляющие много времени, на профайлере
4. CPU график близок к 100 %

🔗 **Подсказка: Скарты: память, потоки, методы, CPU — что будет характерно?**

- **приложение активно аллоцирует объекты** — рост heap usage, частые GC
- **многие потоки заблокированы на синхронизаторах** — многие потоки в BLOCKED/WAITING состоянии в Threads
- **методы вызываются рекурсивно глубоко** — методы, потребляющие много времени, на профайлере
- **высокая конкуренция за CPU** — CPU график близок к 100 %

## 🔗 Шаг 08:

### Миссия 8

📖 Сопоставьте возможности JConsole / VisualVM с тем, как они используются:

Возможности:

**Ответ:**

- GC вызов вручную
- Detect deadlocked threads
- Memory histogram
- CPU profiler (sampling)

Использование:

1. вынудить сбор мусора из консоли
2. отображение распределения классов и количества объектов
3. определение зацикленных блокировок в потоках
4. измерение времени выполнения методов выборочно

🔗 **Подсказка: Подумайте, какая функция для чего: GC, deadlock, гистограммы, профилирование.**

- **GC вызов вручную** — вынудить сбор мусора из консоли
- **Memory histogram** — отображение распределения классов и количества объектов
- **Detect deadlocked threads** — определение зацикленных блокировок в потоках
- **CPU profiler (sampling)** — измерение времени выполнения методов выборочно

## 🔗 Шаг 09:

### Миссия 9

📖 Сопоставьте интерфейсы VisualVM / JConsole с их источниками данных:

Интерфейсы:

**Ответ:**

- JMX bean отображение
- Profiler view
- Heap dump view
- Thread view

Источники:

1. через MBeans через платформенный MBeanServer
2. данные из агент-профайлера JVM
3. импортированный дамп с диска или через jmap
4. информация из ThreadMXBean / JVM runtime

🔗 **Подсказка: Каждый интерфейс получает данные из разных источников JVM.**

- **JMX bean отображение** — через MBeans через платформенный MBeanServer
- **Profiler view** — данные из агент-профайлера JVM
- **Heap dump view** — импортированный дамп с диска или через jmap
- **Thread view** — информация из ThreadMXBean / JVM runtime

## Шаг 10:

### Миссия 10

Расположите шаги, как корректно подключиться к удалённой JVM через VisualVM:

1. запустить JVM с включённой JMX (например, `-Dcom.sun.management.jmxremote`)
2. открыть VisualVM и выбрать Remote Connection
3. ввести хост, порт и, если нужно, учётные данные
4. установить соединение и дождаться данных мониторинга

 **Подсказка: Без включённого JMX визуальная связь невозможна, затем идёт установка подключения.**

**Ответ:**

- запустить JVM с включённой JMX (например, `-Dcom.sun.management.jmxremote`)
- открыть VisualVM и выбрать Remote Connection
- ввести хост, порт и, если нужно, учётные данные
- установить соединение и дождаться данных мониторинга

## Шаг 11:

### Миссия 11

Расположите шаги, как выполнить heap dump и проанализировать утечку памяти:

1. в VisualVM или jmap сделать heap dump
2. открыть дамп в VisualVM или MAT
3. перейти к view «Classes / Histogram» или «Dominators»
4. найти классы с растущим числом экземпляров и исследовать корни удержания

 **Подсказка: Heap dump — это снимок, затем анализ гистограмм и корней удержания.**

**Ответ:**

- в VisualVM или jmap сделать heap dump
- открыть дамп в VisualVM или MAT
- перейти к view «Classes / Histogram» или «Dominators»
- найти классы с растущим числом экземпляров и исследовать корни удержания

## Шаг 12:

### Миссия 12

Расположите шаги, как использовать профилирование CPU в VisualVM:

1. перейти на вкладку Profiler / CPU
2. запустить профилирование (Start)
3. выполнить сценарий в приложении, нагрузка кода
4. остановить профилирование и проанализировать методы по времени

 **Подсказка: Сначала выбрать профилирование, затем запуск, нагрузка, остановка и анализ.**

**Ответ:**

- перейти на вкладку Profiler / CPU
- запустить профилирование (Start)
- выполнить сценарий в приложении, нагрузка кода
- остановить профилирование и проанализировать методы по времени

## Шаг 13:

### Миссия 13

Расположите шаги, как найти утечку памяти в длительно работающем приложении:

1. подключиться к приложению через VisualVM
2. наблюдать рост используемой памяти с течением времени
3. сделать heap dump в момент высокой памяти
4. проанализировать дамп на классы, чьи объекты удерживаются корнями

 **Подсказка: Утечка проявляется как постоянный рост памяти без освобождения.**

**Ответ:**

- подключиться к приложению через VisualVM
- наблюдать рост используемой памяти с течением времени
- сделать heap dump в момент высокой памяти
- проанализировать дамп на классы, чьи объекты удерживаются корнями

## Шаг 14:

### Миссия 14

Расположите шаги, как использовать VisualVM и JConsole совместно для глубокого анализа:

1. открыть JConsole и включить мониторинг потоков и памяти
2. открыть VisualVM для профилирования и heap dump
3. сравнить данные потоков из JConsole и профайлер из VisualVM
4. на основе данных принять меры по оптимизации

 **Подсказка: JConsole хорош для быстрого мониторинга, VisualVM — для глубинного анализа.**

**Ответ:**

- открыть JConsole и включить мониторинг потоков и памяти
- открыть VisualVM для профилирования и heap dump
- сравнить данные потоков из JConsole и профайлер из VisualVM
- на основе данных принять меры по оптимизации

## Шаг 15:

### Миссия 15

Расположите шаги, как настроить фильтрацию в VisualVM для потоков по имени:

1. задать понятные имена потокам в коде через Thread.setName()
2. в VisualVM открыть вкладку Threads
3. включить фильтрацию по имени или поиск
4. искать потоки по имени, а не по id

 **Подсказка: Имена потоков облегчают навигацию при анализе в GUI.**

**Ответ:**

- задать понятные имена потокам в коде через Thread.setName()
- в VisualVM открыть вкладку Threads
- включить фильтрацию по имени или поиск
- искать потоки по имени, а не по id

## Шаг 16:

### Миссия 16

Расположите шаги, как профилировать методы, вызывающие GC:

1. включить профилирование CPU / Allocation в VisualVM
2. выполнить сценарии, вызывающие много аллокаций и освобождений
3. остановить профилирование и отфильтровать методы, вызывающие много аллокаций
4. проанализировать, какие методы вызывают избыточный GC

 **Подсказка: Методы, активно создающие объекты, обычно приводят к частым GC.**

**Ответ:**

- включить профилирование CPU / Allocation в VisualVM
- выполнить сценарии, вызывающие много аллокаций и освобождений
- остановить профилирование и отфильтровать методы, вызывающие много аллокаций
- проанализировать, какие методы вызывают избыточный GC

## Шаг 17:

### Миссия 17

Расположите шаги, как совместно использовать VisualVM, JConsole и heap dumps для глубокого анализа производительности:

1. открыть JConsole для быстрого мониторинга и начальной диагностики
2. подключиться в VisualVM для профилирования и heap dump
3. собирать heap dumps и сравнивать снимки по времени
4. на основе всех данных принимать решение об оптимизации или исправлениях

 **Подсказка: Каждый инструмент дополняет другой: мониторинг, профилирование, дампы.**

**Ответ:**

- открыть JConsole для быстрого мониторинга и начальной диагностики
- подключиться в VisualVM для профилирования и heap dump
- собирать heap dumps и сравнивать снимки по времени
- на основе всех данных принимать решение об оптимизации или исправлениях

## Урок 3 - Flight Recorder и профилировка многопоточных приложений

## Шаг 01:

### Миссия 1

Вы запускаете Java-приложение с несколькими потоками и хотите собрать запись с помощью Flight Recorder. Какой параметр запуска является обязательным?

Подсказка: Нужно явно включить запись.

**Ответ:**

- **-XX:StartFlightRecording**
- -XX:+UseFlightRecorder
- -XX:+EnableProfiling
- -XX:FlightRecorderEnabled

## Шаг 02:

### Миссия 2

Когда вы открываете файл записи JFR (Java Flight Recorder) в инструменте анализа, вы видите событие типа “ThreadStart”. Что оно показывает?

Подсказка: Ищите связь с жизненным циклом потока.

**Ответ:**

- **Сигнал о создании нового потока**
- Сигнал о начале записи JFR
- Сигнал о завершении работы метода
- Сигнал о запуске профилировщика

## Шаг 03:

### Миссия 3

Вы заметили, что при запуске с Flight Recorder приложение слегка замедляется. Что наиболее вероятно является причиной?

Подсказка: Запись события в потоках — не бесплатна.

**Ответ:**

- **Накладные расходы на сбор данных и запись событий**
- Вмешательство сборщика мусора
- Конфликт блокировок внутри JFR
- Неоптимальное планирование потоков

## Шаг 04:

### Миссия 4

Вы хотите ограничить кастомные события, записываемые в JFR, только для части вашего модуля. Как это можно сделать?

Подсказка: События можно фильтровать.

**Ответ:**

- **Задать порог порта записи (threshold) для событий в конфигурации**
- Отключить все события кроме GC
- Установить размер буфера на 0
- Запускать JFR только в одном потоке

## Шаг 05:

### Миссия 5

 **Задача:** Сопоставьте каждое понятие из области JFR или профилировки многопоточных приложений с его описанием.

Понятия:

**Ответ:**

- Sampling
- Event Streaming
- Method Profiling
- Safe Points

Описания:

1. Методический сбор статистики по методам, сколько времени они выполняются.
2. Периодическое сэмплирование состояния стека потоков.
3. Точка, в которой JVM может безопасно делать остановки для GC или профилирования.
4. Непрерывная передача событий в анализатор во время работы приложения.

Подсказка: Подумайте, какое поведение подразумевает «sampling» и что такое «streaming».

- **Sampling** — Периодическое сэмплирование состояния стека потоков.
- **Event Streaming** — Непрерывная передача событий в анализатор во время работы приложения.
- **Method Profiling** — Методический сбор статистики по методам, сколько времени они выполняются.
- **Safe Points** — Точка, в которой JVM может безопасно делать остановки для GC или профилирования.

## Шаг 06:

### Миссия 6

 Задача: Сопоставьте тип события JFR с ситуацией, в которой оно полезно.

Понятия:

**Ответ:**

- ThreadAllocation
- Lock Contention
- Method Sample
- Garbage Collection

Описания:

1. Отслеживание конкуренции блокировок среди потоков.
2. Записи о распределении памяти по потокам.
3. Записи о срабатывании сборки мусора.
4. Оценка того, какие методы потребляют CPU по сэмплам.

Подсказка: Помните: «Allocation» про память, «Lock Contention» про блокировки.

- **ThreadAllocation** — Записи о распределении памяти по потокам.
- **Lock Contention** — Отслеживание конкуренции блокировок среди потоков.
- **Method Sample** — Оценка того, какие методы потребляют CPU по сэмплам.
- **Garbage Collection** — Записи о срабатывании сборки мусора.

## Шаг 07:

### Миссия 7

 Задача: Сопоставьте этап анализа JFR записи с его задачей.

Этапы:

**Ответ:**

- Просмотр отсчетов (samples)
- Сегментирование по времени
- Фильтрация событий
- Скрытие несущественных потоков

Задачи:

1. Выделить интересующий временной диапазон.
2. Убрать шумовые потоки из анализа.
3. Показать горячие методы по CPU.
4. Оставить только события, относящиеся к проблемному коду.

Подсказка: Подумайте, что удобнее делать сначала — сузить ветвие, затем убрать лишнее.

- **Сегментирование по времени** — Выделить интересующий временной диапазон.
- **Фильтрация событий** — Оставить только события, относящиеся к проблемному коду.
- **Просмотр отсчетов (samples)** — Показать горячие методы по CPU.
- **Скрытие несущественных потоков** — Убрать шумовые потоки из анализа.

## Шаг 08:

### Миссия 8

Расположите шаги при детальном анализе JFR записи:

1. Фильтрация по интересующим классам/модулям
2. Открытие распределения по методам или по потокам
3. Сегментирование по времени события (timeline)
4. Выбор горячих точек (hotspots) CPU или аллокации

*Подсказка:* Сначала сузьте окно времени, затем примените фильтры и выделите «горячие» данные.

**Ответ:**

- Сегментирование по времени события (timeline)
- Фильтрация по интересующим классам/модулям
- Открытие распределения по методам или по потокам
- Выбор горячих точек (hotspots) CPU или аллокации

## Шаг 09:

### Миссия 9

Расположите в правильном порядке действия при поиске конкуренции (lock contention) через JFR:

1. Найти события Lock Contention или Monitor Enter
2. Посмотреть стек вызовов для этих событий
3. Сегментировать время, где часто встречаются контеншены
4. Отфильтровать по классу/методу, который вас интересует

*Подсказка:* Сначала найти места, потом углубиться в детали.

**Ответ:**

- Найти события Lock Contention или Monitor Enter
- Сегментировать время, где часто встречаются контеншены
- Отфильтровать по классу/методу, который вас интересует
- Посмотреть стек вызовов для этих событий

## Шаг 10:

### Миссия 10

Расположите шаги для анализа аллокаций через JFR:

1. Выбрать Thread Allocation события
2. Сегментировать по времени пиковых аллокаций
3. Фильтрация по пакету/классу
4. Посмотреть стек вызовов, ведущий к аллокациям

*Подсказка:* Шаги аналогичны анализу блокировок — сначала найти, затем углубиться.

**Ответ:**

- Выбрать Thread Allocation события
- Сегментировать по времени пиковых аллокаций
- Фильтрация по пакету/классу
- Посмотреть стек вызовов, ведущий к аллокациям

## Шаг 11:

### **Миссия 11**

Расположите этапы оптимизации после анализа JFR:

1. Переписать критический метод или снизить синхронизацию
2. Проверить, что устранена конкуренция или аллокации
3. Сделать новую запись JFR для сравнения
4. Сравнить «до» и «после» по метрикам

*Подсказка:* Всегда сравнивайте результат после изменений.

**Ответ:**

- Переписать критический метод или снизить синхронизацию
- Сделать новую запись JFR для сравнения
- Проверить, что устранена конкуренция или аллокации
- Сравнить «до» и «после» по метрикам

## Шаг 12:

### **Миссия 12**

Расположите шаги построения пользовательского события JFR:

1. Создать класс, наследующий `jdk.jfr.Event`
2. Добавить поля и аннотации (`Label`, `Description` и др.)
3. Выбрать, где (в каком месте) вызывать `event.begin()/commit()`
4. Запуск записи JFR с включением этого события

*Подсказка:* Сначала модель события, потом его вызовы и включение в запись.

**Ответ:**

- Создать класс, наследующий `jdk.jfr.Event`
- Добавить поля и аннотации (`Label`, `Description` и др.)
- Выбрать, где (в каком месте) вызывать `event.begin()/commit()`
- Запуск записи JFR с включением этого события

## Шаг 13:

### **Миссия 13**

Расположите шаги профилировки метода с высокой загрузкой CPU:

1. Включить методический профилинг или сэмплинг в конфигурации JFR
2. Запустить нагрузочный тест
3. Собрать JFR запись при нагрузке
4. Проанализировать горячие методы и точки CPU

*Подсказка:* Сначала включите нужный режим, затем нагрузка, затем анализ.

**Ответ:**

- Включить методический профилинг или сэмплинг в конфигурации JFR
- Запустить нагрузочный тест
- Собрать JFR запись при нагрузке
- Проанализировать горячие методы и точки CPU

#### Шаг 14:

##### **Миссия 14**

Расположите шаги анализа производительности многопоточного веб-сервера:

1. Запустить сервер под нагрузкой
2. Собрать JFR запись с разных фаз (вход, пик, спад)
3. Сегментировать временные окна (warm-up, peak, cooldown)
4. Сравнить профили потоков между фазами

*Подсказка:* Анализ через фазовые сегменты помогает увидеть, как нагрузка влияет с течением времени.

**Ответ:**

- Запустить сервер под нагрузкой
- Собрать JFR запись с разных фаз (вход, пик, спад)
- Сегментировать временные окна (warm-up, peak, cooldown)
- Сравнить профили потоков между фазами

#### Шаг 15:

##### **Миссия 15**

Расположите шаги создания собственных событий с разной детализацией (high, low):

1. Определить уровни detail (HIGH, LOW) в аннотации @Label / @Category
2. **Создать два класса** — одно событие высокого детала, одно — низкого
3. Включить оба события в конфигурацию JFR, но с разными порогами
4. В нужных точках кода вызывать событие соответствующего уровня

*Подсказка:* События разного уровня позволяют собирать подробные данные только, когда нужна диагностика.

**Ответ:**

- Определить уровни detail (HIGH, LOW) в аннотации @Label / @Category
- **Создать два класса** — одно событие высокого детала, одно — низкого
- Включить оба события в конфигурацию JFR, но с разными порогами
- В нужных точках кода вызывать событие соответствующего уровня

## Шаг 16:

### Миссия 16

Расположите шаги при сравнении двух записей JFR (базовой и после оптимизации):

1. Открыть обе записи в инструменте анализа
2. Сегментировать по одинаковым временным диапазонам
3. Сравнить распределения CPU / аллокации / блокировки
4. Выделить улучшения и регрессии

*Подсказка:* Важно сравнивать идентичные окна времени.

#### Ответ:

- Открыть обе записи в инструменте анализа
- Сегментировать по одинаковым временным диапазонам
- Сравнить распределения CPU / аллокации / блокировки
- Выделить улучшения и регрессии

## Шаг 17:

### Миссия 17

Расположите этапы комплексной отладки многопоточного приложения с помощью JFR и профилировщика:

1. Запустить приложение с включенным JFR и профилировщиком
2. Сгенерировать нагрузку, стимулирующую узкие места
3. Собрать запись и экспортировать её
4. Проанализировать hotspot-методы, блокировки и аллокации
5. Внедрить оптимизации в код
6. Повторить сбор записи и сравнить результаты

*Подсказка:* Отладка — это итерационный процесс: измерять, исправлять, измерять вновь.

#### Ответ:

- Запустить приложение с включенным JFR и профилировщиком
- Сгенерировать нагрузку, стимулирующую узкие места
- Собрать запись и экспортировать её
- Проанализировать hotspot-методы, блокировки и аллокации
- Внедрить оптимизации в код
- Повторить сбор записи и сравнить результаты

## Модуль 9 - Оптимизация производительности

### Урок 1 - Профилирование потоков

## Шаг 01:

### Миссия 1

Анна запускает своё многопоточное приложение и хочет быстро оценить, какие потоки потребляют CPU. Какой инструмент позволит ей это сделать на лету?

*Подсказка:* Подумайте, что встроено в JDK и работает без сторонних агентов.

#### Ответ:

- **Java Flight Recorder (JFR)**

- JConsole
- VisualVM
- jstack

## 🔧 Шаг 02:

### Миссия 2

Борис заметил, что один поток стабильно загружает CPU 100%, но в JFR-записи он не отображается как горячий. Что наиболее вероятно причина?

Подсказка: Подумайте про методический профилинг и сэмплирование.

**Ответ:**

- **CPU времени слишком мало между safe points (сэмплы пропускаются)**
- Поток был исключён из записи
- JFR не поддерживает многопоточность
- Поток не вызывал методов

## 🔧 Шаг 03:

### Миссия 3

В приложении используется пул потоков, и вы хотите измерить, сколько времени потоки простаивают (idle). Какой подход из списка наиболее адекватен?

Подсказка: Idle означает отсутствие активности в вашем коде.

**Ответ:**

- **Добавить кастомное событие JFR в начале и конце выполнения задачи**
- Писать лог в каждый idle цикл
- Использовать jstack время от времени
- Собирать трассировки через VisualVM постоянно

## 🔧 Шаг 04:

### Миссия 5

📄 Сопоставьте тип профилирования потоков с его характеристикой.

Тип профилирования:

**Ответ:**

- Сэмплирование (sampling)
- Методический профилинг (method profiling)
- Инструментирование (instrumentation)
- Трассировка (tracing)

Характеристики:

1. Сохраняет детальные вызовы каждого метода, но замедляет приложение.
2. Выборочные срезы стека с минимальными накладными расходами.
3. Встроено вызовы кода в теле методов для метрик и метаданных.
4. Записывает полную историю вызовов и переходов между методами.

Подсказка: Подумайте, что будет дороже — записывать всё или выборочные сэмплы.

- **Сэмплирование (sampling)** — Выборочные срезы стека с минимальными накладными расходами.

- **Методический профилинг (method profiling)** — Сохраняет детальные вызовы каждого метода, но замедляет приложение.
- **Инструментирование (instrumentation)** — Встроено вызовы кода в теле методов для метрик и метаданных.
- **Трассировка (tracing)** — Записывает полную историю вызовов и переходов между методами.

## 💡 Шаг 05:

### Миссия 6

📖 Сопоставьте метрику многопоточного приложения с тем, как её можно измерить.

Метрика:

**Ответ:**

- Задержка отклика (latency)
- Пропускная способность (throughput)
- Поточковая конкуренция (lock contention)
- Время простоя потоков (idle time)

Как измерить:

1. События конкуренции блокировок в профилировщике
2. Отслеживание периодов без работы задач в потоке
3. Сравнение числа выполненных операций за секунду
4. Измерение времени между запросом и ответом

Подсказка: Подумайте: latency — это задержка, throughput — количество операций.

- **Задержка отклика (latency)** — Измерение времени между запросом и ответом
- **Пропускная способность (throughput)** — Сравнение числа выполненных операций за секунду
- **Поточковая конкуренция (lock contention)** — События конкуренции блокировок в профилировщике
- **Время простоя потоков (idle time)** — Отслеживание периодов без работы задач в потоке

## 💡 Шаг 06:

### Миссия 7

📖 Сопоставьте проблему производительности с признаком в профилях.

Проблема:

**Ответ:**

- Избыточная блокировка
- Горячий метод (hotspot)  
Чрезмерные аллокации
- Слишком частое переключение контекста (context switching)

Признак в профиле:

1. Высокий процент CPU-индикаторов в одном методе
2. Много событий Lock Contention или Monitor Enter  
Много событий выделения памяти (allocation) в короткие периоды
3. Частые пики времени, когда потоки блокируются и разблокируются

Подсказка: Горячий метод — о методах, контеншены — о блокировках.

- **Горячий метод (hotspot)** — Высокий процент CPU-индикаторов в одном методе
- **Избыточная блокировка** — Много событий Lock Contention или Monitor Enter
- **Чрезмерные аллокации** — Много событий выделения памяти (allocation) в короткие периоды
- **Слишком частое переключение контекста (context switching)** — Частые пики времени, когда потоки блокируются и разблокируются

## 🔔 Шаг 07:

### Миссия 8

📄 Сопоставьте вариант оптимизации с его эффектом на профиль.

Оптимизация:

**Ответ:**

- Уменьшение блокировок (lock-free структуры)
- Кэширование результатов
- Объединение мелких задач в батчи (batching)
- Использование примитивов с низкой стоимостью (например, ConcurrentHashMap)

Эффект:

1. Уменьшение количества Lock Contention событий
2. Снижение нагрузок на GC / аллокации
3. Уменьшение накладных расходов переключения задач
4. Меньшее число синхронизаций и конкуренций внутри коллекций

Подсказка: Подумайте: кэш — это меньше аллокаций, batching — менее часто переключайтесь между задачами.

- **Уменьшение блокировок (lock-free структуры)** — Уменьшение количества Lock Contention событий
- **Кэширование результатов** — Снижение нагрузок на GC / аллокации
- **Объединение мелких задач в батчи (batching)** — Уменьшение накладных расходов переключения задач
- **Использование примитивов с низкой стоимостью (например, ConcurrentHashMap)** — Меньшее число синхронизаций и конкуренций внутри коллекций

## 🔔 Шаг 08:

### Миссия 9

Расположите шаги при выявлении «горячего метода» через профилировщик:

1. Запустить нагрузочное тестирование
2. Собрать профиль (sampling или method profiling)
3. Выделить методы с наибольшим потреблением CPU
4. Просмотреть трассировки стека для этих методов

Подсказка: Сначала нагружаем, потом профилируем, потом анализируем «нагревшиеся» места.

**Ответ:**

- Запустить нагрузочное тестирование
- Собрать профиль (sampling или method profiling)
- Выделить методы с наибольшим потреблением CPU
- Просмотреть трассировки стека для этих методов

## Шаг 09:

### Миссия 10

Расположите шаги при анализе конкуренции потоков:

1. Собрать профиль, включающий Lock Contention события
2. Сегментировать временной диапазон с пиками блокировок
3. Отфильтровать по интересующим методам/классам
4. Просмотреть стеки вызовов для Lock Contention событий

*Подсказка:* После сегментации легче локализовать «узкие места».

**Ответ:**

- Собрать профиль, включающий Lock Contention события
- Сегментировать временной диапазон с пиками блокировок
- Отфильтровать по интересующим методам/классам
- Просмотреть стеки вызовов для Lock Contention событий

## Шаг 10:

### Миссия 11

Расположите шаги при анализе аллокаций в многопоточном приложении:

1. Собрать профиль с событиями allocation
2. Определить периоды пиковых аллокаций
3. Фильтровать по классам/пакетам, где много аллокаций
4. Просмотреть стеки вызовов для allocation-событий

*Подсказка:* Аналогично конкуренции, но про память. Начинайте с профиля, затем сегментации.

**Ответ:**

- Собрать профиль с событиями allocation
- Определить периоды пиковых аллокаций
- Фильтровать по классам/пакетам, где много аллокаций
- Просмотреть стеки вызовов для allocation-событий

## Шаг 11:

### Миссия 12

Расположите этапы итеративной оптимизации после первого анализа профиля:

1. Внедрить оптимизацию (например, уменьшить синхронизацию или аллокации)
2. Запустить новую нагрузку или тест
3. Собрать новую профилировочную запись
4. Сравнить профили «до» и «после» по ключевым метрикам

*Подсказка:* Оптимизация — это повторение: измерил, изменил, измерил опять.

**Ответ:**

- Внедрить оптимизацию (например, уменьшить синхронизацию или аллокации)
- Запустить новую нагрузку или тест
- Собрать новую профилировочную запись
- Сравнить профили «до» и «после» по ключевым метрикам

## Шаг 12:

### Миссия 13

Расположите этапы анализа профилей для разных фаз нагрузки (warm-up, steady, burst):

1. Сегментировать запись на фазы (warm-up, steady, burst)
2. Сравнить горячие методы в каждой фазе
3. Найти методы, которые растут в нагрузке
4. Определить точки, где оптимизация даст максимальный эффект

*Подсказка:* Разные фазы показывают разные горячие источники нагрузки.

**Ответ:**

- Сегментировать запись на фазы (warm-up, steady, burst)
- Сравнить горячие методы в каждой фазе
- Найти методы, которые растут в нагрузке
- Определить точки, где оптимизация даст максимальный эффект

## Шаг 13:

### Миссия 14

Расположите этапы профилирования «реактивного» приложения (event-driven):

1. Запустить приложение и нагружать событиями
2. Собирать распределённый профиль (sampling + события latency)
3. Сегментировать по событиям (например, запрос, обработка, отклик)
4. Анализ горячих точек внутри обработки событий

*Подсказка:* В event-driven системах важно смотреть профили в контексте событий.

**Ответ:**

- Запустить приложение и нагружать событиями
- Собирать распределённый профиль (sampling + события latency)
- Сегментировать по событиям (например, запрос, обработка, отклик)
- Анализ горячих точек внутри обработки событий

## Шаг 14:

### Миссия 15

Расположите шаги профилирования и оптимизации вспомогательных потоков (monitoring, logging):

1. Собрать профиль с включением вспомогательных потоков
2. Идентифицировать потоки, не приносящие полезной нагрузки
3. Ограничить частоту их работы или перевод в менее агрессивный режим
4. Снова профилировать и проверить, не ухудшилась ли производительность

*Подсказка:* Иногда «административные» потоки мешают, их тоже полезно профилировать.

**Ответ:**

- Собрать профиль с включением вспомогательных потоков
- Идентифицировать потоки, не приносящие полезной нагрузки
- Ограничить частоту их работы или перевод в менее агрессивный режим
- Снова профилировать и проверить, не ухудшилась ли производительность

## Шаг 15:

### Миссия 16

Расположите этапы сравнения двух профилей (старого и нового) с учётом статистики:

1. Сопоставить временные сегменты
2. Сравнить распределения CPU / аллокации / блокировок
3. Найти улучшения и регрессии по методам
4. Составить отчёт с рекомендациями для дальнейшего цикла оптимизации

*Подсказка:* Статистический подход позволяет объективно увидеть, что реально изменилось.

**Ответ:**

- Сопоставить временные сегменты
- Сравнить распределения CPU / аллокации / блокировок
- Найти улучшения и регрессии по методам
- Составить отчёт с рекомендациями для дальнейшего цикла оптимизации

## Шаг 16:

### Миссия 17

Расположите полный цикл профилирования и оптимизации многопоточного приложения:

1. Настроить профилирование (sampling / методический / кастомные события)
2. Запустить приложение под нагрузкой
3. Собрать профиль и сегментировать
4. Проанализировать горячие методы, конкуренции, аллокации
5. Внедрить изменения в код
6. Повторить сбор профиля и сравнить результаты
7. Оценить, достигнут ли целевой уровень производительности

*Подсказка:* Это итеративный процесс, и важно завершать его проверкой результата.

**Ответ:**

- Настроить профилирование (sampling / методический / кастомные события)
- Запустить приложение под нагрузкой
- Собрать профиль и сегментировать
- Проанализировать горячие методы, конкуренции, аллокации
- Внедрить изменения в код
- Повторить сбор профиля и сравнить результаты
- Оценить, достигнут ли целевой уровень производительности

## Урок 2 - Avoiding unnecessary synchronization

## Шаг 01:

### Миссия 1

Иван написал метод, обёрнутый в `synchronized`, хотя он читает только локальные переменные. Что правильнее?

*Подсказка:* Синхронизация нужна, если есть совместный доступ.

**Ответ:**

- **Убрать `synchronized`, так как нет совместного состояния**
- Оставить `synchronized`, «на всякий случай»

- Заменить на ReentrantLock
- Добавить volatile к локальным переменным

## 🔮 Шаг 02:

### Миссия 2

Мария обнаружила, что метод, помеченный synchronized, вызывается десятки тысяч раз в секунду, но не получает конкурентного доступа. Как поступить?

Подсказка: Если конкуренция не возникает, синхронизация — лишний overhead.

**Ответ:**

- **Удалить synchronized после профилирования**
- Оставить синхронизацию, чтобы быть осторожной
- Замечание: кэшировать результат внутри synchronized
- Перенести синхронизацию в вызывающий код

## 🔮 Шаг 03:

### Миссия 3

Петр использует блок synchronized вокруг чтения поля, помеченного volatile. Что верно?

Подсказка: volatile уже обеспечивает видимость.

**Ответ:**

- **Можно удалить synchronized, оставить volatile**
- Нужно добавить ещё synchronized на запись
- volatile не нужен, только synchronized достаточно
- Оставить оба для безопасности

## 🔮 Шаг 04:

### Миссия 5

📄 Сопоставьте ситуацию с оптимальным решением по синхронизации.

Ситуация:

**Ответ:**

- Только чтение общего поля после инициализации
- Запись редко, чтение часто
- Многие потоки читают и пишут часто
- Критическая секция вычисляет значение и кеширует его

Решение:

1. Без синхронизации (immutable)
2. Использовать ReadWriteLock
3. Использовать synchronized или ReentrantLock
4. Использовать двойную проверку в synchronized блоке (double-checked locking)

Подсказка: Подумайте, при каком сценарии какой механизм оправдан.

- **Только чтение общего поля после инициализации** — Без синхронизации (immutable)
- **Запись редко, чтение часто** — Использовать ReadWriteLock
- **Многие потоки читают и пишут часто** — Использовать synchronized или ReentrantLock
- **Критическая секция вычисляет значение и кеширует его** — Использовать двойную проверку в synchronized блоке (double-checked locking)

## 🔦 Шаг 05:

### Миссия 6

📖 Сопоставьте механизм с его характеристикой при уменьшении синхронизации.

Механизм:

Ответ:

- `volatile`
- `ReadWriteLock`
- `AtomicReference` / `AtomicInteger`
- `ThreadLocal`

Характеристика:

1. Поддержка неблокирующих операций над простым значением
2. Раздельный доступ на чтение/запись
3. Гарантия видимости без блокировки
4. Каждый поток хранит собственные данные, без синхронизации

Подсказка: Подумайте: `volatile` — видимость, `atomic` — атомарность, `threadlocal` — независимость.

- **`volatile`** — Гарантия видимости без блокировки
- **`ReadWriteLock`** — Раздельный доступ на чтение/запись
- **`AtomicReference` / `AtomicInteger`** — Поддержка неблокирующих операций над простым значением
- **`ThreadLocal`** — Каждый поток хранит собственные данные, без синхронизации

## 🔦 Шаг 06:

### Миссия 7

📖 Сопоставьте тип избыточной синхронизации с её симптомом в профиле.

Тип синхронизации:

Ответ:

- Синхронизация чтения неизменяемого поля
- Синхронизация внутри часто вызываемого метода, где нет гонок
- Синхронизация вокруг блоков с локальными переменными
- Синхронизация при кэшировании значения после вычисления

Симптом в профиле:

1. Низкий процент Lock Contention, но значительное время ожидания синхронизации
2. Высокий overhead без конкуренции
3. Синхронизация не влияет на состояние, но используется часто
4. Метод становится горячим из-за частых проверок синхронизации

Подсказка: Обратитесь к тому, где синхронизация не нужна.

- **Синхронизация чтения неизменяемого поля** — Низкий процент Lock Contention, но значительное время ожидания синхронизации
- **Синхронизация внутри часто вызываемого метода, где нет гонок** — Высокий overhead без конкуренции
- **Синхронизация вокруг блоков с локальными переменными** — Синхронизация не влияет на состояние, но используется часто

- **Синхронизация при кэшировании значения после вычисления** — Метод становится горячим из-за частых проверок синхронизации

## Шаг 07:

### Миссия 8

Расположите шаги для удаления избыточной синхронизации:

1. Проанализировать профиль и найти методы с синхронизацией без контеншена
2. Изолировать синхронизацию только там, где это нужно
3. Убрать synchronized там, где доступ безопасен без него
4. Профилировать снова, чтобы убедиться, что нет регрессий

*Подсказка:* Сначала найдите проблему, затем измените, затем проверьте.

#### Ответ:

- Проанализировать профиль и найти методы с синхронизацией без контеншена
- Изолировать синхронизацию только там, где это нужно
- Убрать synchronized там, где доступ безопасен без него
- Профилировать снова, чтобы убедиться, что нет регрессий

## Шаг 08:

### Миссия 9

Расположите шаги оптимизации с использованием ReadWriteLock:

1. Определить, что операции чтения преобладают
2. Заменить synchronized на ReadWriteLock
3. Разрешать многим читателям одновременно, блокировать при записи
4. Проверить производительность по сравнению с synchronized

*Подсказка:* В сценарии «много чтений — мало записей» ReadWriteLock часто выгоднее.

#### Ответ:

- Определить, что операции чтения преобладают
- Заменить synchronized на ReadWriteLock
- Разрешать многим читателям одновременно, блокировать при записи
- Проверить производительность по сравнению с synchronized

## Шаг 09:

### Миссия 10

Расположите шаги применения Atomic-переменных вместо синхронизации:

1. Определить, что синхронизация используется для атомарных обновлений
2. Заменить synchronized на AtomicInteger / AtomicReference
3. Убрать synchronized-блоки вокруг атомарных операций
4. Проверить корректность и производительность после изменений

*Подсказка:* Atomic-класс — это неблокирующая альтернатива для простых операций.

#### Ответ:

- Определить, что синхронизация используется для атомарных обновлений
- Заменить synchronized на AtomicInteger / AtomicReference
- Убрать synchronized-блоки вокруг атомарных операций

- Проверить корректность и производительность после изменений

## Шаг 10:

### Миссия 11

Расположите шаги внедрения ThreadLocal чтобы избежать синхронизации:

1. Определить данные, которые не должны разделяться между потоками
2. Заменить общий объект на ThreadLocal-переменную
3. Удалить синхронизацию вокруг доступа к этому объекту
4. Запустить нагрузочное тестирование и профилирование

*Подсказка:* ThreadLocal дарит каждому потоку свой экземпляр — нет нужды синхронизировать.

#### Ответ:

- Определить данные, которые не должны разделяться между потоками
- Заменить общий объект на ThreadLocal-переменную
- Удалить синхронизацию вокруг доступа к этому объекту
- Запустить нагрузочное тестирование и профилирование

## Шаг 11:

### Миссия 12

Расположите этапы безопасного удаления synchronized с double-checked locking:

1. Проверить, что поле объявлено volatile
2. Внутри блока synchronized проверить условие ещё раз
3. Вынести проверку вне synchronized для быстрой ветки
4. Удалить лишние синхронизации, если они не нужны

*Подсказка:* Double-checked locking требует строгой декларации volatile и двойной проверки.

#### Ответ:

- Проверить, что поле объявлено volatile
- Внутри блока synchronized проверить условие ещё раз
- Вынести проверку вне synchronized для быстрой ветки
- Удалить лишние синхронизации, если они не нужны

## Шаг 12:

### Миссия 13

Расположите шаги анализа профиля, чтобы выявить места лишней синхронизации:

1. Найти методы с synchronized, но без Lock Contention
2. Оценить процент времени, уходящего на ожидание монитора
3. Сравнить время синхронизации с общим временем метода
4. Разметить, какие synchronized блоки можно убрать или ослабить

*Подсказка:* Обратите внимание на методы с высоким overhead, но без конкуренции.

#### Ответ:

- Найти методы с synchronized, но без Lock Contention
- Оценить процент времени, уходящего на ожидание монитора
- Сравнить время синхронизации с общим временем метода
- Разметить, какие synchronized блоки можно убрать или ослабить

### Шаг 13:

#### Миссия 14

Расположите шаги для безопасной рефакторинга критической секции:

1. Создать тесты на корректность при многопоточных сценариях
2. Удалить `synchronized`, если нет гонок в тестах
3. Запустить нагрузочные тесты и профилировать изменения
4. Откатить изменения, если появились ошибки или регрессии

При удалении синхронизации важны тесты и откат.

**Ответ:**

- Создать тесты на корректность при многопоточных сценариях
- Удалить `synchronized`, если нет гонок в тестах
- Запустить нагрузочные тесты и профилировать изменения
- Откатить изменения, если появились ошибки или регрессии

### Шаг 14:

#### Миссия 16

Расположите этапы комплексной оптимизации синхронизации:

1. Собирать профили с Lock Contention и временем синхронизации
2. Идентифицировать синхронизации без конкуренции
3. Внедрить замену `synchronized` на `volatile` / `atomic` / `ReadWriteLock` / `ThreadLocal`, где возможно
4. Профилировать после изменений и сравнить показатели

*Подсказка:* Это цикл: измерение → рефакторинг → проверка.

**Ответ:**

- Собирать профили с Lock Contention и временем синхронизации
- Идентифицировать синхронизации без конкуренции
- Внедрить замену `synchronized` на `volatile` / `atomic` / `ReadWriteLock` / `ThreadLocal`, где возможно
- Профилировать после изменений и сравнить показатели

### Шаг 15:

#### Миссия 17

Расположите полный рабочий цикл оптимизации синхронизации в приложении:

1. Запустить сбор профиля с синхронизацией всех критических секций
2. Найти методы с избыточной синхронизацией
3. Рефакторить, заменяя `synchronized` там, где можно обойтись без него
4. Запустить нагрузочные тесты после изменений
5. Собрать новый профиль и сравнить с базовым
6. Оценить улучшения и возможные регрессии
7. При необходимости откатить или скорректировать изменения

*Подсказка:* Такой цикл — стандартный путь к уменьшению избыточной синхронизации.

**Ответ:**

- Запустить сбор профиля с синхронизацией всех критических секций

- Найти методы с избыточной синхронизацией
- Рефакторить, заменяя `synchronized` там, где можно обойтись без него
- Запустить нагрузочные тесты после изменений
- Собрать новый профиль и сравнить с базовым
- Оценить улучшения и возможные регрессии
- При необходимости откатить или скорректировать изменения

## Урок 3 - Lock-free и non-blocking структуры данных

### Шаг 01:

#### Миссия 1

Аня использует `ConcurrentLinkedQueue` для очереди задач — нужно ли дополнительно синхронизировать операции `offer/poll`?

Подсказка: Это lock-free структура.

**Ответ:**

- **Нет, операции уже неблокирующие**
- Да, для безопасности стоит синхронизировать
- Нужно оборачивать в `synchronized` блок
- Использовать внешний `ReentrantLock`

### Шаг 02:

#### Миссия 2

Женя хочет использовать `ConcurrentHashMap` для кеша с многопоточными чтениями и записями. Нужно ли оборачивать вызовы `get/put` в `synchronized`?

Подсказка: `ConcurrentHashMap` предоставляет внутреннюю безопасность.

**Ответ:**

- **Нет, map уже сам справляется с конкуренцией**
- Да, для гарантии последовательности
- Да, для чтения стоит `synchronized`
- Нужно `synchronized` только в `write`

### Шаг 03:

#### Миссия 3

Коля рассматривает `AtomicInteger` vs `synchronized counter`. В чём преимущество `AtomicInteger`?

Подсказка: Обратите внимание на накладные расходы блокировки.

**Ответ:**

- **`AtomicInteger` использует CAS и не блокируется**
- Он быстрее потому что снабжён `caching`
- Он гарантирует порядок операций
- Он использует `synchronized` внутри

### Шаг 04:

#### Миссия 4

Оля хочет реализовать lock-free стек на базе `AtomicReference`. Что является ключевым при реализации `push/pop`?

Подсказка: Используйте `compare-and-set (CAS)`.

**Ответ:**

- **Использовать CAS на указатель вершины**
- Синхронизировать методы push/pop
- Использовать volatile без CAS
- Использовать ReentrantLock лишь внутри push

## Шаг 05:

### **Миссия 5**

 Сопоставьте структуру данных с её характеристикой lock-free / non-blocking.

Структура:

**Ответ:**

- ConcurrentLinkedQueue
- AtomicInteger
- Lock-free стек на базе CAS
- ConcurrentSkipListMap

Характеристика:

1. CAS-основанное обновление одиночного значения
2. Неблокирующий список из очередей
3. CAS-применение для вершины стека
4. Сложная неблокирующая сортированная map структура

Подсказка: Подумайте, какая структура что реализует.

- **ConcurrentLinkedQueue** — Неблокирующий список из очередей
- **AtomicInteger** — CAS-основанное обновление одиночного значения
- **Lock-free стек на базе CAS** — CAS-применение для вершины стека
- **ConcurrentSkipListMap** — Сложная неблокирующая сортированная map структура

## Шаг 06:

### **Миссия 6**

 Сопоставьте термин lock-free / wait-free / obstruction-free с объяснением.

Термин:

**Ответ:**

- Lock-free
- Wait-free
- Obstruction-free

Объяснение:

1. Каждая операция завершается за конечное число шагов, вне зависимости от других потоков
2. По крайней мере один поток прогрессирует в системе (время идет)
3. Операция гарантируется завершится, если она выполняется в отсутствие конкуренции

Подсказка: Wait-free — самый строгий, obstruction-free — самый слабый.

- **Lock-free** — По крайней мере один поток прогрессирует в системе (время идет)
- **Wait-free** — Каждая операция завершается за конечное число шагов, вне зависимости от других потоков

- **Obstruction-free** — Операция гарантируется завершится, если она выполняется в отсутствие конкуренции

## 🔗 Шаг 07:

### Миссия 7

📖 Сопоставьте технику реализации lock-free структуры с её преимуществом.

Техника:

**Ответ:**

- Compare-And-Swap (CAS)
- Atomic Field Updaters
- StampedReference / versioned CAS
- Elimination backoff (для стека)

Преимущество:

1. Обход проблемы ABA
2. Изменение полей без создания атомиков вручную
3. Уменьшение конфликтов на contention
4. Базовая неблокирующая операция CAS без блокировки

Подсказка: ABA решается через versioning, backoff — для уменьшения конкуренции.

- **Compare-And-Swap (CAS)** — Базовая неблокирующая операция CAS без блокировки
- **Atomic Field Updaters** — Изменение полей без создания атомиков вручную
- **StampedReference / versioned CAS** — Обход проблемы ABA
- **Elimination backoff (для стека)** — Уменьшение конфликтов на contention

## 🔗 Шаг 08:

### Миссия 8

Расположите шаги реализации простого lock-free стека:

1. Использовать AtomicReference для вершины (top)
2. В методе push: прочитать текущее top и задать новый.next = old\_top
3. С помощью CAS установить new как top, если old там всё ещё стоит
4. В методе pop: аналогично читать топ и делать CAS для удаления

Подсказка: Традиционный алгоритм push/pop через CAS.

**Ответ:**

- Использовать AtomicReference для вершины (top)
- В методе push: прочитать текущее top и задать новый.next = old\_top
- С помощью CAS установить new как top, если old там всё ещё стоит
- В методе pop: аналогично читать топ и делать CAS для удаления

## 🔗 Шаг 09:

### Миссия 9

Расположите шаги при защите от ABA с использованием StampedReference:

1. Хранить и версию вместе с указателем
2. Читать оба значения: указатель и версия
3. Делать CAS только если и указатель и версия совпадают с ожидаемым

#### 4. Увеличивать версию при каждом обновлении

*Подсказка:* ABA — когда указатель возвращается к старому состоянию без версии.

**Ответ:**

- Хранить и версию вместе с указателем
- Читать оба значения: указатель и версия
- Делать CAS только если и указатель и версия совпадают с ожидаемым
- Увеличивать версию при каждом обновлении

### Шаг 10:

#### Миссия 10

Расположите шаги оптимизированного стекового алгоритма с Elimination Backoff:

1. Попытка CAS как обычно
2. При неудаче попробовать обмен элементов с другим потоком (eliminate)
3. Если обмен не удался, откатываться и повторять CAS
4. Профилировать и настраивать backoff параметры

*Подсказка:* Elimination снижает конкуренцию при высокой нагрузке.

**Ответ:**

- Попытка CAS как обычно
- При неудаче попробовать обмен элементов с другим потоком (eliminate)
- Если обмен не удался, откатываться и повторять CAS
- Профилировать и настраивать backoff параметры

### Шаг 11:

#### Миссия 11

Расположите шаги внедрения non-blocking очереди Michael-Scott:

1. Использовать AtomicReference для head и tail
2. При enqueue: вставлять узел и делать CAS на tail.next, затем CAS на tail
3. При dequeue: читать head.next и делать CAS заменить head
4. Обрабатывать случаи, когда tail отстаёт (help advance tail)

*Подсказка:* Алгоритм Michael-Scott — классическая non-blocking очередь.

**Ответ:**

- Использовать AtomicReference для head и tail
- При enqueue: вставлять узел и делать CAS на tail.next, затем CAS на tail
- При dequeue: читать head.next и делать CAS заменить head
- Обрабатывать случаи, когда tail отстаёт (help advance tail)

### Шаг 12:

#### Миссия 12

Расположите этапы анализа и оптимизации non-blocking структуры:

1. Профилировать CAS неудачи (retries)
2. Оценить частоту конфликтов (collisions)
3. Настроить backoff стратегии или уровни спинна
4. Снова профилировать, чтобы удостовериться в улучшении

*Подсказка:* Частые попытки CAS показывают узкие места.

**Ответ:**

- Профилировать CAS неудачи (retries)
- Оценить частоту конфликтов (collisions)
- Настроить backoff стратегии или уровни спинна
- Снова профилировать, чтобы удостовериться в улучшении

### Шаг 13:

#### **Миссия 13**

Расположите шаги безопасной комбинации блокирующих и неблокирующих подходов:

1. Использовать non-blocking структуру там, где возможно
2. Оставить блокировки в критических местах с сильной конкуренцией
3. Обеспечить fallback к synchronized при CAS неудачах
4. Профилировать гибридный подход и сравнить с чистым non-blocking

*Подсказка:* Иногда гибрид даёт лучший баланс безопасности и производительности.

**Ответ:**

- Использовать non-blocking структуру там, где возможно
- Оставить блокировки в критических местах с сильной конкуренцией
- Обеспечить fallback к synchronized при CAS неудачах
- Профилировать гибридный подход и сравнить с чистым non-blocking

### Шаг 14:

#### **Миссия 14**

Расположите шаги по переходу от блокирующей очереди к lock-free варианту:

1. Выбрать non-blocking реализацию (например, Michael-Scott)
2. Заменить блокирующую очередь в коде
3. Обеспечить корректность (тесты, атомарные гарантии)
4. Провести нагрузочное тестирование и сравнение производительности

*Подсказка:* Переход требует не просто подмены, но проверки корректности.

**Ответ:**

- Выбрать non-blocking реализацию (например, Michael-Scott)
- Заменить блокирующую очередь в коде
- Обеспечить корректность (тесты, атомарные гарантии)
- Провести нагрузочное тестирование и сравнение производительности

### Шаг 15:

#### **Миссия 15**

Расположите этапы рефакторинга lock-free структуры с versioning и fallback:

1. Добавить версионирование (StampedReference) для базовых CAS операций
2. Реализовать fallback путь с простым CAS или synchronized
3. Обрабатывать случаи, когда версия не совпадает (итерации)
4. Профилировать, чтобы убедиться, что fallback не часто используется

*Подсказка:* Versioning + fallback повышает устойчивость к ABA и к конфликтам.

**Ответ:**

- Добавить версионирование (StampedReference) для базовых CAS операций
- Реализовать fallback путь с простым CAS или synchronized
- Обращать случаи, когда версия не совпадает (итерации)
- Профилировать, чтобы убедиться, что fallback не часто используется

 **Шаг 16:**

**Миссия 16**

Расположите шаги комплексной проверки и отладки lock-free структуры:

1. Разработать стресс-тесты с высоким уровнем конкуренции
2. Собрать метрики CAS retries, failures, latency
3. Сравнить с альтернативными реализациями (блокирующими)
4. Подправить backoff алгоритмы, versioning, fallback

*Подсказка:* Lock-free структуры требуют особого тестирования под нагрузкой.

**Ответ:**

- Разработать стресс-тесты с высоким уровнем конкуренции
- Собрать метрики CAS retries, failures, latency
- Сравнить с альтернативными реализациями (блокирующими)
- Подправить backoff алгоритмы, versioning, fallback

 **Шаг 17:**

**Миссия 17**

Расположите полный жизненный цикл разработки lock-free компонента:

1. Выбрать подходящую lock-free структуру или алгоритм
2. Реализовать CAS-подход с учетом ABA и fallback
3. Покрыть многопоточными тестами сценарии гонок
4. Провести нагрузочное тестирование и собрать метрики
5. Оптимизировать backoff, versioning, устранить лишние retry
6. Сравнить с блокирующими аналогами и убедиться в выигрышах
7. Внедрить в продакшен и мониторить эффекты под нагрузкой

*Подсказка:* Lock-free — это не просто алгоритм, это целый жизненный цикл: проектирование, тест, оптимизация.

**Ответ:**

- Выбрать подходящую lock-free структуру или алгоритм
- Реализовать CAS-подход с учетом ABA и fallback
- Покрыть многопоточными тестами сценарии гонок
- Провести нагрузочное тестирование и собрать метрики
- Оптимизировать backoff, versioning, устранить лишние retry
- Сравнить с блокирующими аналогами и убедиться в выигрышах
- Внедрить в продакшен и мониторить эффекты под нагрузкой

 **Урок 4 - Batch processing и task partitioning**

 **Шаг 01:**

**Миссия 1**

Сопоставьте термины с их значением в контексте пакетной обработки задач.

Термины:

**Ответ:**

- Task partitioning
- Batch size
- Load balancing
- Task granularity

Определения:

1. **Степень детализации задач** — насколько мелкими являются подзадачи
2. Механизм распределения работы между потоками
3. Общее число элементов, обрабатываемых за одну операцию
4. Разбиение задачи на части

Подсказка: Batch size и granularity тесно связаны, но не синонимы.

- **Task partitioning** — Разбиение задачи на части
- **Batch size** — Общее число элементов, обрабатываемых за одну операцию
- **Load balancing** — Механизм распределения работы между потоками
- **Task granularity** — Степень детализации задач — насколько мелкими являются подзадачи

## Шаг 02:

### Миссия 2

Сопоставьте стратегию с её эффектом в многопоточном батчинг-коде.

Стратегии:

**Ответ:**

- Фиксированный размер батча
- Динамическое масштабирование батча
- Перераспределение задач между потоками во время выполнения
- Параллелизм на уровне блоков данных

Эффекты:

1. Уменьшение накладных расходов, если размер выбран оптимально
2. Позволяет адаптироваться к вариации времени обработки элементов
3. Позволяет сглаживать нагрузку при неравномерном распределении данных
4. Ускоряет обработку больших данных одновременно несколькими потоками

Подсказка: Подумайте, какая стратегия позволяет реагировать на изменчивость черёд времени.

- **Фиксированный размер батча** — Уменьшение накладных расходов, если размер выбран оптимально
- **Динамическое масштабирование батча** — Позволяет адаптироваться к вариации времени обработки элементов
- **Перераспределение задач между потоками во время выполнения** — Позволяет сглаживать нагрузку при неравномерном распределении данных
- **Параллелизм на уровне блоков данных** — Ускоряет обработку больших данных одновременно несколькими потоками

## 🔮 Шаг 03:

### Миссия 3

Сопоставьте виды узких мест с тем, как они проявляются в batch & task partitioning.

Тип узкого места:

**Ответ:**

- Низкая параллельность
- Большие накладные расходы
- Неравномерное распределение нагрузки
- Зависание потока из-за резервного ожидания

Как проявляется:

1. Один или несколько потоков простаивают, пока один обрабатывает большой блок
2. Слишком маленький размер батча ведёт к чрезмерному времени переключения задач
3. Когда один поток получает слишком много задач, другие меньше
4. Когда один поток ждет данные, пока другие всё ещё работают

Подсказка: Подумайте, какой сценарий соответствует «лишним переключениям» и «ожиданию».

- **Низкая параллельность** — Один или несколько потоков простаивают, пока один обрабатывает большой блок
- **Большие накладные расходы** — Слишком маленький размер батча ведёт к чрезмерному времени переключения задач
- **Неравномерное распределение нагрузки** — Когда один поток получает слишком много задач, другие меньше
- **Зависание потока из-за резервного ожидания** — Когда один поток ждет данные, пока другие всё ещё работают

## 🔮 Шаг 04:

### Миссия 4

Сопоставьте персонажей и их подход в задаче батчей:

Персонажи:

**Ответ:**

- Олег
- Мария
- Антон
- Ирина

Подходы:

1. Выбирает очень крупные батчи, ненавидит переключения
2. Разрезает на очень мелкие части, чтобы быть гибким
3. Следит за статистикой и динамически корректирует размер
4. Оставляет неизменные блоки и перераспределяет задачи между потоками

Подсказка: Представьте, кто из них консервативный, а кто адаптивный.

- **Олег** — Выбирает очень крупные батчи, ненавидит переключения
- **Мария** — Разрезает на очень мелкие части, чтобы быть гибким
- **Антон** — Следит за статистикой и динамически корректирует размер
- **Ирина** — Оставляет неизменные блоки и перераспределяет задачи между потоками

## Шаг 05:

### Миссия 5

Расположите этапы оптимизации batch processing от простых правок к финальной проверке:

1. Проверка результата тестами
2. Замена фиксированного размера на динамическое масштабирование
3. Сбор метрик производительности (время обработки, накладные расходы)
4. Моделирование разных размеров батча экспериментально

Подсказка: Сначала собираем данные, затем экспериментируем, затем внедряем изменение и тестируем итог.

#### Ответ:

- Сбор метрик производительности (время обработки, накладные расходы)
- Моделирование разных размеров батча экспериментально
- Замена фиксированного размера на динамическое масштабирование
- Проверка результата тестами

## Шаг 06:

### Миссия 6

Расположите компоненты адаптивного батчинг-механизма:

1. Обработка данных текущего батча
2. Сбор статистики времени исполнения
3. Анализ метрик и корректировка размера следующего батча
4. Инициация нового батча с рассчитанным размером

Подсказка: Каждый цикл: обработка → сбор → анализ → запуск нового.

#### Ответ:

- Обработка данных текущего батча
- Сбор статистики времени исполнения
- Анализ метрик и корректировка размера следующего батча
- Инициация нового батча с рассчитанным размером

## Шаг 07:

### Миссия 7

Расположите шаги распределения задач между потоками:

1. Получение списка доступных задач
2. Оценка текущей загрузки потоков
3. Назначение задач тем потокам, у которых меньше загрузка
4. Запуск потоков на выполнение задач

Подсказка: Выбрать сначала что делать, потом кому, потом запустить.

#### Ответ:

- Получение списка доступных задач
- Оценка текущей загрузки потоков
- Назначение задач тем потокам, у которых меньше загрузка
- Запуск потоков на выполнение задач

## Шаг 08:

### Миссия 8

Расположите действия от наименее затратного к наиболее затратному с точки зрения переключений потоков:

1. Очень крупный батч
2. Умеренно крупный батч
3. Средний батч
4. Очень мелкий батч

Подсказка: Чем меньше батч, тем чаще переключения.

#### Ответ:

- Очень крупный батч
- Умеренно крупный батч
- Средний батч
- Очень мелкий батч

## Шаг 09:

### Миссия 9

Расположите стратегии оптимизации по возрастанию сложности их внедрения:

1. Статический фиксированный батч
2. Перераспределение задач между потоками
3. Динамическое масштабирование размера батча
4. Авто-тюнинг на базе адаптивного алгоритма

Подсказка: Сначала простые приемы, затем сложные.

#### Ответ:

- Статический фиксированный батч
- Перераспределение задач между потоками
- Динамическое масштабирование размера батча
- Авто-тюнинг на базе адаптивного алгоритма

## Шаг 10:

### Миссия 10

Расположите стадии анализа производительности при оптимизации batch-процесса:

1. Инструментирование кода и сбор метрик
2. Выявление узких мест
3. Генерация гипотез изменения размера батча
4. Тестирование гипотез и сравнение результатов

Подсказка: Линейный аналитический цикл: измерить, проанализировать, предложить, проверить.

#### Ответ:

- Инструментирование кода и сбор метрик
- Выявление узких мест
- Генерация гипотез изменения размера батча
- Тестирование гипотез и сравнение результатов

## Шаг 11:

### Миссия 11

Расположите действия, когда адаптивный механизм слишком медленно реагирует на изменения:

1. Уменьшить шаг корректировки размера
2. Увеличить частоту обновления метрик
3. Добавить «заглушку» минимального изменения размера
4. Пересмотреть алгоритм адаптации полностью

Подсказка: Сначала лёгкие правки, потом серьёзные изменения.

#### Ответ:

- Уменьшить шаг корректировки размера
- Увеличить частоту обновления метрик
- Добавить «заглушку» минимального изменения размера
- Пересмотреть алгоритм адаптации полностью

## Шаг 12:

### Миссия 12

Расположите направления дальнейшего улучшения после базовой оптимизации:

1. Интеграция адаптивного масштабирования батчей
2. Параллелизм внутри батчей (разделение батча дальше)
3. Оптимизация алгоритма обработки внутри батча
4. Непрерывный мониторинг и авто-тюнинг в продакшен

Подсказка: От логики к глубине к контролю.

#### Ответ:

- Интеграция адаптивного масштабирования батчей
- Параллелизм внутри батчей (разделение батча дальше)
- Оптимизация алгоритма обработки внутри батча
- Непрерывный мониторинг и авто-тюнинг в продакшен

## Шаг 13:

### Миссия 13

Анна проектирует модуль, который читает большую партию сообщений и обрабатывает их в батчах. Какой выбор наиболее разумен для минимизации накладных расходов?

Подсказка: Частые мелкие операции могут быть дороже одной большой.

#### Ответ:

- **Группировать сообщения в батчи размером N и обрабатывать**
- Обработать каждое сообщение сразу
- Откладывать все до конца дня и обработать разом

## Шаг 14:

### Миссия 14

Сопоставьте понятия с их смыслами в контексте batch/task partitioning.

Понятия:

**Ответ:**

- Batch size
- Task partitioning
- Load balancing
- Granularity

Определения:

1. Количество элементов, обрабатываемых за одну операцию
2. Разделение задачи на подзадачи
3. Распределение работы, чтобы потоки были загружены равномерно
4. Уровень детализации, насколько мелкими являются части

Подсказка: Batch size и granularity часто путают.

- **Batch size** — Количество элементов, обрабатываемых за одну операцию
- **Task partitioning** — Разделение задачи на подзадачи
- **Load balancing** — Распределение работы, чтобы потоки были загружены равномерно
- **Granularity** — Уровень детализации, насколько мелкими являются части

 **Шаг 15:**

**Миссия 15**

Сопоставьте стратегии с их преимуществами:

Стратегии:

**Ответ:**

- Фиксированный батч
- Динамическое масштабирование батча
- Перераспределение задач в процессе
- Иерархическое разбиение

Преимущества:

1. Простота реализации
2. Адаптация к изменчивым временам задач
3. Сглаживание локальных перегрузок
4. Подзадачи можно дальше распараллелить внутри

Подсказка: Подумайте, что даёт гибкость, а что — масштабируемость.

- **Фиксированный батч** — Простота реализации
- **Динамическое масштабирование батча** — Адаптация к изменчивым временам задач
- **Перераспределение задач в процессе** — Сглаживание локальных перегрузок
- **Иерархическое разбиение** — Подзадачи можно дальше распараллелить внутри

 **Шаг 16:**

**Миссия 16**

Сопоставьте виды узких мест и их симптомы:

Тип узкого места:

**Ответ:**

- Высокие накладные расходы

- Неравномерное распределение
- Низкая параллельность
- Зависания ожидания

Симптомы:

1. Много переключений, низкая эффективность
2. Один поток сильно перегружен, другие простые
3. Потоки простаивают, пока один обрабатывает
4. Поток ждёт данных от других

Подсказка: Подумайте, какой симптом указывает на что.

- **Высокие накладные расходы** — Много переключений, низкая эффективность
- **Неравномерное распределение** — Один поток сильно перегружен, другие простые
- **Низкая параллельность** — Потоки простаивают, пока один обрабатывает
- **Зависания ожидания** — Поток ждёт данных от других

## 🔮 Шаг 17:

### Миссия 17

Сопоставьте участников и их подход:

Люди:

**Ответ:**

- Саша
- Оля
- Павел
- Наташа

Подход:

1. **Использует статичные батчи** — проще всего
2. **Каждая задача** — максимально мелкая  
Регулирует размер батча на основе метрик
3. Внедряет перераспределение между потоками

Подсказка: Кто консервативен, а кто динамичен?

- **Саша** — Использует статичные батчи — проще всего
- **Оля** — Каждая задача — максимально мелкая
- **Павел** — Регулирует размер батча на основе метрик
- **Наташа** — Внедряет перераспределение между потоками

## 🔮 Шаг 18:

### Миссия 18

Сопоставьте типы механизмов адаптации и когда они применимы:

Механизмы:

**Ответ:**

- Фиксированный шаг изменения
- Пропорциональный адаптивный
- Пороговая корректировка
- Авто-тюнинг на основе ML

Когда применить:

1. **Когда прирост времени невелик** — мелкие корректировки
2. Относительно простые правила и линейная реакция
3. Только при превышении порога менять
4. При больших объёмах данных и сложных зависимостях

Подсказка: Подумайте, когда нужен порог, а когда — сложный подход.

- **Фиксированный шаг изменения** — Когда прирост времени невелик — мелкие корректировки
- **Пропорциональный адаптивный** — Относительно простые правила и линейная реакция
- **Пороговая корректировка** — Только при превышении порога менять
- **Авто-тюнинг на основе ML** — При больших объёмах данных и сложных зависимостях

## Шаг 19:

### Миссия 19

Расположите шаги оптимизации batch-процесса от начала до конца:

1. Сбор метрик производительности
2. Формирование гипотез изменения размера
3. Проведение экспериментов с разными размерами
4. Выбор лучшего варианта и внедрение

Подсказка: Измерить → попробовать → выбрать → внедрить.

**Ответ:**

- Сбор метрик производительности
- Формирование гипотез изменения размера
- Проведение экспериментов с разными размерами
- Выбор лучшего варианта и внедрение

## Модуль 10 - Практический проект

### Урок 1 - Проектирование многопоточной системы

## Шаг 01:

### Миссия 1

Вася проектирует систему очередей задач, где задачи обрабатываются пакетами. Что стоит выбрать в первую очередь?

Подсказка: Частые мелкие батчи могут обременить систему.

**Ответ:**

- **Начать с фиксированного разумного размера батча**
- Обработать все элементы сразу
- Разбивать на батчи минимального размера

## Шаг 02:

### Миссия 2

Наташа запускает 4 потока, каждый получает пакет на 1000 задач. Иногда один поток перегружается. Что логичнее изменить?

Подсказка: Размер пакета влияет на гибкость распределения.

**Ответ:**

- **Уменьшить размер пакета и перераспределять задачи**
- Увеличить пакет ещё больше
- Снизить число потоков

### Шаг 03:

#### **Миссия 3**

Сергей хочет уменьшить накладные расходы на переключения при частой смене задач. Что стоит сделать?

Подсказка: Очень маленькие задания приводят к большому числу переключений.

**Ответ:**

- **Увеличить минимальный размер задачи, обрабатываемой одним потоком**
- Делить задачи ещё мельче
- Снизить число потоков до единицы

### Шаг 04:

#### **Миссия 4**

Ирина планирует заранее оценить лучший размер пакета. Как ей поступить?

Подсказка: Теория полезна, но эмпирика важнее.

**Ответ:**

- **Сделать эксперименты с разными размерами и сравнить**
- Выбрать фиксированный наугад
- Делить всё до минимума

### Шаг 05:

#### **Миссия 5**

Ольга заметила, что один поток часто простаивает, пока другой занят большим пакетом. Что лучше внедрить?

Подсказка: Динамическое перераспределение задач помогает сгладить дисбаланс.

**Ответ:**

- **Внедрить перераспределение задач между потоками во время работы**
- Оставить статическое распределение
- Ограничить один поток к работе всё время

### Шаг 06:

#### **Миссия 6**

Сопоставьте термины с их значением в контексте проектирования многопоточных систем.

Понятия:

**Ответ:**

- Task partitioning
- Batch processing
- Load balancing
- Granularity

Определения:

1. Разделение задачи на подзадачи
2. Обработка группы элементов за один проход
3. Распределение задач между потоками для выравнивания нагрузки
4. Уровень детализации, насколько мелкими являются подзадачи

Подсказка: Batch и partitioning часто путают.

- **Task partitioning** — Разделение задачи на подзадачи
- **Batch processing** — Обработка группы элементов за один проход
- **Load balancing** — Распределение задач между потоками для выравнивания нагрузки
- **Granularity** — Уровень детализации, насколько мелкими являются подзадачи

## Шаг 07:

### Миссия 7

Сопоставьте стратегии распределения с их преимуществами:

Стратегии:

**Ответ:**

- Фиксированный батч
- Динамическое масштабирование
- Перераспределение задач на лету
- Иерархическое разбиение

Преимущества:

1. Простота реализации
2. Адаптация к переменным временам задач
3. Сглаживание локальных перегрузок
4. Возможность внутреннего распараллеливания подзадач

Подсказка: Подумайте, что даёт гибкость, а что — масштабируемость.

- **Фиксированный батч** — Простота реализации
- **Динамическое масштабирование** — Адаптация к переменным временам задач
- **Перераспределение задач на лету** — Сглаживание локальных перегрузок
- **Иерархическое разбиение** — Возможность внутреннего распараллеливания подзадач

## Шаг 08:

### Миссия 8

Сопоставьте типы узких мест и их проявления в многопоточной системе:

Тип узкого места:

**Ответ:**

- Высокие накладные расходы
- Неравномерное распределение
- Низкая параллельность
- Ожидание ресурсов

Проявления:

1. Много переключений и низкая эффективность

2. Один поток нагружен, остальные простаивают
3. Потоки простаивают, пока один обрабатывает крупную задачу
4. Поток ждёт данных или блокируется, пока другие работают

Подсказка: По симптомам можно распознать узкое место.

- **Высокие накладные расходы** — Много переключений и низкая эффективность
- **Неравномерное распределение** — Один поток нагружен, остальные простаивают
- **Низкая параллельность** — Потоки простаивают, пока один обрабатывает крупную задачу
- **Ожидание ресурсов** — Поток ждёт данных или блокируется, пока другие работают

## Шаг 09:

### Миссия 9

Сопоставьте разработчиков с их стилем работы:

Разработчики:

**Ответ:**

- Алексей
- Виктория
- Игорь
- Марина

Стиль:

1. Использует фиксированные батчи без изменений
2. Делит задачи на множество мелких частей
3. Адаптирует размер батча на основе метрик
4. Внедряет перераспределение задач между потоками

Подсказка: Кто статичен, кто динамичен?

- **Алексей** — Использует фиксированные батчи без изменений
- **Виктория** — Делит задачи на множество мелких частей
- **Игорь** — Адаптирует размер батча на основе метрик
- **Марина** — Внедряет перераспределение задач между потоками

## Шаг 10:

### Миссия 10

Расположите этапы проектирования алгоритма обработки задач от начала до внедрения:

1. Сбор требований и описание сценариев
2. Разбиение задач и выбор структуры потоков
3. Реализация и тестирование
4. Запуск и наблюдение за производительностью

Подсказка: Проектирование → реализация → эксплуатация.

**Ответ:**

- Сбор требований и описание сценариев
- Разбиение задач и выбор структуры потоков
- Реализация и тестирование
- Запуск и наблюдение за производительностью

## Шаг 11:

### Миссия 11

Расположите шаги построения адаптивной многопоточной системы:

1. Определение метрик и сбор данных
2. Формирование гипотез изменения
3. Реализация механизма адаптации
4. Контроль и корректировка в продакшене

Подсказка: Измерять → пробовать → реализовать → контролировать.

**Ответ:**

- Определение метрик и сбор данных
- Формирование гипотез изменения
- Реализация механизма адаптации
- Контроль и корректировка в продакшене

## Шаг 12:

### Миссия 12

Расположите этапы перераспределения задач между потоками:

1. Анализ загрузки каждого потока
2. Выбор менее загруженных потоков
3. Передача задач потокам с меньшей нагрузкой
4. Запуск потоков с новыми задачами

Подсказка: Сначала измеряем, затем распределяем, затем запускаем.

**Ответ:**

- Анализ загрузки каждого потока
- Выбор менее загруженных потоков
- Передача задач потокам с меньшей нагрузкой
- Запуск потоков с новыми задачами

## Шаг 13:

### Миссия 13

Расположите стратегии масштабирования системы по возрастанию сложности внедрения:

1. Увеличить число потоков
2. Добавить перераспределение задач
3. Внедрить динамическое масштабирование батчей
4. Авто-тюнинг на основе ML/аналитики

Подсказка: Простые изменения → сложные.

**Ответ:**

- Увеличить число потоков
- Добавить перераспределение задач
- Внедрить динамическое масштабирование батчей
- Авто-тюнинг на основе ML/аналитики

## Шаг 14:

### Миссия 14

Расположите варианты оптимизации с точки зрения стоимости внедрения (от дешёвых к дорогим):

1. Увеличение размера батчей
2. Мелкая перераспределяемая разбивка
3. Адаптивное масштабирование
4. Авто-тюнинг и ML-анализ

Подсказка: Чем умнее решение — тем выше стоимость.

#### Ответ:

- Увеличение размера батчей
- Мелкая перераспределяемая разбивка
- Адаптивное масштабирование
- Авто-тюнинг и ML-анализ

## Шаг 15:

### Миссия 15

Расположите подходы к обработке задач по накладным расходам (от наименьших к наибольшим):

1. Огромные батчи
2. Умеренно большие батчи
3. Средние батчи
4. Очень мелкие батчи

Подсказка: Меньший размер — больше переключений.

#### Ответ:

- Огромные батчи
- Умеренно большие батчи
- Средние батчи
- Очень мелкие батчи

## Шаг 16:

### Миссия 16

Расположите шаги, если адаптация системы реагирует слишком медленно:

1. Увеличить частоту обновления метрик
2. Уменьшить шаг корректировки
3. Добавить минимальный порог изменений
4. Переписать алгоритм адаптации

Подсказка: Начинайте с лёгких корректировок, затем двигайтесь к фундаментальным изменениям.

#### Ответ:

- Увеличить частоту обновления метрик
- Уменьшить шаг корректировки
- Добавить минимальный порог изменений

- Переписать алгоритм адаптации

## Шаг 17:

### Миссия 17

Расположите направления дальнейшего развития проекта по уровню сложности:

1. Расширить логирование и метрики
2. Внедрить перераспределение задач
3. Интегрировать адаптивное масштабирование батчей
4. Авто-тюнинг с ML и самообучение

Подсказка: От мониторинга к самообучению.

#### Ответ:

- Расширить логирование и метрики
- Внедрить перераспределение задач
- Интегрировать адаптивное масштабирование батчей
- Авто-тюнинг с ML и самообучение

## Урок 2 - Реализация потоков, синхронизация и ошибки

## Шаг 01:

### Миссия 1

Петя создает поток, который работает в фоне, но он заметил, что поток не запускается. Какова возможная первичная ошибка?

Подсказка: Проверьте, вызван ли метод `start()`, а не `run()`.

#### Ответ:

- Он вызвал `run()`, а не `start()`
- Не создал объект `Thread`
- Не реализовал интерфейс `Runnable`

## Шаг 02:

### Миссия 2

Маша запускает две нити, которые должны работать параллельно, но они блокируются друг другом. Что первоочередно проверить?

Подсказка: Возможно, оба потока ждут одну и ту же блокировку.

#### Ответ:

- Наличие **deadlock** — циклическое ожидание локов
- Потоки не запущены
- `InterruptedException` не ловится

## Шаг 03:

### Миссия 3

Игорь использует `synchronized`-блок внутри метода, но всё равно наблюдает гонку данных. В чём может быть дело?

Подсказка: Возможно, не весь доступ к общему ресурсу синхронизирован.

**Ответ:**

- **Другие части кода обращаются к ресурсу вне synchronized**
- Synchronized блок невозможен в методе
- Нельзя синхронизировать на this

 **Шаг 04:**

#### **Миссия 4**

Ксения создала пул потоков и передала задачи, но видит RejectedExecutionException. Что вероятно не настроено?

Подсказка: Проверьте параметры очереди и лимиты пула.

**Ответ:**

- **Очередь пула переполнена и нет запасного пространства**
- Пул не создан
- Метод submit() не используется

 **Шаг 05:**

#### **Миссия 5**

Саша ловит InterruptedException внутри потока, но затем игнорирует прерывание и продолжает работу. Почему это опасно?

Подсказка: Игнорирование прерывания мешает корректной остановке.

**Ответ:**

- **Поток не реагирует на запрос остановки**
- InterruptedException никогда не выбрасывается
- Нельзя ловить InterruptedException

 **Шаг 06:**

#### **Миссия 6**

Сопоставьте термины с описаниями ошибок многопоточности:

Термины:

**Ответ:**

- Deadlock
- Race condition
- Starvation
- Livelock

Описание ошибок:

1. Потоки постоянно меняют состояние без продвижения
2. Один поток захватывает ресурс, блокируя других навсегда
3. Несогласованность при одновременном доступе к данным
4. Некоторые потоки не получают доступа к ресурсам долгое время

Подсказка: Deadlock — циклическое ожидание; race — рассинхронизация; starvation — голод; livelock — активность без прогресса.

- **Deadlock** — Один поток захватывает ресурс, блокируя других навсегда
- **Race condition** — Несогласованность при одновременном доступе к данным
- **Starvation** — Некоторые потоки не получают доступа к ресурсам долгое время

- **Livelock** — Потоки постоянно меняют состояние без продвижения

## Шаг 07:

### Миссия 7

Сопоставьте методы синхронизации с их характеристиками:

Методы:

**Ответ:**

- synchronized
- ReentrantLock
- ReadWriteLock
- AtomicInteger

Характеристики:

1. Поддерживает явное lock/unlock и условия ожидания
2. Простая блокировка метода или блока
3. Разделение чтения и записи с разными блокировками
4. Низкоуровневая атомарная операция без блокировок

Подсказка: Подумайте, какой механизм даёт гибкость, а какой — простоту.

- **synchronized** — Простая блокировка метода или блока
- **ReentrantLock** — Поддерживает явное lock/unlock и условия ожидания
- **ReadWriteLock** — Разделение чтения и записи с разными блокировками
- **AtomicInteger** — Низкоуровневая атомарная операция без блокировок

## Шаг 08:

### Миссия 8

Сопоставьте виды ошибок с участками кода:

Ошибка:

**Ответ:**

- Несинхронизированное поле
- Неправильное разблокирование
- Блокировка ресурсов в неверном порядке
- Игнорирование прерывания

Следствие:

1. Deadlock при входе в synchronized блоки
2. Race condition на поле
3. IllegalMonitorStateException или застой
4. Поток не завершается корректно при interrupt

Подсказка: Проверьте порядок lock/unlock и правильность прерывания.

- **Несинхронизированное поле** — Race condition на поле
- **Неправильное разблокирование** — IllegalMonitorStateException или застой
- **Блокировка ресурсов в неверном порядке** — Deadlock при входе в synchronized блоки
- **Игнорирование прерывания** — Поток не завершается корректно при interrupt

## 🔧 Шаг 09:

### Миссия 9

Сопоставьте состояния потока с их описанием:

Состояния:

**Ответ:**

- NEW
- RUNNABLE
- BLOCKED
- TERMINATED

Описание:

1. Поток завершил выполнение
2. Поток создаётся, но start() не вызван
3. Поток готов к выполнению, но ожидает CPU
4. Поток ждёт захвата монитора/блока

Подсказка: NEW — до запуска; BLOCKED — под монитором.

- **NEW** — Поток создаётся, но start() не вызван
- **RUNNABLE** — Поток готов к выполнению, но ожидает CPU
- **BLOCKED** — Поток ждёт захвата монитора/блока
- **TERMINATED** — Поток завершил выполнение

## 🔧 Шаг 10:

### Миссия 10

Расположите стадии запуска и завершения потока в правильном порядке:

1. Создание объекта Thread
2. Вызов start()
3. Метод run() выполняется
4. Поток завершает работу и становится TERMINATED

Подсказка: Thread: new → start → run → завершение.

**Ответ:**

- Создание объекта Thread
- Вызов start()
- Метод run() выполняется
- Поток завершает работу и становится TERMINATED

## 🔧 Шаг 11:

### Миссия 11

Расположите этапы обработки блокировки с ReentrantLock:

1. lock()
2. код внутри защищённого блока
3. unlock()
4. finally-блок, гарантирующий unlock

Подсказка: Always unlock в finally.

**Ответ:**

- lock()
- код внутри защищённого блока
- finally-блок, гарантирующий unlock
- unlock()

 **Шаг 12:**

**Миссия 12**

Расположите шаги от возникновения deadlock до диагностики:

1. Потоки захватывают ресурсы в несогласованном порядке
2. **Оба потока ждут друг друга** — взаимное ожидание
3. Система замирает
4. Диагностика с помощью потокового дампа (stack trace)

Подсказка: От нарушения порядка → взаимное ожидание → замирание → диагностика.

**Ответ:**

- Потоки захватывают ресурсы в несогласованном порядке
- **Оба потока ждут друг друга** — взаимное ожидание
- Система замирает
- Диагностика с помощью потокового дампа (stack trace)

 **Шаг 13:**

**Миссия 13**

Расположите стадии обработки InterruptedException в корректном порядке:

1. Метод вызывает sleep или wait
2. InterruptedException выбрасывается
3. catch-блок получает исключение
4. Поток корректно завершает работу или очищает ресурсы

Подсказка: Сначала ожидается прерывание, затем обработка, затем завершение.

**Ответ:**

- Метод вызывает sleep или wait
- InterruptedException выбрасывается
- catch-блок получает исключение
- Поток корректно завершает работу или очищает ресурсы

 **Шаг 14:**

**Миссия 14**

Расположите подходы к устранению гонок по степени надёжности:

1. Использование volatile
2. synchronized блок
3. Lock / ReentrantLock
4. Использование атомарных классов и структур данных

Подсказка: volatile — слабый механизм, атомики — безопаснее для отдельных полей.

**Ответ:**

- Использование volatile
- synchronized блок
- Lock / ReentrantLock
- Использование атомарных классов и структур данных

 **Шаг 15:**

**Миссия 15**

Расположите методы от наименее дорогого к наиболее затратному по блокировкам:

1. Чтение с ReadLock при отсутствии записи
2. Использование атомиков для инкремента
3. synchronized блок
4. Полная эксклюзивная блокировка всех ресурсов

Подсказка: Чем шире блокировка — тем дороже.

**Ответ:**

- Чтение с ReadLock при отсутствии записи
- Использование атомиков для инкремента
- synchronized блок
- Полная эксклюзивная блокировка всех ресурсов

 **Шаг 16:**

**Миссия 16**

Расположите шаги от ошибки IllegalMonitorStateException до её устранения:

1. Попытка вызвать notify()/wait() вне synchronized
2. Бросается IllegalMonitorStateException
3. Идентифицируем место вызова notify()/wait()
4. Перенос этих вызовов внутрь synchronized блока

Подсказка: notify/wait должны быть внутри синхронизированного блока.

**Ответ:**

- Попытка вызвать notify()/wait() вне synchronized
- Бросается IllegalMonitorStateException
- Идентифицируем место вызова notify()/wait()
- Перенос этих вызовов внутрь synchronized блока

 **Шаг 17:**

**Миссия 17**

Расположите этапы защиты от deadlock в многопоточном проекте:

1. Выбрать фиксированный глобальный порядок захвата ресурсов
2. Внедрить таймауты при попытках захвата блокировок
3. Детектировать циклическое ожидание и логировать
4. Использовать tryLock с возвратом и откатом операции

Подсказка: От простых правил к активной детекции и откату.

**Ответ:**

- Выбрать фиксированный глобальный порядок захвата ресурсов
- Внедрить таймауты при попытках захвата блокировок
- Детектировать циклическое ожидание и логировать
- Использовать tryLock с возвратом и откатом операции

## Урок 3 - Интеграция Executor, CompletableFuture, Concurrent Collections

### Шаг 01:

#### **Миссия 1**

Олег решил ускорить загрузку данных из нескольких API и использовал ExecutorService, но всё равно видит, что задачи выполняются последовательно. Что, скорее всего, пошло не так?

Подсказка: Executor нужно не только создать, но и использовать для запуска задач.

**Ответ:**

- **Олег запустил задачи напрямую, а не через submit()**
- ExecutorService не поддерживает параллелизм
- Он использовал Thread вместо Executor
- Методы API слишком быстрые

### Шаг 02:

#### **Миссия 2**

Катя запускает несколько задач через CompletableFuture.runAsync, но они не используют кастомный пул. Почему?

Подсказка: CompletableFuture может использовать ForkJoinPool по умолчанию, если не указать явный Executor.

**Ответ:**

- **Она не передала Executor как аргумент в runAsync()**
- Кастомный Executor был слишком мал
- Задачи не совместимы с CompletableFuture
- runAsync() не работает с пулом потоков

### Шаг 03:

#### **Миссия 3**

Макс использует submit() с Callable, но не получает результата. Что он забыл?

Подсказка: Callable возвращает результат через Future, но его нужно явно запросить.

**Ответ:**

- **Он не вызвал get() у возвращённого Future**
- Он должен был использовать Thread вместо Callable
- Callable не возвращает значения
- Он не переопределил run()

### Шаг 04:

#### **Миссия 4**

Женя использует ConcurrentHashMap и видит, что значения неожиданно меняются. В чём может быть дело?

Подсказка: Даже потокобезопасные коллекции не спасают от логических ошибок.

**Ответ:**

- **В нескольких потоках он модифицирует значение через `get()` и `put()`, без атомарной операции**
- `ConcurrentHashMap` не потокобезопасен
- Его `Executor` работает в однопоточном режиме
- Он не вызывает `synchronized` вручную

## Шаг 05:

### Миссия 5

Таня собирает список результатов с помощью `CompletableFuture.allOf(...)`, но получает `null`. Почему?

Подсказка: `CompletableFuture.allOf` не возвращает список значений напрямую.

**Ответ:**

- **`allOf()` возвращает `CompletableFuture`, нужно собирать значения вручную**
- Нельзя использовать `allOf()` с результатами
- Её задачи не завершаются
- Она использует `runAsync()`

## Шаг 06:

### Миссия 6

Сопоставьте методы `CompletableFuture` с их назначением:

**Ответ:**

- `thenApply()`
- `thenAccept()`
- `thenRun()`
- `exceptionally()`

1. Обработка результата с возвращением нового значения
2. Выполнение действия без возвращаемого значения
3. Запуск кода после завершения без доступа к результату
4. Обработка ошибок и возврат альтернативы

Подсказка: Обратите внимание, какие методы возвращают значения, а какие нет.

- **`thenApply()` — Обработка результата с возвращением нового значения**
- **`thenAccept()` — Выполнение действия без возвращаемого значения**
- **`thenRun()` — Запуск кода после завершения без доступа к результату**
- **`exceptionally()` — Обработка ошибок и возврат альтернативы**

## Шаг 07:

### Миссия 7

Сопоставьте методы `ExecutorService` с их описанием:

**Ответ:**

- `submit()`
- `invokeAll()`
- `shutdown()`

- `awaitTermination()`

1. Ожидание завершения всех задач после остановки пула
2. Остановка приёма новых задач
3. Запуск одной задачи и получение `Future`
4. Запуск всех задач и ожидание их завершения

Подсказка: Разные методы контролируют не только запуск, но и завершение пула.

- **`submit()`** — Запуск одной задачи и получение `Future`
- **`invokeAll()`** — Запуск всех задач и ожидание их завершения
- **`shutdown()`** — Остановка приёма новых задач
- **`awaitTermination()`** — Ожидание завершения всех задач после остановки пула

## Шаг 08:

### Миссия 8

Виталик пытается выбрать подходящую concurrent-коллекцию. Помогите ему сопоставить тип коллекции с её применением:

**Ответ:**

- `ConcurrentLinkedQueue`
  - `CopyOnWriteArrayList`
  - `ConcurrentHashMap`
  - `LinkedBlockingQueue`
1. Очередь без блокировки, подходит для высоконагруженных задач
  2. Чтение чаще, чем запись, список редко изменяется
  3. Хэш-таблица для параллельного доступа с сегментами
  4. Блокирующая очередь с возможностью ожидания элементов

Подсказка: Учитывайте, как часто предполагается модификация и сколько потоков одновременно обращаются к коллекции.

- **`ConcurrentLinkedQueue`** — Очередь без блокировки, подходит для высоконагруженных задач
- **`CopyOnWriteArrayList`** — Чтение чаще, чем запись, список редко изменяется
- **`ConcurrentHashMap`** — Хэш-таблица для параллельного доступа с сегментами
- **`LinkedBlockingQueue`** — Блокирующая очередь с возможностью ожидания элементов

## Шаг 09:

### Миссия 9

Сопоставьте методы `CompletableFuture` с их поведением при работе с несколькими задачами:

**Ответ:**

- `allOf()`
  - `anyOf()`
  - `join()`
  - `get()`
1. Ожидание завершения всех переданных задач
  2. Ожидание завершения хотя бы одной задачи
  3. Ожидание результата без checked-исключений
  4. Ожидание результата с обязательной обработкой исключений

Подсказка: Некоторые методы удобны, но не требуют try-catch. Некоторые — наоборот.

- **allOf()** — Ожидание завершения всех переданных задач
- **anyOf()** — Ожидание завершения хотя бы одной задачи
- **join()** — Ожидание результата без checked-исключений
- **get()** — Ожидание результата с обязательной обработкой исключений

## 🔮 Шаг 10:

### Миссия 10

Сопоставьте поведение различных типов очередей из пакета `java.util.concurrent`:

**Ответ:**

- `ArrayBlockingQueue`
  - `LinkedBlockingQueue`
  - `PriorityBlockingQueue`
  - `DelayQueue`
1. Очередь с ограниченным размером и блокировкой
  2. Очередь с приоритетом и блокировкой
  3. Очередь с отложенной доставкой элементов
  4. Очередь на связном списке, может быть неограниченной

Подсказка: Приоритеты, лимиты и задержки — ключевые различия.

- **`ArrayBlockingQueue`** — Очередь с ограниченным размером и блокировкой
- **`LinkedBlockingQueue`** — Очередь на связном списке, может быть неограниченной
- **`PriorityBlockingQueue`** — Очередь с приоритетом и блокировкой
- **`DelayQueue`** — Очередь с отложенной доставкой элементов

## 🔮 Шаг 11:

### Миссия 11

Никита реализует асинхронную цепочку обработки запроса. Расположите шаги в правильном порядке:

1. Создание `CompletableFuture` с `supplyAsync()`
2. Преобразование результата через `thenApply()`
3. Логирование финального результата через `thenAccept()`
4. Обработка возможной ошибки через `exceptionally()`

Подсказка: Сначала создаётся цепочка, потом обрабатывается результат, затем ошибки.

**Ответ:**

- Создание `CompletableFuture` с `supplyAsync()`
- Преобразование результата через `thenApply()`
- Логирование финального результата через `thenAccept()`
- Обработка возможной ошибки через `exceptionally()`

## 🔮 Шаг 12:

### Миссия 12

Упорядочите действия для корректной работы с `ExecutorService`:

1. Создание пула потоков через `Executors.newFixedThreadPool()`

2. Отправка задач с помощью `submit()`
3. Вызов `shutdown()`
4. Ожидание завершения с помощью `awaitTermination()`

Подсказка: После выполнения задач нужно корректно закрыть пул, иначе программа может не завершиться.

**Ответ:**

- Создание пула потоков через `Executors.newFixedThreadPool()`
- Отправка задач с помощью `submit()`
- Вызов `shutdown()`
- Ожидание завершения с помощью `awaitTermination()`

### Шаг 13:

#### Миссия 13

Расположите шаги использования `CopyOnWriteArrayList` для многопоточного чтения:

1. Создание списка `CopyOnWriteArrayList`
2. Добавление элементов
3. Запуск потоков, читающих список
4. Удаление элементов после чтения

Подсказка: Эта коллекция идеально подходит, когда нужно читать много, а менять — редко.

**Ответ:**

- Создание списка `CopyOnWriteArrayList`
- Добавление элементов
- Запуск потоков, читающих список
- Удаление элементов после чтения

### Шаг 14:

#### Миссия 14

Расставьте действия при запуске нескольких задач с сохранением результатов:

1. Создание списка `Callable` задач
2. Вызов `invokeAll()` с этим списком
3. Получение списка `Future`
4. Обработка результатов с помощью `get()`

Подсказка: `invokeAll` позволяет сразу работать с группой задач и получать результаты синхронно.

**Ответ:**

- Создание списка `Callable` задач
- Вызов `invokeAll()` с этим списком
- Получение списка `Future`
- Обработка результатов с помощью `get()`

### Шаг 15:

#### Миссия 15

Алина автоматизирует агрегацию данных из нескольких микросервисов. Упорядочите шаги использования `CompletableFuture` для конкурентной загрузки:

1. Создание списка `CompletableFuture` с вызовами `supplyAsync()`
2. Комбинирование задач с помощью `allOf()`
3. Ожидание завершения `allOf` с `join()`
4. Сбор результатов из исходных `CompletableFuture`

Подсказка: `allOf` не возвращает данные напрямую, нужно сохранить оригинальные фьючеры для доступа к результатам.

**Ответ:**

- Создание списка `CompletableFuture` с вызовами `supplyAsync()`
- Комбинирование задач с помощью `allOf()`
- Ожидание завершения `allOf` с `join()`
- Сбор результатов из исходных `CompletableFuture`

## Шаг 16:

### Миссия 16

Денис реализует систему логирования, где события из нескольких потоков должны сохраняться в очереди и обрабатываться в порядке поступления. Упорядочите шаги:

1. Создание `LinkedBlockingQueue`
2. Запуск потоков, добавляющих события в очередь
3. Запуск отдельного потока-потребителя
4. Чтение и запись логов из очереди

Подсказка: Продюсеры и потребители должны работать параллельно, но очередь — общий буфер между ними.

**Ответ:**

- Создание `LinkedBlockingQueue`
- Запуск потоков, добавляющих события в очередь
- Запуск отдельного потока-потребителя
- Чтение и запись логов из очереди

## Шаг 17:

### Миссия 17

Вика оптимизирует загрузку отчётов в веб-приложении. Каждый отчёт загружается асинхронно, результат кэшируется. Упорядочите действия:

1. Проверка кэша на наличие отчёта
2. **Если нет** — запуск `CompletableFuture` с загрузкой
3. Добавление отчёта в `ConcurrentHashMap`
4. Возврат отчёта пользователю

Подсказка: Используйте конкурентную коллекцию как кэш. Асинхронная загрузка — только если отчёт ещё не сохранён.

**Ответ:**

- Проверка кэша на наличие отчёта
- **Если нет** — запуск `CompletableFuture` с загрузкой
- Добавление отчёта в `ConcurrentHashMap`

- Возврат отчёта пользователю

## Урок 4 - Оптимизация и тестирование производительности

### Шаг 01:

#### Миссия 1

Борис экспериментирует с пулом потоков и хочет выбрать правильный размер пула. Что является ключевым фактором при выборе размера пула?

Подсказка: Подумайте об I/O-зависимости задачи и количестве ядер.

**Ответ:**

- **Соотношение I/O и CPU-времени задачи**
- Произвольное большое число
- **Минимальное число — 1**
- Число потоков = число пользователей

### Шаг 02:

#### Миссия 2

Кирилл внедряет счетчики для мониторинга времени ожидания блокировок. Какой из подходов менее затратен?

Подсказка: Часто менее затратны ненавязчивые (lock-free) методы наблюдения.

**Ответ:**

- **Использовать атомарные счётчики**
- Оборачивать каждую секцию try/catch
- Создавать отдельный монитор-поток
- Вставлять паузы внутри блокировок

### Шаг 03:

#### Миссия 3

Дарья хочет протестировать, как изменится производительность при разных размерах очередей задач (task queue). Какой подход ей подойдёт?

Подсказка: Попробуйте A/B-эксперименты с нагрузкой.

**Ответ:**

- **Провести нагрузочные тесты с разными размерами очередей**
- Выбрать среднее значение из догадки
- Оставить очередь бесконечной
- Использовать синхронизацию вместо очередей

### Шаг 04:

#### Миссия 4

Егор подозревает, что конкурентный доступ к HashMap является узким местом. Какой класс стоит рассмотреть как замену?

Подсказка: В JDK есть реализации, специально для мультипоточности.

**Ответ:**

- **ConcurrentHashMap**
- Collections.synchronizedMap(HashMap)

- TreeMap
- Hashtable

## 🔗 Шаг 05:

### Миссия 5

Лена хочет измерить латентность блокировок и конкуренции между потоками в реальной нагрузке. Что из перечисленного поможет ей?

Подсказка: Использование высокоточных таймеров и профилировщиков.

#### Ответ:

- **JMH с измерением латентности**
- System.out.println с метками времени
- Использовать Thread.sleep
- Вручную считывать System.nanoTime каждый вызов

## 🔗 Шаг 06:

### Миссия 6

Задача: Сопоставление стратегий оптимизации многопоточности и подходящих ситуаций.

Стратегии:

#### Ответ:

- Уменьшение времени блокировки
- Использование lock-free алгоритмов
- Разделение данных на сегменты
- Переход на конвейерную обработку

Ситуации:

1. Потоки часто блокируются на одном мониторе
2. Высокая конкуренция на одном объекте без критических зависимостей
3. Наблюдается узкое место из-за глобального состояния
4. Последовательные этапы можно «расцепить» в конвейеры

#### 🔗 Подсказка:

Подумайте, какая стратегия подходит для каждой характерной проблемы: уменьшить время, убрать блокировки, сегментировать или строить конвейер.

- **Уменьшение времени блокировки** — Потоки часто блокируются на одном мониторе
- **Использование lock-free алгоритмов** — Высокая конкуренция на одном объекте без критических зависимостей
- **Разделение данных на сегменты** — Наблюдается узкое место из-за глобального состояния
- **Переход на конвейерную обработку** — Последовательные этапы можно «расцепить» в конвейеры

## 🔗 Шаг 07:

### Миссия 7

Задача: Сопоставьте метрику мониторинга с тем, что она показывает.

Метрики:

**Ответ:**

- Throughput (пропускная способность)
- Latency (задержка)
- CPU utilization
- Lock contention rate

Что показывает:

1. Количество операций в секунду
2. Среднее время ответа операции
3. Процент загрузки CPU
4. Частота конфликтов при захвате блокировки

🔗 **Подсказка: Подумайте: throughput — про «сколько в секунду», latency — про «время ожидания» и т.д.**

- **Throughput (пропускная способность)** — Количество операций в секунду
- **Latency (задержка)** — Среднее время ответа операции
- **CPU utilization** — Процент загрузки CPU
- **Lock contention rate** — Частота конфликтов при захвате блокировки

🔗 **Шаг 08:**

**Миссия 8**

Сопоставьте прием оптимизации с его потенциальным риском.

Прием:

**Ответ:**

- Инлайн функций внутри критической секции
- Предварительное распределение буферов (pre-allocation)
- Использование CAS-циклов
- Снижение синхронизации через локальные копии

Потенциальный риск:

1. Увеличение размера кода и сложность отладки
2. Избыточное использование памяти
3. Жесткий цикл соревнования и спиннинг
4. Непоследовательность данных / рассинхронизация

🔗 **Подсказка: Проанализируйте: что может пойти не так при агрессивной оптимизации внутри секций.**

- **Инлайн функций внутри критической секции** — Увеличение размера кода и сложность отладки
- **Предварительное распределение буферов (pre-allocation)** — Избыточное использование памяти
- **Использование CAS-циклов** — Жесткий цикл соревнования и спиннинг
- **Снижение синхронизации через локальные копии** — Непоследовательность данных / рассинхронизация

🔗 **Шаг 09:**

**Миссия 9**

Сопоставьте инструмент с тем, что он помогает измерять или анализировать в многопоточности.

Инструменты:

**Ответ:**

- JMH
- VisualVM / JFR (Java Flight Recorder)
- Flight Recorder + Разбор событий
- Асинхронные логгеры (например, Log4j2 Async)

Что он позволяет делать:

1. Точная микро-бенчмарка
2. Слежение за распределением потоков и блокировками
3. Анализ трасс блокировок и ожиданий
4. Снижать накладные расходы логирования

🔗 **Подсказка: Подумайте: JMH — для микро-бенчмарков, а другие инструменты — для анализа runtime.**

- **JMH** — Точная микро-бенчмарка
- **VisualVM / JFR (Java Flight Recorder)** — Слежение за распределением потоков и блокировками
- **Flight Recorder + Разбор событий** — Анализ трасс блокировок и ожиданий
- **Асинхронные логгеры (например, Log4j2 Async)** — Снижать накладные расходы логирования

🔗 **Шаг 10:**

### Миссия 10

Расположите этапы настройки производительного теста JMH:

1. Выбрать режим (Throughput, AverageTime и др.)
2. Настроить параметры warmup и measurement
3. Написать код теста с аннотациями @Benchmark
4. Запустить тесты и собрать статистику
5. Проанализировать результаты и исключить шумовые измерения

🔗 **Подсказка:**

Сначала пишем тест, затем конфигурируем параметры, потом запускаем и анализируем.

**Ответ:**

- Написать код теста с аннотациями @Benchmark
- Выбрать режим (Throughput, AverageTime и др.)
- Настроить параметры warmup и measurement
- Запустить тесты и собрать статистику
- Проанализировать результаты и исключить шумовые измерения

🔗 **Шаг 11:**

### Миссия 11

Расположите стадии производительного тестирования с нагрузкой:

1. Запуск контрольного сценария (baseline)
2. Изменение параметров (пул, очередь, размер байтов)

3. Сбор метрик (throughput, latency, contention)
4. Интерпретация отклонений
5. Повторный запуск под нагрузкой

🔗 **Подсказка:**

Начинаем с baseline, потом меняем параметры, запускаем, собираем и анализируем.

**Ответ:**

- Запуск контрольного сценария (baseline)
- Изменение параметров (пул, очередь, размер байтов)
- Сбор метрик (throughput, latency, contention)
- Интерпретация отклонений
- Повторный запуск под нагрузкой

🔗 **Шаг 12:**

**Миссия 12**

Расположите шаги проведения масштабного A/B теста производительности:

1. Сформулировать гипотезу (что изменится)
2. Разделить трафик на контроль и экспериментальные группы
3. Внести изменения только в экспериментальную группу
4. Собирать метрики и трассы логов
5. Сравнить статистику и принять решение

🔗 **Подсказка:**

Обращайте внимание: гипотеза → разделение → изменение → сбор → анализ.

**Ответ:**

- Сформулировать гипотезу (что изменится)
- Разделить трафик на контроль и экспериментальные группы
- Внести изменения только в экспериментальную группу
- Собирать метрики и трассы логов
- Сравнить статистику и принять решение

🔗 **Шаг 13:**

**Миссия 13**

Расположите шаги интеграционного тестирования перформанса системой:

1. Подготовить тестовую среду, похожую на боевую
2. Развернуть метрики и трассировки на тестовой среде
3. Запустить нагрузочные сценарии с “реальными” нагрузками
4. Собрать и визуализировать метрики
5. Сравнить с эталонной историей и выявить отклонения

🔗 **Подсказка: Начинаем с подготовки среды → метрики → нагрузка → сбор → анализ.**

**Ответ:**

- Подготовить тестовую среду, похожую на боевую
- Развернуть метрики и трассировки на тестовой среде
- Запустить нагрузочные сценарии с “реальными” нагрузками
- Собрать и визуализировать метрики
- Сравнить с эталонной историей и выявить отклонения

## 🔗 Шаг 14:

### Миссия 14

Расположите шаги построения отчёта по оптимизации многопоточного приложения:

1. Составить список экспериментальных сценариев
2. Собрать метрики до и после оптимизаций
3. Проанализировать отклонения и закономерности
4. Подготовить визуализации (графики, диаграммы)
5. Сформулировать рекомендации и дальнейшие шаги

🔗 **Подсказка: Первое — экспериментальные сценарии, затем сбор, анализ, визуализация, рекомендации.**

**Ответ:**

- Составить список экспериментальных сценариев
- Собрать метрики до и после оптимизаций
- Проанализировать отклонения и закономерности
- Подготовить визуализации (графики, диаграммы)
- Сформулировать рекомендации и дальнейшие шаги

## 🔗 Шаг 15:

### Миссия 15

Анна столкнулась с проблемой: её многопоточное приложение часто блокируется на конкурирующих ресурсах. Какой из подходов поможет минимизировать блокировки и повысить производительность?

Подсказка: Подумайте о способах уменьшения времени удержания блокировок и использовании более лёгких механизмов синхронизации.

**Ответ:**

- **Использовать lock-free алгоритмы и структуры данных**
- Увеличить количество потоков без изменений в синхронизации
- Добавить больше synchronized блоков вокруг всего кода
- Игнорировать блокировки и надеяться на лучшее

## 🔗 Шаг 16:

### Миссия 16

Иван оптимизирует многопоточный код и сталкивается с разными видами синхронизации. Сопоставьте типы блокировок с их характеристиками.

Подсказка: Обратите внимание, где возможна повторная блокировка, а где — конкуренция потоков.

Типы блокировок:

**Ответ:**

- ReentrantLock
- Synchronized
- ReadWriteLock
- SpinLock

Характеристики:

1. Позволяет многим потокам читать, но эксклюзивно блокирует запись

2. Поддерживает повторное захватывание блокировки одним потоком
3. Использует busy-waiting, не отдавая управление ОС
4. Встроенный механизм JVM, проще в использовании

- **ReentrantLock** — Поддерживает повторное захватывание блокировки одним потоком
- **Synchronized** — Встроенный механизм JVM, проще в использовании
- **ReadWriteLock** — Позволяет многим потокам читать, но эксклюзивно блокирует запись
- **SpinLock** — Использует busy-waiting, не отдавая управление ОС